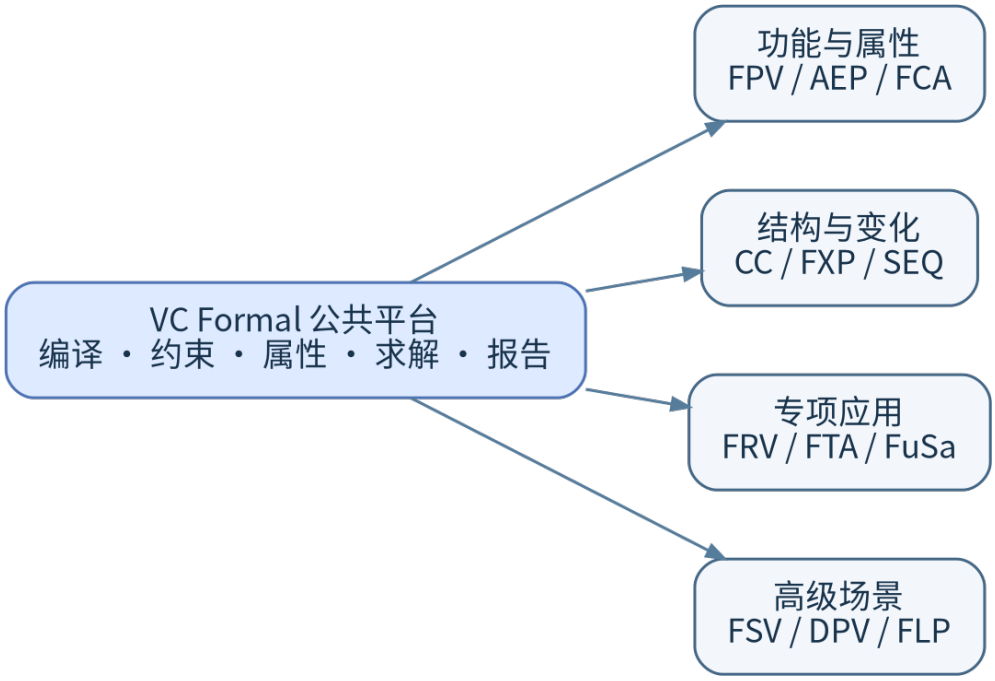
VC Formal 中文形式验证手册

源码内嵌修订版

形式验证原理 · SystemVerilog Assertions · 12 类应用



文档基线：X-2025.06-SP1

2026 年 10 月 · 手册修订 1.1

非 Synopsys 官方译本；命令与行为以安装版本为准

目录

VC Formal 中文形式验证手册｜总目录与阅读指南 **1**
版权及使用说明 1
1. 手册结构与学习顺序 2
2. 如何直接阅读和复制代码 3
3. 阅读本手册需要避免的四种误解 4
4. 版本勘误与高风险参数 4
5. 推荐的正式工程检查点 5

形式验证：从数学原理到芯片工程 **6**
1. 为什么需要形式验证 6
2. 有限状态模型：形式验证究竟检查什么 6
3. 工具如何找到反例或完成证明 7
4. “属性”“约束”“覆盖目标”并不相同 8
5. 结果的工程含义 9
6. 与仿真、静态检查、ATPG、等价检查的区别 9
7. 一个完整的思考例子：仲裁器 10
8. 从规格到验证的推荐过程 10
9. 初学者常见误解 11
10. 本手册学习索引与参考依据 11

VC Formal 平台、环境与命令执行 **12**
1. 平台定位与基本结构 12
2. 应用名称与 Tcl 模式的区别 13
3. 环境准备 13
4. Tcl 外壳与帮助系统 16
5. 读入、编译与展开 RTL 16
6. 时钟与复位定义 17
7. 检查准备状态 17
8. 属性管理、运行、输出 18
9. 计算资源与回归 18
10. 会话、图形调试与工作区 18
11. 命令出现错误时的检查顺序 19

12. 官方资料索引 19

形式验证工程组织与结果判读 20

1. 从设计需求建立验证对象 20
2. 推荐项目目录 20
3. 完整执行顺序 21
4. 读懂反例：逐周期定位 21
5. 证明状态与报告编写 22
6. 时钟、复位与环境建模审查 22
7. 小型设计推荐的脚本骨架 22
8. 状态回归和证明性能 25
9. 验收条件建议 25
10. 参考 25

SystemVerilog Assertions（SVA）完整中文教程 26

1. SVA 解决什么问题 26
2. 先理解三种角色 26
3. 并发断言的时钟、采样与失效条件 27
4. 重叠蕴含与非重叠蕴含 28
5. sequence 和 property 的职责 28
6. 序列连接与延迟运算符 28
7. 序列的组合 29
8. 属性运算符：时间上的持续和终止 30
9. 常用采样函数 31
10. 立即断言和并发断言的区别 32
11. 属性局部变量、`let` 和参数化 32
12. `bind`：无需改动设计源码注入 SVA 33
13. 常见系统级断言模板 34
14. 空洞性、覆盖目标与不恰当假设 35
15. 容易写错的规则与修正 35
16. 多时钟属性、X 值与复杂语法的限制 36
17. 完整示例：有界计数器及 VC Formal 工程 36
18. 工程练习（建议自行运行与审查） 38
19. 语法检索表 39
20. 参考与责任界定 39

FPV：功能属性验证应用完整指南 40

1. 定位、能力与不适用范围 40
2. 标准流程与输入输出 40
3. 单时钟计数器：完整 RTL、SVA 与运行流程 41
4. 如何组织断言 44
5. 常用 Tcl 命令逐项讲解 45

6. 四个逐步进阶的用例 . . . 46
7. 如何分析结果 . . . 49
8. 优化证明效率而不损害结论 . . . 49
9. 常见故障与排查 . . . 49
10. 进阶主题 . . . 50
11. 参考定位 . . . 50

AEP：自动提取属性应用完整指南 **51**
1. AEP 的角色与应用价值 . . . 51
2. AEP 的典型检查项目 . . . 51
3. 初学者第一个 AEP 案例：数组越界 . . . 52
4. 标准运行步骤 . . . 53
5. 状态机检查：死锁、活锁与公平性 . . . 54
6. AEP 的命令作用与注意事项 . . . 54
7. 常见问题与处理方案 . . . 55
8. 推荐的 AEP + FPV 双阶段工作方法 . . . 55
9. 官方参考 . . . 55

FXP：形式未知值传播验证完整指南 **56**
1. 为什么未知值传播值得单独验证 . . . 56
2. 注入点与观察点的概念 . . . 56
3. FXP 自动生成检查的类型 . . . 57
4. 标准脚本与运行顺序 . . . 57
5. 自定义 X 注入、去除与断言 . . . 58
6. 教学用例 A：未复位的数据寄存器 . . . 58
7. 教学用例 B：显式 X 赋值与越界 . . . 60
8. 教学用例 C：有意屏蔽不确定数据 . . . 61
9. 调试：由下游症状回溯到上游来源 . . . 61
10. 设计规模很大时怎么做 . . . 61
11. 常见问题 . . . 61
12. FXP 与 AEP、FPV 的分工 . . . 62
13. 官方依据与版本提醒 . . . 62

CC：形式连通性检查完整指南 **63**
1. 这项应用解决的问题 . . . 63
2. 连通性规格包含哪些信息 . . . 63
3. 最小实例：多路选择器 . . . 64
4. CSV 导入：大规模 SoC 的推荐方法 . . . 65
5. 源目的别名与设计层级 . . . 66
6. 专属命令与报告 . . . 66
7. 三种常见用例 . . . 66
8. 失败分类与排查 . . . 67

9. 对大型项目的建议 . . . 67
10. 参考 . . . 67

FCA：形式覆盖率分析完整指南 68
1. FCA 与 FPV 的区别 . . . 68
2. 支持的主要覆盖对象 . . . 68
3. 第一个 FCA 用例：输入条件与代码分支可达性 . . . 69
4. 主要流程：完整 Tcl . . . 69
5. 从仿真数据库开始 . . . 70
6. 命令分类 . . . 70
7. Auto Scale 及资源策略 . . . 71
8. 排除文件需要审查 . . . 71
9. 常见排查 . . . 71
10. 命名差异和官方参考 . . . 72

SEQ：顺序等价检查完整指南 73
1. 为什么不能只比较组合逻辑 . . . 73
2. 初学者例子：两种计数器写法 . . . 73
3. 编译两个独立设计库并运行 . . . 74
4. 对应关系是什么 . . . 76
5. 顺序优化类型与对应关系 . . . 76
6. 主流程命令说明 . . . 76
7. 如何解释不等价结果 . . . 77
8. 高级优化策略 . . . 77
9. 教学练习 . . . 77
10. 参考 . . . 77

FRV：形式寄存器验证完整指南 78
1. 典型目标和输入资料 . . . 78
2. 核心概念 . . . 78
3. 寄存器规格与 RTL 的对照 . . . 79
4. 一个最小脚本模板 . . . 82
5. `frv_load` 与检查器配置 . . . 83
6. 结果分析的方法 . . . 83
7. 与 UVM RAL、仿真和 FPGA/SoC 寄存器校验的配合 . . . 84
8. 工程清单 . . . 84
9. 参考资料 . . . 84

FTA：形式测试平台分析完整指南 85
1. FTA 解决的不是 RTL 正确性，而是属性环境的质量 . . . 85
2. 工作过程 . . . 85
3. 不能直接从 `-fmode FTA` 启动 . . . 86

4. 示范 Tcl：多阶段流程的轮廓 . . . 86
5. 故障分类与结果含义 . . . 87
6. 关键命令及用途 . . . 87
7. 两个评估示例 . . . 87
8. FTA 与故障仿真的区别 . . . 91
9. 参考 . . . 91

FuSa：功能安全形式分析完整指南 92
1. 场景与关键概念 . . . 92
2. 前置要求和故障输入 . . . 92
3. 四类分析 . . . 92
4. 标准脚本结构 . . . 93
5. 逐步提高分析精度 . . . 94
6. 核心命令说明 . . . 94
7. 案例：带奇偶校验的安全寄存器 . . . 94
8. 与 ISO 26262 / FMEDA 的关系 . . . 97
9. 常见错误 . . . 97
10. 官方资料 . . . 97

FSV：形式安全信息流验证完整指南 98
1. 目标：检查本不应出现的信息传播 . . . 98
2. 什么是信息流与非干扰 . . . 98
3. 典型命令结构 . . . 99
4. 可控的允许传播 . . . 99
5. 案例：保护调试输出的完整工程 . . . 100
6. 信息流安全与普通功能断言相互补充 . . . 102
7. 黑盒与抽象的风险 . . . 102
8. 重要命令 . . . 102
9. 常见错误及处理 . . . 102
10. 官方参考 . . . 103

DPV：数据路径验证与 C/C++—RTL 事务等价完整指南 104
1. 解决的问题与 SEQ 的区别 . . . 104
2. 典型应用示例：8 位加法器 . . . 104
3. C++ 事务接口的结构 . . . 105
4. 创建双设计环境 . . . 106
5. 通过阶段对应建立事务关系 . . . 107
6. 证明与报告 . . . 110
7. 完整可验证事务的六个核对项 . . . 110
8. 常用命令及作用 . . . 111
9. 数据路径证明困难时的策略 . . . 111
10. 官方参考 . . . 111

FLP / LPC：低功耗连通性形式验证完整指南 **112**
1. 核心问题 . . . 112
2. 重要对象 . . . 112
3. 该版本的工作模式 . . . 113
4. 最小系统案例 . . . 114
5. UPF 可信性与 PMC . . . 116
6. FLP CC 和普通 CC 的重要区别 . . . 116
7. 主要命令与用途 . . . 116
8. 三个需要分别检查的场景 . . . 116
9. 调试方法 . . . 117
10. 参考 . . . 117

反例调试、证明性能与复杂性控制 **118**
1. 为什么形式验证会困难 . . . 118
2. 报告状态决定不同的调试路径 . . . 119
3. 失败属性：逐周期回溯 . . . 119
4. 约束检查：最应优先防止的错误 . . . 119
5. 查找状态空间问题 . . . 119
6. 改善证明性能的合理顺序 . . . 120
7. `fvtask`：按性质组织工作 . . . 120
8. 抽象及裁剪驱动器 . . . 120
9. 困难证明的辅助分析 . . . 120
10. Proof Architect：组合式证明思路 . . . 121
11. 有规律地建立证明性能实验 . . . 121
12. 官方参考 . . . 121

覆盖率评估、验证质量与结果验收 **122**
1. 为什么不能只报告“全部属性通过” . . . 122
2. 建立规格—属性对应表 . . . 122
3. 验收需要保留哪些文件 . . . 123
4. 对“Proven”的独立复核 . . . 123
5. 不完成或豁免的处理要求 . . . 123
6. 组合使用 FCA 与 FTA . . . 123
7. 质量评审问题 . . . 124
8. 自动化结果摘要 . . . 124
9. 参考 . . . 124

术语表、应用选择与官方资料定位 **125**
1. 术语表 . . . 125
2. 遇到问题先选哪个应用 . . . 126
3. 官方 UG 章节与页码 . . . 126
4. 确认过的重要版本差异 . . . 128

5. 文档范围与版权说明 128

VC Formal 快速参考指南（中文整理版） 129

1. VC Formal 简介 129
2. FPV：功能属性验证应用 130
3. AEP：自动提取属性应用 136
4. CC：连通性检查应用 138
5. FCA：形式覆盖率分析应用 140
6. FXP：形式 X 传播验证应用 142
7. SEQ：顺序等价检查应用 144
8. FRV：形式寄存器验证应用 146
9. FTA：形式测试平台分析应用 148
10. FuSa：功能安全应用 150
11. FSV：形式安全验证应用 152
12. DPV：数据路径验证应用 154
13. LPC：低功耗连通性检查应用 156
14. 一页版速查 157

VC Formal 中文形式验证手册｜总目录与阅读指南

VC Formal 形式验证原理、工程流程与专用应用中文教程
文档基线： Synopsys VC Formal X-2025.06-SP1
本手册修订： 2026-10-17 · 1.1 源码内嵌修订版
内容形式：分册 Markdown + PDF + 离线 HTML；教程正文直接列出完整示例代码

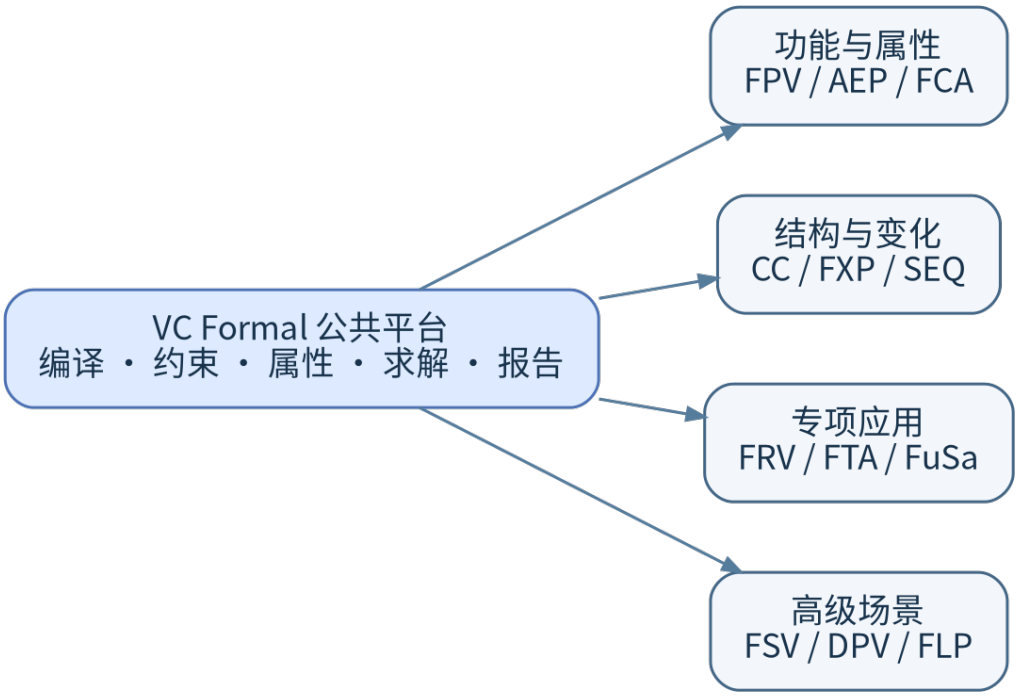


图 1: VC Formal 方法总览

版权及使用说明

本手册根据使用者提供的 **VC Formal User Guide**（X-2025.06-SP1，2025 年 9 月，1,168 页）及 **VC Formal Quick Reference Guide**（同版本，34 页）独立编写，供中文工程教学与实施参考，不是 Synopsys 官方译本，也不代替其产品文档、许可证约定及支持意见。保留英文命令、选项、报错与 SystemVerilog 关键字，以便与工具帮助直接比对。示意图由本手册重新绘制；参考 PDF 本身未随交付物复制。

重要：当前制作环境未安装并授权运行 VC Formal。示例已按所提供的手册比对写法，并进行静态

资源、路径及格式检查；**没有实际的形式证明结果，也不宣称所有示例已经通过工具编译**。随文出现的 `Proven`、`Falsified`、`Inconclusive` 等词用于解释可能出现的结果，不代表这里生成了对应报告。

1. 手册结构与学习顺序

顺序	文档	主要内容	适合读者
第 1 册	形式验证.md	形式验证起源、状态空间、可达性、归纳证明、抽象、反例、覆盖	所有人
第 2 册	VC_Formal_ 平台与环境.md	产品应用、Tcl 模式、编译、许可证、时钟复位、求解运行	入门与工具使用者
第 3 册	验证工程与结果判读.md	项目目录、规格拆解、假设审计、分层证明、属性状态解读	验证负责人
第 4 册	SVA 语法合集.md	IEEE 1800 断言基础、时序运算符、采样函数、典型硬件规则	RTL 与验证工程师
第 5 册	FPV.md	功能属性验证、断言编写、环境约束、反例分析	FPV 使用者
第 6 册	AEP.md	自动提取数组、算术、FSM 等检查及其限制	RTL 开发与质量检查
第 7 册	FXP.md	X 来源、传播、初始化、观察点与失效分析	RTL/SoC 集成
第 8 册	CC.md	连通性检查、端点、条件选择、CSV 接口	SoC 集成
第 9 册	FCA.md	可达性、形式覆盖分析、仿真覆盖补充	覆盖率负责人
第 10 册	SEQ.md	顺序等价、规格实现两侧、状态关系与复位	RTL 优化与 ECO
第 11 册	FRV.md	寄存器行为、总线协议、RALF/IP-XACT	CSR 设计人员
第 12 册	FTA.md	故障注入、检测能力、测试环境约束	故障分析人员
第 13 册	FuSa.md	功能安全、故障诊断、可测性与评价结果	安全芯片设计
第 14 册	FSV.md	硬件信息流安全、机密信息与观察点	安全验证
第 15 册	DPV.md	数据路径计算与 C/C++/RTL 对照	数字信号处理、算法 RTL

顺序	文档	主要内容	适合读者
第 16 册	FLP_LPC.md	UPF、电源域、低功耗连通性与条件	低功耗设计
第 17 册	调试与证明性能.md	反例调试、假设检查、状态爆炸、任务拆分与资源使用	所有人
第 18 册	覆盖与验证验收.md	覆盖计划、可达性检查、结果分类、回归验收	技术负责人
第 19 册	术语表与资料索引.md	中英术语、资料出处、命令索引与阅读指引	所有人
附册	VC_Formal_Quick_Reference_Guide_中文整理版.md	按命令查找、变量与版本勘误	已有经验者

路线 A：第一次使用形式验证

推荐按第 1→2→3→4→5→17→18 册学习。先理解“属性成立”的条件，再建立单时钟单复位的练习环境。不要从运行一条 `check_fv` 命令直接跳到完整 SoC。

路线 B：已有 UVM / 仿真经验的 RTL 工程师

先阅读第 4 册中采样事件、`|->`、`|=>`、`$past`、`disable iff` 等章节，再做 FPV 和 FXP。将现有仿真检查转换为形式属性时，首先明确行为必须被保证还是由环境保证。

路线 C：SoC 集成、低功耗与寄存器

先阅读第 2 ~ 3 册，再选择 CC、FRV 和 FLP 专题，并在第 17 册学习如何排除错误的输入约束、黑盒或复位设置。

路线 D：形式验证项目负责人

重点阅读第 1、3、17、18 册，建立可追溯的规格条目、属性表、约束审查记录、回归基线及验收报告。

2. 如何直接阅读和复制代码

源码在教程文章内，不再只给文件路径。16 篇工具及工程章节包含 81 处完整源码框，覆盖 58 份互不重复的源码与配置文件：22 个 SystemVerilog 文件、21 个 Tcl 文件、12 个编译列表、1 个 CSV 和 2 个 C++ 文件。多个章节复用同一案例时，为保证单独阅读而重复列出必要代码。

每段完整代码先标明“保存为”，接着给出源码正文；文件列表、绑定文件、运行脚本以及执行命令也在本章。复制代码框内容时不包含显示行号，不需要打开额外附件。独立工程文件仍保留，只是

方便使用编辑器修改的副本。

阅读 合订本.md 可搜索全部文字与源码；PDF 合订本 适合连续阅读和打印；离线 HTML 内嵌插图与样式，可在浏览器直接打开，不需要网络或外部图片目录。

完整小型工程与专用流程模板分别标注。FRV、FuSa、DPV、FLP 等仍需要实际寄存器规格、故障数据、HECTOR 包装或 UPF 的流程，不会因为模板全文列出就被称为即用工程。专章新增的简化 RTL/SVA 练习也明确标为配套 FPV 示例，不冒充原生专用应用的验证结果。

15 幅插图继续提供 SVG、PNG 和可编辑 DOT。**附录 _ 修订前笔记/** 保存原笔记，仅供历史对照，不属于本修订版的活动教程；其中旧排版不作为当前正文示例。

3. 阅读本手册需要避免的四种误解

误解一：“Proven 就是整个设计已经正确。” 不对。只说明给定属性在给定模型与假设下被证明，无法替代遗漏的功能规格。

误解二：“无反例就证明成功。” 不对。超时、未知、取消、资源耗尽等结果都不能当成证明通过。

误解三：“自动提取断言覆盖全部功能。” 不对。AEP 擅长设计规则类问题，系统级功能意图仍需人工定义规格与属性。

误解四：“同一份命令在所有 X-2025.06-SP1 资料中一致。” 不完全成立。两份参考资料存在少数名称或参数不一致的情况，下文专门列出。

4. 版本勘误与高风险参数

位置	发现	本手册处理
FCA 应用	Quick Reference Guide 的实例写 `set_fml_appmode FCA`，而 User Guide 第 65、395、430 页的示例写 `set_fml_appmode COV`	中文专章优先给出 **`COV`**；同时要求查看本地帮助与 `get_fml_appmode`
FTA 启动	User Guide 第 65 页提示不能用 `vcf -fmode FTA` 直接启动	在会话里用 `set_fml_appmode FTA`
FRV 的 RALF 使用	User Guide 第 630 页建议在 `frv_load -ral` 中给出 `-top`，快速指南示例未列出；该版 UG 对 `-parameter(s)` 也存在写法差异	使用与地址块对应的 `-top`；通过本地 `frv_load -help` 确认参数拼写
DPV 新流程开关	Quick Reference Guide 与 User Guide 在 Hector 新流程变量的写法上不一致	同时陈列两种资料中的原写法，要求以安装版本为准

位置	发现	本手册处理
FLP CC	User Guide 第 1041 页明确表示不支持 delayed connectivity checks	不把普通 CC 延迟连接规则直接复制到 FLP 模式

5. 推荐的正式工程检查点

开始运行之前：记录工具版本、license、源代码提交、顶层、约束、属性列表和初始化状态；运行期间：记录求解时间、资源预算、属性状态；运行结束后：逐条复核反例、超时、不可达覆盖以及可疑的“全过”结果。审核者需要能够在同版本下重新执行并追溯每条结论的前提。

首选入口： 形式验证原理 → 平台与环境 → SVA 完整教程 → FPV 实战；以上工具章节均内嵌完整案例源码。

形式验证：从数学原理到芯片工程

第 1 册 · 基础理论　版本 1.0 · 2026-10-17
适用读者：数字集成电路设计、功能验证、设计可测性与芯片系统验证工程师。
阅读顺序：本册 → VC Formal 平台与环境 → 验证工程与结果判读 → SVA 语法合集 → 各应用手册。

1. 为什么需要形式验证

仿真回答“设计在**本次输入激励**下如何运行”；形式验证尝试回答“对于**满足约束的所有合法输入行为**，设计是否始终符合指定规则”。两者并不互相替代。仿真善于运行长时间系统场景、软件交互和模拟接口；形式验证善于证明局部控制逻辑、协议约束、寄存器访问以及难以用随机激励命中的组合条件。工具搜索到一个能够违反规则的输入与状态序列时，会给出反例；如果建立了有效的数学证明，则给出该属性在所建模型和假设条件下成立的结论。

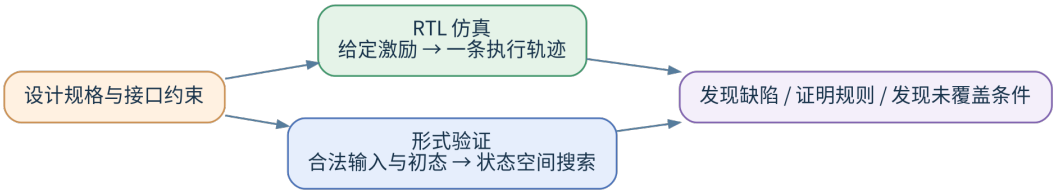


图 2: 形式验证与仿真的关系

典型问题包括：先进先出缓冲区是否发生上溢和下溢；多路仲裁器能否同时授权两个请求方；复位后控制状态是否正确；握手请求能否在指定期限得到响应；中断是否会在清除之后错误保持；重构后的控制逻辑是否与原设计一致；安全数据能否流向公开端口；未知值是否泄漏到关键控制输出。

不适合单独依赖形式验证的问题包括：没有可形式化规格说明的复杂系统行为、第三方模拟器件的模拟特性、未经正确建模的软件运行环境，以及大量没有抽象处理的存储阵列或复杂数据处理导致的计算难题。形式验证的结论取决于模型、约束、属性和工具求解状态。

2. 有限状态模型：形式验证究竟检查什么

把一个同步数字模块抽象为状态转移系统：

$$\mathcal{M} = (S, I, T, S_0), \qquad s_{t+1} = T(s_t, i_t)$$

其中，S 为所有寄存器状态的集合，I 为可用输入，T 为当前状态与输入决定下一状态的逻辑，S_0 为**允许的初始状态**。一个安全性属性可以写成：

$$\forall t \geq 0,\ \forall (s_0, i_0, \ldots, i_t) \in \text{LegalTraces}, \quad P(s_t, i_t) = 1.$$

这里的 `LegalTraces` 极其重要，它由复位初始条件、输入协议假设、时钟关系和外部环境模型共同限定。属性被证明，不代表“在任何想象中的物理条件下都正确”，只代表**在被建模的所有合法执行轨迹中**成立。如果约束不合理地排除了错误场景，证明结果就可能失去工程价值。

2.1 安全性与活性

类别	要表达的规则	示例	常见求解关注点
安全性（Safety）	坏事情永不发生	缓冲区满时不得额外写入	是否存在有限长度反例
有界响应	事件必须在有限窗口响应	请求发生后 1 ~ 3 周期出现应答	计数范围、复位与连续请求
活性（Liveness）	好事情终将发生	请求最终被服务	公平性、无限执行、死锁和活锁
可达性（Reachability）	某状态是否能够到达	仲裁状态是否被访问	覆盖证明或可达见证
等价性（Equivalence）	两个系统的可观测行为一致	改写后的设计与参考设计一致	输入、状态与延迟关系

有界响应往往容易写成安全性检查；无限期“最终应答”涉及环境是否持续提供条件。例如，如果从设备永远可以拒绝服务，那么主设备无法保证最终完成。不要用无法由本模块控制的假设来证明本模块“有活性”。

2.2 关于初始状态与复位

未经初始化的寄存器在形式模型中可能代表任意初始取值。`create_reset` 用于声明复位信号及其极性，`sim_run`、`sim_save_reset` 等命令用于建立与保存可供证明使用的初始化状态，但必须确认复位真正驱动到目标寄存器，而不能假定声明命令自然令所有寄存器变为零。复位后的第一拍、同步/异步复位、部分复位寄存器需要区别处理。参见《VC Formal User Guide》§ 4.11 ~ 4.13（约第 132 ~ 154 页）。

3. 工具如何找到反例或完成证明

3.1 可满足性求解与有界模型检查

对于最大展开深度 k，可构造：

$$S_0(s_0) \wedge \bigwedge_{t=0}^{k-1} T(s_t, i_t, s_{t+1}) \wedge \bigvee_{t=0}^{k} \neg P(s_t, i_t).$$

若可满足，满足赋值对应一条反例；若不可满足，仅能表明在该展开深度内未找到违反，**不能由此推出无限深度已证明**。有界模型检查（Bounded Model Checking，BMC）常适合较短缺陷路径。

3.2 数学归纳与增强不变量

归纳证明要求初始时刻性质成立，并证明“假设性质对当前状态成立，则对下一状态也成立”。简单归纳可能因为包含不可达状态而失败，这时可引入更强的不变量（例如缓冲区占用计数与读写指针之间的关系）。归纳深度增加不必然解决证明困难；关键在于性质与状态约束是否充分。

3.3 属性引导可达性与其他证明引擎

工业工具可结合 SAT/SMT、归纳、插值、PDR/IC3、抽象及问题分解等多种方法；不同属性和设计结构的性能差异很大。VC Formal 的实际引擎组合与配置由软件版本、应用模式、许可证和资源控制决定，不应把某个求解方法当作该工具唯一的实现。

3.4 抽象、分解与可能的误报

只保留某个属性关心的逻辑集合可降低求解规模，这被称为影响锥（Cone of Influence，COI）约简。进一步将乘法器、存储器或内部网络抽象成不完全受原逻辑约束的变量，通常形成更宽泛的行为集合，因此可能产生原始设计并不可能发生的反例。遇到这种反例应检查被抽象的信号与约束，而不是直接修改 RTL。



图 3: 从设计与属性到证明结论

4. “属性”“约束”“覆盖目标”并不相同

- **断言（`assert`）**：设计应当满足的规则；失败通常意味着 RTL、规格、约束或属性实现中有问题。

- **假设（assume）**：定义允许的外部行为；它不是 DUT 需要证明的规则。错误假设会掩盖缺陷。
- **覆盖（cover）**：要求搜索一个能够到达的事件或状态；它用于确认目标场景可以发生，而不是要求每个时刻都发生。

例如，计数器输入 `inc` 为外部激励，模型可假设 `inc` 不与某非法操作同时有效；但不能假设 `count < 8` 来“证明”计数器不会溢出，因为这相当于假设了正要验证的结论。

4.1 约束不能替代验证

合理：`assume property (@(posedge clk) disable iff(!rst_n) req |-> valid_input);`，前提是外部协议确实要求请求时数据有效。危险：`assume property (@(posedge clk) count <= MAX);`，如果 `count` 是 DUT 的内部状态。更危险：假设 `req==0`，然后证明所有请求应答断言，因为它们将永远不会触发。

4.2 空洞性与可达性检查

对于 `a |-> b`，如果 `a` 永远不成立，断言可能被判定成立。这叫做**空洞成立**（vacuity）。针对关键规则至少应同时添加 `cover property (a)`，必要时再检查 `cover property (a ##1 b)`。工具的空洞检查不是对完整规格的自动理解，人工仍需审查触发条件。

5. 结果的工程含义

状态	能得出的结论	不能得出的结论	后续动作
Proven	对当前模型、初始状态和假设已经证明	系统级规格自动正确	审查属性范围、前提和空洞性
Falsified	存在可触发该属性失败的候选反例	一定存在芯片功能缺陷	检查反例、假设、复位和抽象
Inconclusive	在当前资源、策略或时间内尚无明确结论	该属性正确或错误	缩小问题、调优、延长资源
Covered	找到了满足覆盖目标的路径	设计全部功能受到验证	检查路径是否真实且有意义
Uncoverable	在当前模型和约束下不可到达	目标在真实系统里也不可到达	检查约束与复位、用例定义
Not run / Disabled	属性未被验证	属性已通过验证	检查启用列表和任务配置

状态名称依应用、版本及报告格式略有变化。不能把“检查结束”、“没有失败条目”、“达到某个深度”直接写成“全部已证明”。工具输出还可能包括 `checking`、`unknown`、`waived` 等管理状态。参见 User Guide §7.1（第 247 页起）。

6. 与仿真、静态检查、ATPG、等价检查的区别

技术	主要目标	典型输入	典型输出
RTL 仿真	在给定激励下观察行为	Testbench、激励、RTL	波形、断言失败、覆盖率
规则检查/静态检查	查找结构与编码问题	RTL、规则集	报错或告警
属性形式验证	证明逻辑规则或找反例	RTL、SVA、形式约束	证明、反例、未完成
等价检查	比较两个版本	参考与实现设计、对应关系	等价或差异证据
ATPG	为制造缺陷模型生成测试向量	网表、故障模型、扫描结构	测试图样、故障覆盖率
MBIST	在存储器上运行算法发现制造缺陷	存储宏、控制器、March 测试	测试结果、故障定位信息

特别说明：形式验证与 ATPG 都使用逻辑求解技术，但其目标并不相同。RTL 断言证明不能替代制造测试；ATPG 高故障覆盖率也不能证明功能协议正确。

7. 一个完整的思考例子：仲裁器

设计：两个请求 `req[1:0]`，输出授权 `gnt[1:0]`。初步规格：

1. 同一时刻最多有一个授权；
2. 未收到请求的端口不得得到授权；
3. 在持续请求且另一端口有公平竞争条件时，请求不会无限期饥饿；
4. 复位之后授权清零。

最先写的属性通常是：

```
ap_no_double_grant: assert property (
  @(posedge clk) disable iff (!rst_n) $onehot0(gnt));
ap_grant_requires_request: assert property (
  @(posedge clk) disable iff (!rst_n) ((gnt & ~req) == 2'b00));
cp_contention: cover property (
  @(posedge clk) disable iff (!rst_n) req == 2'b11);
```

以上第二条**只适用于同周期请求—授权协议**；如果授权允许滞后一拍，则应与 `$past(req)` 或挂起请求记录相匹配。第三条用于防止所有验证都发生在“根本没有竞争”的场景。公平性属性必须考虑仲裁策略、请求是否持续、时钟是否运行以及端口是否能响应，因此不应脱离协议就写出无条件的 `eventually`。

8. 从规格到验证的推荐过程

1. 阅读接口规格，列出模块能保证的行为、外部必须保证的条件，以及排除的非法使用方式。
2. 把规则分成复位、功能安全性、进展性、交叉接口、异常处理和参数化条件。
3. 每条规则编写可阅读的自然语言解释、SVA 表达式、触发示例与应失败的反例。

4. 建立时钟、复位、初始状态和必要的输入假设，不要提前把内部状态约束成预期结果。
5. 检查属性是否生效，并利用覆盖目标验证关键触发场景可达。
6. 证明安全性；记录未完成的原因；修正设计后回归全部属性。
7. 以版本化的 RTL、SVA、Tcl、日志、结果报告和豁免说明组成可复现的结果包。

详见 验证工程与结果判读。

9. 初学者常见误解

“我有 100% 代码覆盖率，所以所有功能都正确。” 代码覆盖率描述哪些代码结构被执行，不等同于正确性。**“模型只要能跑出 Proven 就代表覆盖完整。”** 属性可能遗漏关键规格或触发条件不可达。**“形式验证不需要 Testbench。”** 虽然不必使用传统仿真激励，仍需通过假设、状态初始化和环境模型定义合法行为。**“加入假设后证明更快一定是好事。”** 如果假设屏蔽关键场景，速度提高反而意味着可信度下降。**“每个未完成都表示工具有问题。”** 更多时候是属性范围、运算规模、状态空间或外部环境模型仍需优化。

10. 本手册学习索引与参考依据

- SVA 系统教材：SVA 语法合集.md。
- 工具共性：VC_Formal_ 平台与环境.md、验证工程与结果判读.md。
- 首次动手：FPV.md、案例工程/README.md。
- 实战进阶：调试与证明性能.md、覆盖与验证验收.md。
- 官方版本参考：用户提供的 VC Formal User Guide, X-2025.06-SP1，§1、§3、§4、§6、§7、§17、§18；以及 VC Formal Quick Reference Guide 同版本。版权属于 Synopsys，本文为独立撰写的中文学习教程，不是官方文件的逐页翻译。

VC Formal 平台、环境与命令执行

源码内嵌版。本章的完整案例以“保存为”标明文件路径，并在下方直接列出全文。代码框不带显示行号，可直接复制；**案例工程/** 只表示目录结构，不要求另找源码附件。包含占位符或保护性 `error` 的专用流程另行明确标为模板。

第 2 册 · 工具基础　依据 VC Formal X-2025.06-SP1 用户指南 §1 ~ 4、§8 及同版本快速参考指南。本文所有 `$` 表示操作系统终端，`vcf>` 表示 VC Formal Tcl 提示符；示例中的文件、层级和信号必须按工程替换。

1. 平台定位与基本结构

VC Formal 是 Synopsys 的形式验证平台，以共同的编译、展开、属性管理、求解、报告及调试环境提供不同应用。它支持以传统 RTL 与约束出发的功能属性验证，也支持自动提取问题、基于结构连接列表的检查、顺序等价分析、安全性分析等专用任务。

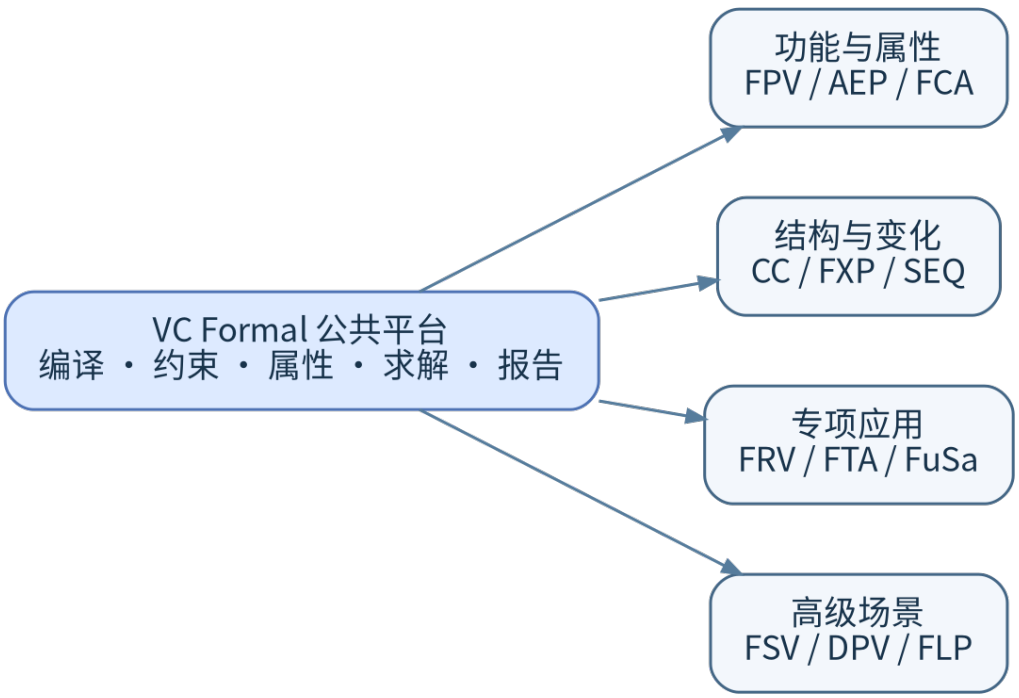


图 4: VC Formal 应用布局

一个验证任务一般由六个部分构成：RTL 或参考模型；SVA/CSV/寄存器描述/故障模型等规格输入；时钟复位与环境假设；证明引擎与资源策略；反例及结果报告；必要时的 Verdi 图形调试。不同应用共享 Tcl 外壳，但**应用模式、专业命令与输入数据类型并不统一**。

2. 应用名称与 Tcl 模式的区别

中文用途	文档中的应用名称	Tcl 模式（优先以 User Guide 为准）	专用资料
功能属性证明	FPV	`FPV`	FPV.md
自动提取设计检查	AEP	`AEP`	AEP.md
连通性	CC	`CC`	CC.md
形式覆盖率分析	FCA	**`COV`**	FCA.md
未知值传播	FXP	`FXP`	FXP.md
顺序等价	SEQ	`SEQ`	SEQ.md
寄存器验证	FRV	`FRV`	FRV.md
测试平台分析	FTA	`FTA`（通常由 FPV 会话切换）	FTA.md
功能安全	FuSa	`FUSA`	FuSa.md
信息流安全	FSV	`FSV`	FSV.md
数据路径对照	DPV	`DPV`	DPV.md
低功耗连通性	FLP/LPC	`CC` + 低功耗命令（该使用流程）	FLP_LPC.md

经过源文件对照的重要更正：2025 年 9 月快速指南的 FCA 命令行示例使用 `set_fml_appmode FCA`，但同版本 1,168 页 User Guide 在第 65、395、430 页明确展示 `COV`。本教程的 FCA 脚本以 `COV` 为首选，启动后通过 `get_fml_appmode` 确认。FTA 模式在 User Guide 第 65 页明确指出不应直接使用 `vcf -fmode FTA` 启动，而应在 VC Formal 会话内用 `set_fml_appmode FTA`。这两处必须与实际安装版本核对。

3. 环境准备

正式环境通常需要 Linux 主机、已安装的 VC Formal / VC Static 相关组件、相应许可证，以及可用的 Verdi（仅在图形调试需要时）。具体支持的平台、许可证名称、最低内存与批量计算系统应以单位内部安装说明及实际版本的安装指南为准，不能从本文推断许可证一定已开通。

首先在终端确认：

```
which vcf
vcf -help
pwd
```

工具可以从 `-f` 读取 Tcl 脚本。推荐从**项目根目录**执行，以使源文件列表和日志位置一致：

```
vcf -f scripts/run_fpv.tcl -batch
```

```
vcf -f scripts/run_fpv.tcl -verdi
vcf -f scripts/run_fpv.tcl
```

典型授权选项示例：`-fml_elite`、`-fml_token`；只有在内部许可证策略支持且确实有需要时再使用。实际提示和参数通过 `vcf -help` 检查，不要把 shell 指令原样放入 Tcl 脚本。

3.1 首次运行所需的全部文件

环境准备后，可以先保存以下最小工程。每个文件的完整内容都在本节；不必先下载或寻找独立案例附件。后续章节再解释这些命令和属性的细节。

3.1.1 被验证的计数器 RTL

规格：复位后计数为零；使能时从 0 计到 9 后回到 0；没有使能时保持。RTL 文件只包含设计，不把待验证结论写成环境假设。

保存为 案例工程/rtl/counter.sv：

```
module counter (
    input  logic       clk,
    input  logic       rst_n,
    input  logic       enable,
    output logic [3:0] count
);
  always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n)             count <= 4'd0;
    else if (enable) begin
      if (count == 4'd9)    count <= 4'd0;
      else                  count <= count + 4'd1;
    end
  end
endmodule
```

3.1.2 完整断言与绑定文件

`history_valid` 在复位后建立历史有效标志。`ap_range` 检查范围，`ap_hold` 检查保持，`ap_wrap` 检查回绕；两条 `cover` 检查最高计数值及回绕场景是否能够出现。末尾 `bind` 将检查器接入同名设计信号，不需要修改 RTL。

保存为 案例工程/formal/counter_props.sv：

```
module counter_props (
    input logic       clk, rst_n, enable,
    input logic [3:0] count
);
  logic history_valid;
  always_ff @(posedge clk or negedge rst_n)
    if (!rst_n) history_valid <= 1'b0;
    else        history_valid <= 1'b1;
```

```
  ap_range: assert property (@(posedge clk) disable iff (!rst_n)
      count <= 4'd9);

  ap_hold: assert property (@(posedge clk) disable iff (!rst_n)
      history_valid && $past(!enable) |-> count == $past(count));

  ap_wrap: assert property (@(posedge clk) disable iff (!rst_n)
      history_valid && $past(enable && count == 4'd9)
      |-> count == 4'd0);

  cp_reaches_nine: cover property (@(posedge clk) disable iff (!rst_n)
      count == 4'd9);
  cp_wrap: cover property (@(posedge clk) disable iff (!rst_n)
      count == 4'd9 ##1 count == 4'd0);
endmodule

bind counter counter_props u_counter_props (.*);
```

3.1.3 编译文件列表

先列 RTL，再列包含 bind 的属性文件；路径相对于运行 VC Formal 的 **案例工程/** 目录，而不是 Tcl 文件所在目录。

保存为 案例工程/filelist_counter.f：

```
./rtl/counter.sv
./formal/counter_props.sv
```

3.1.4 完整运行脚本

依次选择应用、读入设计与 SVA、定义时钟复位、保存初始状态、检查设置、列出属性并开始证明。初始化步骤需要在实际安装环境核对复位状态，不能把声明复位等同于已经完成复位。

保存为 案例工程/scripts/fpv_counter.tcl：

```
# Run from the example project root with: vcf -f scripts/fpv_counter.tcl -batch
set_fml_appmode FPV
read_file -top counter -sva -format sverilog -vcs {-f filelist_counter.f}
create_clock clk -period 100
create_reset rst_n -sense low
sim_run -stable
sim_save_reset
check_fv_setup
report_fv_setup -list
report_fv -list
set_fml_var fml_max_time 1H
set_fml_var fml_max_memory 8GB
check_fv -block
report_fv -verbose
```

运行命令（Bash）：

```
# 从包含“案例工程”目录的上一级目录开始。
cd 案例工程
mkdir -p logs
set -o pipefail
vcf -f scripts/fpv_counter.tcl -batch 2>&1 | tee logs/fpv_counter.log
```

这些命令应在本机具备相应 VC Formal 版本和许可证时执行。日志中的属性状态才是证明结论；命令退出、脚本结束或没有打印错误都不能代替属性结果检查。

结果阅读。在初始状态正确的前提下，正确版预期满足这些断言，并存在到达 9 和回绕的覆盖轨迹。这不是本次 VC Formal 的实际运行结果。该属性集合也不是全部计数器规格：例如每次普通递增的精确关系仍可另加属性，不能仅凭范围成立就认定全部功能正确。

4. Tcl 外壳与帮助系统

```
help
help report*
report_fv -help
man report_fv
view_help report_fv
report_fml_var fml_max_*
report_app_var *fml*
get_fml_appmode
```

`help` 与 `-help` 用于核对命令接受的选项；`man`、`view_help` 调出较长帮助；`report_fml_var` 用于查询形式变量，`report_app_var` 用于应用变量。变量归属不能随便互换：有的需要 `set_fml_var`，有的需要 `set_app_var`。**优先执行安装版本的帮助而不是直接复制网络脚本**。

5. 读入、编译与展开 RTL

下列两种是供学习的可选路径，通常选其中一种，而不是把 `read_file` 和 `analyze/elaborate` 在同一工程里无条件重复：

```
# 一步读入并完成展开
read_file -top counter -sva -format sverilog -vcs {-f filelist.f}

# 或者分步编译
analyze -format sverilog -vcs {-f filelist.f}
elaborate counter -sva
```

`read_file` 负责读入源与创建形式设计；`analyze` 处理 HDL 分析；`elaborate` 指定顶层并进行展开；`-sva` 表示需要处理断言；`-vcs` 允许传递 VCS 风格的选项；`-format` 应与 HDL 类型对应。`-f filelist.f` 指向相对路径的设计列表，必须处理宏、包含目录、编译顺序及需要的库。参见 UG §4.2（第 118 页起）。

建议文件列表写为：

```
+incdir+./rtl/include
./rtl/counter.sv
./formal/counter_properties.sv
```

有些示例直接把 bind 文件放在最后一行；当属性用包中的定义或其他模块时应按语言编译要求安排顺序。遇到语法或层级错误，先检查源文件展开成功再分析证明失败。

6. 时钟与复位定义

```
create_clock clk -period 100
create_reset rst_n -sense low
set_constant test_mode -value 1'b0
set_change_at -clock clk -default
```

create_clock 声明逻辑时钟；这里的 -period 100 为形式时钟配置中的周期单位，不应未经说明就解释成目标芯片的物理时间；create_reset 描述复位信号及有效电平；set_constant 建模测试模式等稳定输入；set_change_at 用于约束输入在哪一时钟边沿或时序位置允许变化（特别注意多时钟设计）。这些命令不等同于 STA 中完成时序约束，也不自动描述完整的上电过程。

6.1 建立复位后的初始化状态

```
sim_run -stable
sim_run 2 -clk clk
sim_save_reset
view_trace -reset
```

这是操作顺序示意，不是对任何 DUT 都成立的复位过程。如果复位需要由外部拉高或拉低，必须按真实复位时序添加 sim_force、sim_run 等操作；保存前应使用 sim_get 或可视化追踪检查寄存器值。不要因为示例写了两拍就将所有芯片复位程序固定成两拍。参见 UG §4.13。

6.2 多时钟处理

对每一个独立时钟域分别确定时钟定义、输入变化时机和复位释放规则。不要把异步时钟误设成固定整数倍的同步时钟来获得快速的证明。对于异步握手，接口级假设应来自 CDC 规格，而不是简单假定采样不会遇到亚稳态。形式分析的二值逻辑模型一般不直接证明模拟亚稳态的物理概率。

7. 检查准备状态

```
report_blackbox
report_formal_setup
report_fv_complexity
check_fv_setup
report_fv_setup -list
```

核对无意中的黑盒、未定义实例、时钟/复位检测、设计复杂度及组合反馈等结构问题。check_fv_setup 的结果属于**环境检查**，不等于完成功能证明。过大的黑盒范围可能令断言

看起来更容易，也可能产生不实际的反例。

8. 属性管理、运行、输出

```
# 下面是运行阶段的常用命令
report_fv -list
set_fml_var fml_max_time 2H
set_fml_var fml_max_memory 16GB
check_fv -block
report_fv
report_fv -verbose
```

`fvassert`、`fvassume`、`fvcover` 可以在 Tcl 环境中创建或改变属性角色。优先用 SV 文件管理复杂时序 SVA，便于维护、测试、版本审查。属性状态需要按分组与任务核对；`check_fv` 未指定 `-block` 时可能处于后台运行方式，输出报告时要确认检查完成。运行后的文字报告与反例都应随任务保存。

9. 计算资源与回归

```
set_fml_var fml_effort default
set_fml_var fml_max_time 4H
set_fml_var fml_max_memory 32GB
report_fml_engines
report_fml_jobs
report_fml_hosts
# 如果本机与网格环境支持，可配置调度方式：
# set_grid_usage -type RSH=4
```

`fml_effort` 是求解努力程度的预置策略。更高的资源投入不能保证所有属性最终完成；过度并行也可能受单机内存和许可证数量限制。大型项目推荐按模块、属性类别和运行目的组织任务，并在同等硬件条件下比较优化效果。可选的 `fvlearn_config -local_dir rma_db` 适用于符合条件的回归加速流程，详见 UG §5。

10. 会话、图形调试与工作区

```
save_session -session my_session
restore_session -session my_session
view_trace -property my_assertion
```

工具重启时可采用 `vcf -restore -session my_session`。注意会话文件和相应数据库是否一起备份，设计或约束更改后不应直接将旧结果当作新版本的证明。Verdi 的属性列表、源代码定位、反例波形和结构视图可用于追踪失败属性；对于没有图形环境的服务器可以先保留文字报告再移交调试。

11. 命令出现错误时的检查顺序

1. 命令不识别：先查 `get_fml_appmode`、该应用专属帮助以及版本差异，尤其注意 `COV`/`FCA`。
2. 文件未找到：确认执行目录，检查 `filelist.f` 中的相对路径、`+incdir+`、宏定义和权限。
3. 属性列表为空：检查 `-sva`、`bind` 编译是否生效、顶层、属性是否被禁用。
4. 大量未知初始状态：重新确认复位/初始化保存，避免把初始状态直接强制为期望结果。
5. 反例路径异常：检查外部假设、时钟关系、非法环境输入及黑盒。
6. 显示已证明但无法触发：补充 `cover`、空洞性分析和约束可满足性检查。
7. 证明超时：先缩小验证目标和影响锥，再调整引擎、内存和运行时限。

12. 官方资料索引

- User Guide §2 第 53 ~ 65 页：环境、启动与模式。
- §3 第 66 ~ 114 页：约束、属性、SVA 与属性管理。
- §4 第 115 ~ 170 页：编译、时钟、复位、初始状态及设计检查。
- §5 ~ 8 第 171 ~ 344 页：回归加速、运行控制、结果调试与工作区。
- Quick Reference Guide 第 1 ~ 34 页：各应用最常用命令的快速定位。**快速指南并不是完整命令参考书**。

形式验证工程组织与结果判读

源码内嵌版。本章的完整案例以“保存为”标明文件路径，并在下方直接列出全文。代码框不带显示行号，可直接复制；**案例工程/** 只表示目录结构，不要求另找源码附件。包含占位符或保护性 `error` 的专用流程另行明确标为模板。

第 3 册 · 共通工作流程。本册适用于 VC Formal 的 FPV、AEP、CC、FXP、SEQ 等大多数应用；各应用的专属例程见相应章节。

1. 从设计需求建立验证对象

建议为每条规则建立一行审查记录：**需求编号、自然语言规则、采样时钟、复位条件、触发条件、允许的延迟、输入假设、属性名称、覆盖见证、报告状态、评审人**。自然语言规则应足够具体，使不接触 SVA 的设计工程师也可以判断反例究竟违反了什么。

例如，规则 **REQ-03：接受请求以后最迟在第 3 个时钟上升沿给出应答**，并不意味着 `req` 一高就要出现 `ack`。首先确认接受发生在 `req && ready`，请求是否允许撤回，`ack` 是脉冲还是电平，以及响应期间能否接受第二个请求。只有这些定义明确后才能编写正确的 SVA。

2. 推荐项目目录

```
project/
├── rtl/                    # 待验证设计，不得含隐含形式假设
├── formal/
│   ├── properties.sv       # 断言、假设、覆盖目标
│   ├── bind.sv             # 可选的非侵入式绑定
│   └── waivers/            # 带原因、审核与到期条件的豁免
├── scripts/
│   ├── setup.tcl           # 通用编译、时钟、复位
│   ├── run_fpv.tcl         # 应用专属检查
│   └── filelist.f
├── reports/                # 文字和机器可读报告
├── sessions/               # 会话与可视化调试数据
└── README.md               # 软件版本、运行命令与结论
```

不要把运行日志与 RTL 混在一起，也不要提交可能包含许可证、客户路径或公司机密的整个工具工作目录到公开代码仓库。

3. 完整执行顺序



图 5: 可复现验证流程

3.1 编译通过不代表环境正确

第一步执行 `read_file` 或 `analyze/elaborate`，核对顶层、模块实例、语言、参数值。第二步执行时钟与复位定义，保存初始化状态。第三步执行 `report_blackbox`、`check_fv_setup`，确认没有未预期的黑盒和关键错误。第四步查看属性数量与类型；如果有 20 条预期断言但报告只列出 3 条，必须在求解前调查。

3.2 先验证假设，再验证 DUT

优先运行简单覆盖目标：复位退出、请求出现、握手成功、缓冲区填满、缓冲区清空和错误条件能够触发。若关键事件覆盖目标不可达，先检查假设是否过度限制输入。输入假设应有规格出处；特别禁止为了获得 `Proven` 把 DUT 输出或内部状态直接约束为正确取值。

3.3 先短后长的证明计划

第一次运行用较短时间验证环境完整性与可发现的浅层缺陷；第二次把剩余难属性按功能块分组，按重要性分配时间、内存与计算节点；最终运行固定输入版本并记录工具版本、命令与状态。这样既能早发现工程错误，又能减少在错误环境里长时间求解的成本。

4. 读懂反例：逐周期定位

反例至少应该包含：属性触发周期、导致失败的周期、触发前的设计状态、相关输入变化、reset 状态、接受信号与相关时钟。检查时按顺序回答：

1. 是否处于允许的运行状态？复位是否已结束，起始状态是否合理？
2. 是否违反接口对输入的要求？如果是，应该将其写成环境假设还是 DUT 应处理的非法输入？
3. 失败的准确周期是什么？`|->` 与 `|=>` 是否写反？`$past` 是否在未初始化时使用？
4. 内部状态在此周期之前如何变化？是否存在 NBA 更新与 SVA 采样的时间差？
5. 是否人为加入抽象/黑盒而导致不可行状态？
6. 修改 RTL、修正规格描述还是修改属性？必须区分并留下审查记录。

4.1 反例说明模板

```
属性：AP_FIFO_NO_OVERFLOW
模块版本：rtl_commit=...
实际检查范围：posedge clk; disable iff(!rst_n)
失败位置：反例第 17 周期
```

```
输入：push=1, pop=0, full=1
状态：occupancy=8（DEPTH=8）
期望：不得接受写入，occupancy 保持 8
实际：occupancy 变为 9
处置：修复计数更新条件，并添加满状态覆盖目标
```

上表只是一条**假设性的教学反例**，并非本手册实际运行 VC Formal 得到的结果。

5. 证明状态与报告编写

必须报告属性总数与分布，包括 Proven、Falsified、Inconclusive、Covered、Uncoverable、Disabled、Waived 及其他输出状态。对每条断言写清时钟、复位、有效前提和设计版本。`report_fv -list` 适合总览；`report_fv -verbose` 适合排查；对单条失败使用 `view_trace -property <name>`。`report_fv` 字段随应用和版本有区别，不能把若干种状态强行压成“通过/失败”二选一。

5.1 “Proven”与“覆盖完整”分别检查

强断言 `assert property(req |-> ack)` 被证明，但 `req` 从未出现，其结论仍可能是空洞的。一般需要独立的触发覆盖目标。验证完成至少检查：断言是否被启用、关键 antecedent 可达、是否存在矛盾假设、证明是否包含完整设计范围、所有豁免是否有审查依据。

5.2 为什么未完成不等于失败

`Inconclusive` 仅说明当前求解资源与方法不足；可能是大整数运算、内部存储阵列、长时间依赖、跨多个控制状态或过大影响锥导致。应尝试简化断言、拆分属性、建立合法的不变量和合理抽象，而不应直接增加一个使属性易证但不合理的假设。

6. 时钟、复位与环境建模审查

对象	必须核对的问题	常见错误
主时钟	哪一边沿更新/采样，是否有门控	SVA 在错误边沿采样
异步复位	生效极性和释放关系	`disable iff` 之外仍使用非法 `$past`
多时钟	相位与允许变化时刻	错将异步时钟固定同步
总线数据	`valid && ready` 时何时稳定	把每拍数据都假设不变
存储器	上电初值与读写冲突规则	将未初始化读数据强制为零
黑盒	哪些信号成为任意值	意外屏蔽实现错误
测试端口	正常模式/测试模式的关系	为了便于证明错误强制测试端口

7. 小型设计推荐的脚本骨架

```
# 一条教学用途的 FPV 批量运行示例
set_fml_appmode FPV
```

```
read_file -top counter -sva -format sverilog -vcs {-f filelist.f}
create_clock clk -period 100
create_reset rst_n -sense low
sim_run -stable
sim_save_reset
check_fv_setup
report_fv -list
set_fml_var fml_max_time 1H
check_fv -block
report_fv -list
report_fv -verbose
```

使用条件：DUT 的初始状态在该复位配置下正确建立、路径从执行目录解析、属性确实已被编译；本节随后直接给出完整示例；实际使用时仍应补充适合目标设计的复位激励并核验状态。如果缺少许可证，本示例仅是经文档对照的参考脚本，不是已经执行成功的日志。

7.1 将骨架落实为完整计数器工程

上一段的 `dut/filelist.f` 是通用名称。下面给出实际顶层、文件列表和全部源码，作为结果判读练习的固定输入。

7.1.1 被验证的计数器 RTL

规格：复位后计数为零；使能时从 0 计到 9 后回到 0；没有使能时保持。RTL 文件只包含设计，不把待验证结论写成环境假设。

保存为 案例工程/rtl/counter.sv：

```
module counter (
    input  logic       clk,
    input  logic       rst_n,
    input  logic       enable,
    output logic [3:0] count
);
  always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n)             count <= 4'd0;
    else if (enable) begin
      if (count == 4'd9)    count <= 4'd0;
      else                  count <= count + 4'd1;
    end
  end
endmodule
```

7.1.2 完整断言与绑定文件

`history_valid` 在复位后建立历史有效标志。`ap_range` 检查范围，`ap_hold` 检查保持，`ap_wrap` 检查回绕；两条 `cover` 检查最高计数值及回绕场景是否能够出现。末尾 `bind` 将检查器接入同名设计信号，不需要修改 RTL。

保存为 案例工程/formal/counter_props.sv：

```
module counter_props (
    input logic       clk, rst_n, enable,
    input logic [3:0] count
);
  logic history_valid;
  always_ff @(posedge clk or negedge rst_n)
    if (!rst_n) history_valid <= 1'b0;
    else        history_valid <= 1'b1;

  ap_range: assert property (@(posedge clk) disable iff (!rst_n)
      count <= 4'd9);

  ap_hold: assert property (@(posedge clk) disable iff (!rst_n)
      history_valid && $past(!enable) |-> count == $past(count));

  ap_wrap: assert property (@(posedge clk) disable iff (!rst_n)
      history_valid && $past(enable && count == 4'd9)
      |-> count == 4'd0);

  cp_reaches_nine: cover property (@(posedge clk) disable iff (!rst_n)
      count == 4'd9);
  cp_wrap: cover property (@(posedge clk) disable iff (!rst_n)
      count == 4'd9 ##1 count == 4'd0);
endmodule

bind counter counter_props u_counter_props (.*);
```

7.1.3 编译文件列表

先列 RTL，再列包含 bind 的属性文件；路径相对于运行 VC Formal 的 案例工程/ 目录，而不是 Tcl 文件所在目录。

保存为 案例工程/filelist_counter.f：

```
./rtl/counter.sv
./formal/counter_props.sv
```

7.1.4 完整运行脚本

依次选择应用、读入设计与 SVA、定义时钟复位、保存初始状态、检查设置、列出属性并开始证明。初始化步骤需要在实际安装环境核对复位状态，不能把声明复位等同于已经完成复位。

保存为 案例工程/scripts/fpv_counter.tcl：

```
# Run from the example project root with: vcf -f scripts/fpv_counter.tcl -batch
set_fml_appmode FPV
read_file -top counter -sva -format sverilog -vcs {-f filelist_counter.f}
create_clock clk -period 100
```

```
create_reset rst_n -sense low
sim_run -stable
sim_save_reset
check_fv_setup
report_fv_setup -list
report_fv -list
set_fml_var fml_max_time 1H
set_fml_var fml_max_memory 8GB
check_fv -block
report_fv -verbose
```

运行命令（Bash）：

```
# 从包含“案例工程”目录的上一级目录开始。
cd 案例工程
mkdir -p logs
set -o pipefail
vcf -f scripts/fpv_counter.tcl -batch 2>&1 | tee logs/fpv_counter.log
```

这些命令应在本机具备相应 VC Formal 版本和许可证时执行。日志中的属性状态才是证明结论；命令退出、脚本结束或没有打印错误都不能代替属性结果检查。

结果阅读。在初始状态正确的前提下，正确版预期满足这些断言，并存在到达 9 和回绕的覆盖轨迹。这不是本次 VC Formal 的实际运行结果。该属性集合也不是全部计数器规格：例如每次普通递增的精确关系仍可另加属性，不能仅凭范围成立就认定全部功能正确。

8. 状态回归和证明性能

相同的属性在不同的 RTL、SVA、时钟约束或工具版本下不应直接比较结果。推荐记录固定的 run manifest：源码校验值、文件列表、Tcl 参数、许可证环境、CPU 与内存资源、运行时间、属性状态汇总、工具报告路径。对于复杂设计，建立按功能分类的任务：复位初始化、寄存器读写、主状态机、协议握手、数据通路和异常控制。

9. 验收条件建议

一个适合作为版本交付的形式验证结果包通常应满足：所有必检断言均明确列出；失败已定位和评审；未完成属性标出后续验证措施；所有豁免有理由与责任人；约束集合经过独立审查；关键触发条件有覆盖见证；代码/时钟/复位/工具版本可以重现。这里的“验收”指项目质量要求，而不是 VC Formal 某个按钮自动代表的最终结论。

详见 调试与证明性能.md 及 覆盖与验证验收.md。

10. 参考

VC Formal User Guide, X-2025.06-SP1，§3 ~ 8、§18 ~ 20；Quick Reference Guide，FPV 与各应用的运行及报告部分。

SystemVerilog Assertions（SVA）完整中文教程

源码内嵌版。本章的完整案例以“保存为”标明文件路径，并在下方直接列出全文。代码框不带显示行号，可直接复制；**案例工程/** 只表示目录结构，不要求另找源码附件。包含占位符或保护性 `error` 的专用流程另行明确标为模板。

第 4 册 · 断言语言与设计规则　版本 1.1（源码内嵌修订版）· 2026-10-17
面向 SystemVerilog IEEE 1800 的主要并发断言语法与工程应用；具体 VC Formal 支持范围和限制见 User Guide §3.7 ~ 3.9（第 89 ~ 105 页）及附录 B（第 1109 页起）。部分标准允许的写法可能因工具版本、求解方式或多时钟条件而不可用，使用前需验证实际编译。

1. SVA 解决什么问题

SVA 是用 SystemVerilog 描述时间相关规则的一组语言结构。它不仅可以检查单个信号是否为正确值，还能表达“请求以后数拍内必须应答”“在握手完成之前数据必须保持稳定”等跨周期关系。SVA 可以用于仿真，亦可以在形式验证时被工具转化为逻辑属性。SVA 本身并不让环境自动合法；外部输入要求仍然需要明确的假设和复位模型。



图 6: SVA 触发与周期关系

2. 先理解三种角色

```
ap_safety: assert property (@(posedge clk) disable iff (!rst_n)
                            !(write && full && !read));
as_input:  assume property (@(posedge clk) disable iff (!rst_n)
                            req |-> input_valid);
cp_full:  cover  property (@(posedge clk) disable iff (!rst_n)
                            full);
```

`assert property` 定义需要被验证的设计责任；`assume property` 描述环境向设计提供的约束；`cover property` 请求寻找满足某条行为的输入轨迹。不要把设计错误的结论改写为 `assume` 以获得更多的 `Proven`。

3. 并发断言的时钟、采样与失效条件

3.1 标准结构

```
property p_example;
  @(posedge clk) disable iff (!rst_n)
  trigger |-> ##[1:3] response;
endproperty

ap_example: assert property (p_example);
```

- `@(posedge clk)`：属性在时钟上升沿采样。
- `disable iff (!rst_n)`：复位低有效期间关闭或中止该属性的活动求值。
- `trigger`：时序要求的触发部分；称为前件（antecedent）。
- `|->`：重叠蕴含；后件从前件序列匹配结束的那个采样周期开始。
- `##[1:3] response`：允许未来第 1、2 或 3 个采样周期响应。

SVA 并发断言在时钟事件的采样值上判断表达式，通常会看到 **NBA 非阻塞赋值更新之前**采样到的值。因此，在 `always_ff @(posedge clk)` 中用非阻塞赋值更新的寄存器，其新值通常在下一次采样沿才被断言看到。检查“状态更新后”的规则时往往需要 `|=>` 或 `$past`。

3.2 复位并不是采样函数历史自动清零

`disable iff` 是属性失效条件；不能将其理解为 `$past` 自动替换成复位值。比如初次进入正常工作状态时，上一拍可能仍处于复位。如果规则依赖上一拍已经处于有效工作状态，应增加 `past_valid` 寄存器或适当的 `$past(rst_n)` 前提。

```
logic past_valid;
always_ff @(posedge clk or negedge rst_n)
  if (!rst_n) past_valid <= 1'b0;
  else        past_valid <= 1'b1;

ap_hold: assert property (@(posedge clk) disable iff (!rst_n)
  past_valid && hold |-> data == $past(data));
```

上例的 `past_valid` 为工具提供可信的前一采样点。在不同时间点立即释放/断言复位时，异步 `disable iff` 和时钟采样的交互仍应按 DUT 的实际复位语义审查。

3.3 default clocking / default disable iff

```
default clocking cb @(posedge clk); endclocking
default disable iff (!rst_n);

ap_onehot: assert property ($onehot0(grant));
```

适用于同一时钟下的属性模块。多时钟或不同复位域，建议为每条关键属性显式指定时钟和失效条件，避免默认时钟覆盖不匹配的接口。

4. 重叠蕴含与非重叠蕴含

表达式	正确读法	a 于第 5 拍成立时
`a |-> b`	同一触发位置检查 b	第 5 拍检查 b
`a |=> b`	下一个采样位置检查 b	第 6 拍检查 b
`a |-> ##2 b`	后件延迟 2 拍	第 7 拍检查 b
`a |=> ##2 b`	下拍开始再延迟 2 拍	第 8 拍检查 b

对**长度大于 1 的前件序列**，后件从前件**完成匹配的那个周期**计算，而不简单从前件的第一个周期计算。逻辑上可以把 `a |=> b` 视为 `a |-> ##1 b`（对这里简单的布尔时序表达），但是复杂序列与时间区域需要按定义分析。

5. sequence 和 property 的职责

```
sequence s_accept;
  valid && ready;
endsequence

sequence s_finish_within_3;
  ##[1:3] done;
endsequence

property p_finish;
  @(posedge clk) disable iff (!rst_n)
  s_accept |-> s_finish_within_3;
endproperty

ap_finish: assert property(p_finish);
```

`sequence` 描述如何匹配一个有限时序片段；`property` 组合时序规则、蕴含、停止和其他属性运算符。复杂序列可以接受参数和局部变量，用于协议检测、延迟记录及数据关联。推荐给可重复使用的时序模式起易读的名称。

6. 序列连接与延迟运算符

6.1 固定延迟 ##n

```
sequence s_two_cycles;
  a ##2 b;
endsequence
```

这要求 a 后第二个采样时刻 b 成立。`a ##0 b` 要求二者在同一采样时刻满足。当 a 是多周期序列时，`##n` 从 a 匹配结束处计算。

6.2 范围延迟 ##[m:n]

```
ap_ack_1_4: assert property (@(posedge clk) disable iff (!rst_n)
  $rose(req) |-> ##[1:4] ack);
```

ack 可以在第 1 ~ 4 拍任一时刻满足。要注意**逐拍重新发起的并行检查**：如果 req 连续高 4 拍，使用 req 作为前件就会发起 4 条独立的响应义务。使用 $rose(req) 仅表示检测上升沿，仍不一定等价于总线“接受请求”的定义。

6.3 连续重复 [*]

```
// busy 必须持续 3 拍
ap_busy: assert property (@(posedge clk) disable iff (!rst_n)
  start |-> ##1 busy[*3]);

// ready 可以延迟出现，示意有界等待
ap_wait: assert property (@(posedge clk) disable iff (!rst_n)
  start |-> (!ready)[*0:3] ##1 ready);
```

[*3] 表示连续 3 次匹配；[*1:4] 表示连续 1 ~ 4 次；[*0:$] 允许零次且上限无限，和后续表达式连接时可能形成意料外的空匹配。尤其是空匹配与蕴含组合，必须通过覆盖和波形验证真正在检查的规则。

6.4 非连续重复 [=] 与 goto 重复 [->]

```
sequence s_three_events;
  event_a[=3];
endsequence

sequence s_second_hit;
  event_a[->2];
endsequence
```

[=3] 描述在一段匹配范围内出现三次非连续的 event_a，可在最后一次命中后继续匹配非事件周期；[->2] 直到第二次事件命中处结束。两者区别会改变和后续 ##1 b 的关系，复杂协议建议先写小型波形测试确认其末端匹配位置。

7. 序列的组合

7.1 and / or / intersect

```
sequence s_both;
  (a ##1 b) and (c ##2 d);
endsequence

sequence s_one;
  (a ##1 b) or (a ##2 b);
endsequence
```

```
sequence s_same_window;
  (a ##1 b) intersect (c ##1 d);
endsequence
```

and 要求两个序列都匹配，但它们的结束位置不要求相同；intersect 要求匹配区间的开始和结束一致。or 允许任意一个序列匹配。不要把序列 and 与普通同周期位运算 && 混淆。

7.2 throughout：范围内持续成立

```
ap_hold_during_busy: assert property (@(posedge clk) disable iff (!rst_n)
  start |-> (enable throughout (busy[*1:5] ##1 done)));
```

expr throughout sequence 表示布尔表达式在被匹配序列涉及的全部采样位置保持真。对于简单请求保持协议，通常 req && !ack |=> req 更直接、更易审查。

7.3 within：一个时序片段位于另一个片段范围中

```
sequence s_short;
  a ##1 b;
endsequence
sequence s_outer;
  start ##[1:5] stop;
endsequence
sequence s_inside;
  s_short within s_outer;
endsequence
```

within 可以表达一个事件序列完整发生在更大序列匹配区间之内。涉及可变时长时要特别警惕匹配歧义。使用工具的 cover 检查预期的代表性路径。

7.4 first_match

```
sequence s_first_ack;
  first_match(##[1:8] ack);
endsequence
```

用于从候选匹配中选取首次匹配，减少后续序列对不同匹配长度的歧义。不是所有需要“最早应答”的规则都要使用 first_match；如果只要求区间内任一次应答，则直接使用范围延迟更清楚。

8. 属性运算符：时间上的持续和终止

8.1 until 与 until_with

```
ap_keep_valid: assert property (@(posedge clk) disable iff (!rst_n)
  req |-> (valid until_with ack));
```

`until` 不要求条件在终止条件发生的那个周期仍成立；`until_with` 要求在终止周期也成立。弱形式可能不要求终止条件真的出现，因此如果要求“必须最终完成”，还需要强形式或独立的有界完成检查。

8.2 `strong` 与 `weak`

```
ap_eventual: assert property (@(posedge clk) disable iff (!rst_n)
  req |-> strong(##[1:$] ack));
```

`strong` 表示时序匹配最终必须结束（具体可证明性视形式应用、活性支持以及公平性而定）；`weak` 允许一些有限轨迹或无限未完成场景下的宽松解释。无条件无限等待往往困难，实际项目应先考虑规范是否可以给出合理的最大响应延迟。强活性属性通常需要假设外部持续提供时钟与资源。

8.3 `accept_on` / `reject_on`

属性可在规定事件发生时被提前接受或拒绝。例如，一次事务在取消信号到来后不再要求原始应答，是否“取消即成功”必须符合协议而不能仅为了避免失败。`disable iff` 通常用于复位；`accept_on` 可用于协议意义上的正常取消；`reject_on` 可用于协议明确规定的提前失败条件。两者与采样时刻有关，项目需要按工具实现验证。

9. 常用采样函数

函数	含义	典型用途	容易忽略的地方
`$past(expr, n)`	读取前 n 次采样的值	下一周期状态关系	起始历史值未定义
`$rose(expr)`	检测从非真到真的变化	捕获请求新事务	不等同于 `req && ready`
`$fell(expr)`	检测从真到非真的变化	撤销请求	异步与 X 变化要谨慎
`$stable(expr)`	当前值与上一采样相同	等待期间数据保持	初始样本的比较问题
`$changed(expr)`	与上一采样不同	寻找状态翻转	重复周期持续匹配
`$isunknown(expr)`	表达式包含 X 或 Z	仿真与 X 检查	二值形式模型解释可不同
`$onehot(expr)`	恰有一位为 1	必须存在且唯一授权	空闲时全零会失败
`$onehot0(expr)`	最多一位为 1	允许空闲的授权	不能保证至少有一位
`$countones(expr)`	统计为 1 的位数	配额/令牌检查	大位宽求解可能昂贵
`$initstate`	初始时间检测	初始化相关规则	工具、上下文支持需核对

9.1 `$past` 的正确使用

```
ap_count_inc: assert property (@(posedge clk) disable iff (!rst_n)
  past_valid && $past(inc && !clear)
```

```
    |-> count == $past(count) + 1'b1);
```

这是在**当前采样周期**检查上一周期已满足加一条件后的寄存器值。若计数器会饱和或回绕，还必须补上对应前提。需要特别确认 past_valid 的第一个有效周期与设计运行时序一致。

9.2 $stable 的常见错误

对 valid && !ready |=> $stable(data) 的解释要看接口在下一拍是否仍继续 valid。若数据在 valid && ready 发生后下一拍可以自由变化，则这个规则可能过度严格。更常用且直观的条件是：

```
ap_stable_when_stalled: assert property (@(posedge clk) disable iff (!rst_n)
  valid && !ready |=> valid && $stable(data));
```

这个写法要求：上一拍未被接受的传输，下一拍仍保持 valid 和数据，恰好符合很多 ready/valid 接口的要求。允许在下一拍被接受并不改变“采样的当前数据仍应保持”的要求。

10. 立即断言和并发断言的区别

```
always_comb begin
  assert (!(grant[0] && grant[1]))
    else $error("two simultaneous grants");
end

ap_grant: assert property (@(posedge clk) disable iff (!rst_n)
  $onehot0(grant));
```

立即断言在过程执行时检查瞬时条件，可能受组合逻辑的调度次序、暂态变化及 X 值影响；并发断言按事件采样，便于表达时序关系。形式验证中要遵循该工具对即时/延迟断言支持的说明；不能把所有仿真调试语句当成可被形式引擎直接证明的属性。

11. 属性局部变量、let 和参数化

11.1 let：抽象复杂布尔条件

```
let accepted(v, r) = (v && r);

ap_no_drop: assert property (@(posedge clk) disable iff (!rst_n)
  accepted(valid, ready) |-> !error);
```

let 表达式可复用，减少同一接口条件在几十条属性中被不同工程师写出微小差异。是否使用 let 由编译器支持情况决定。

11.2 sequence 局部变量：关联请求与响应数据

```
property p_data_return;
  logic [7:0] saved_id;
```

```
  @(posedge clk) disable iff (!rst_n)
    (req, saved_id = req_id) |-> ##[1:4]
      (rsp && rsp_id == saved_id);
endproperty
ap_data_return: assert property (p_data_return);
```

此属性为**每一次匹配的请求**保存其标识并要求窗口内匹配应答。若允许多个并发请求且同一标识重复使用，仍需根据协议处理事务编号、队列深度、重排规则和独立超时。局部变量并非 RTL 中真的增加了寄存器，工具可能为断言引入匹配状态，过度复杂的动态序列会增加形式求解成本。

11.3 参数化属性

```
module ready_valid_checker #(
  parameter int DATA_W = 32,
  parameter int RESP_MAX = 8
) (
  input logic clk, rst_n, valid, ready,
  input logic [DATA_W-1:0] data
);
  ap_stall: assert property (@(posedge clk) disable iff(!rst_n)
    valid && !ready |=> valid && $stable(data));
  cp_transfer: cover property (@(posedge clk) disable iff(!rst_n)
    valid && ready);
endmodule
```

参数化断言适合复用在多个实例；但 RESP_MAX 等参数如果在属性中用于常量延迟，必须遵守工具对常量表达式的要求。

12. bind：无需改动设计源码注入 SVA

```
module counter_checker (
  input logic clk, rst_n, inc,
  input logic [3:0] count
);
  ap_range: assert property (@(posedge clk) disable iff(!rst_n)
    count <= 4'd9);
  cp_full: cover property (@(posedge clk) disable iff(!rst_n)
    count == 4'd9);
endmodule

bind counter counter_checker u_checker (
  .clk(clk), .rst_n(rst_n), .inc(inc), .count(count)
);
```

bind 在指定设计模块或实例上附加检查器，不必改动交付 RTL。但要确保端口名与被绑定模块一致，检查器的编译顺序正确，且顶层编译开启断言支持（例如相应流程中的 -sva）。**不要在形式测试文件里用 assign 强行驱动 DUT 的内部状态来代替外部约束**。

13. 常见系统级断言模板

13.1 复位值

```
ap_reset_state: assert property (@(posedge clk)
  !rst_n |-> state == IDLE);
```

此示例适用于复位在该采样时钟上足够早生效，且状态在这一采样点应已为 IDLE 的协议模型。若同步复位在 posedge clk 的 NBA 才生效，应换成正确的下一拍检查；异步复位应另外确认有效瞬间与采样要求。

13.2 一热与互斥

```
ap_mutex: assert property (@(posedge clk) disable iff(!rst_n)
  $onehot0(grant));
ap_one_grant_when_pending: assert property (@(posedge clk) disable iff(!rst_n)
  pending |-> $onehot(grant));
```

第二条仅适用于协议要求有请求时必须立刻授权，否则应改成有界最终授权，并定义公平条件。

13.3 请求—应答

```
ap_ack: assert property (@(posedge clk) disable iff(!rst_n)
  (req && accept) |-> ##[1:3] ack);
cp_req: cover property (@(posedge clk) disable iff(!rst_n)
  req && accept);
```

该写法有一个明确的受理事件；如果 ack 可能与受理同周期，则窗口改成 ##[0:3]。若协议允许接受多个请求，需要采用事务关联检查。

13.4 背压期间保持

```
ap_hold: assert property (@(posedge clk) disable iff(!rst_n)
  valid && !ready |=> valid && $stable(payload));
```

对于标准 ready/valid 流控制，后一拍可以 ready 变为 1，此时仍须在握手时提供原来的 payload。

13.5 FIFO 不上溢、不下溢与计数范围

```
ap_depth: assert property (@(posedge clk) disable iff(!rst_n)
  occupancy <= DEPTH);
ap_no_empty_pop: assert property (@(posedge clk) disable iff(!rst_n)
  pop_accept |-> !empty);
ap_no_full_push: assert property (@(posedge clk) disable iff(!rst_n)
  push_accept |-> (!full || pop_accept));
```

这里假定 FIFO 允许满状态同时读写（具体需看实现），push_accept/pop_accept 是**已接受操作**而不是原始请求。若 push 为请求而硬件可以拒绝，那么 push && full 并非必然错误。计数器溢

出的检查应与接受操作和旁路条件一致。

13.6 超时与重试

```
ap_timeout: assert property (@(posedge clk) disable iff(!rst_n)
  launch |-> ##[1:8] (done || timeout));
```

本例只保证在第 1 ~ 8 拍的某一拍看到完成或超时，没有保证二者互斥；如果规定互斥，另外增加 `!(done && timeout)` 断言。

13.7 有限状态机状态合法性

```
ap_state_legal: assert property (@(posedge clk) disable iff(!rst_n)
  state inside {ST_IDLE, ST_REQ, ST_WAIT, ST_DONE});
```

`inside` 中的常量值必须与 RTL 一致。枚举类型有时可以使用更清晰的编码检查；该属性只能证明取值合法，不能证明所有转移符合协议。

13.8 安全权限控制

```
ap_secure_write: assert property (@(posedge clk) disable iff(!rst_n)
  write_accept && secure_reg_sel |-> privileged);
```

还需要检查非法请求是否被阻止、读路径是否泄漏以及复位后的权限状态。单条写入权限断言不代表已经证明没有信息泄漏。

14. 空洞性、覆盖目标与不恰当假设

```
ap_req_ack: assert property (@(posedge clk) disable iff(!rst_n)
  req |-> ##[1:3] ack);
cp_req_seen: cover property (@(posedge clk) disable iff(!rst_n)
  req);
cp_req_ack: cover property (@(posedge clk) disable iff(!rst_n)
  req ##[1:3] ack);
```

反例：如果在其他文件里写了 `assume property(!req)`，`ap_req_ack` 可能轻松被证明，但是 `cp_req_seen` 不可达。**验证环境需要既让合法关键场景可达，又排除真实接口所禁止的输入行为。**

工程里建议同时维护三类检查：设计断言；环境前提的注释与审查；关键规格场景覆盖。形式覆盖与仿真覆盖不是同一概念，应分别分析。

15. 容易写错的规则与修正

错误写法或理解	风险	推荐修正
`req |-> ack` 代表“下一拍应答”	实际检查同拍	使用 `req |=> ack` 或 `##1`
`req |=> ##2 ack` 代表延迟 2 拍	实际延迟 3 拍	使用 `req |-> ##2 ack`
直接 `$past(data)` 不检查历史	起始周期样本无效	以 `past_valid` 或 `$past(rst_n)` 守卫
`req |-> ##[0:$] ack` 必然要求最终应答	弱匹配及工具处理可能不符合意图	使用有界响应或合适的强活性形式
`req |-> ack` 证明通过就认为请求已发生	前件可能不可达	添加 `cover property(req)`
在所有周期假设 `ready=1`	屏蔽背压缺陷	只根据环境规格约束 `ready`
用 `$onehot(grant)` 检查允许全零的授权	空闲周期错误失败	使用 `$onehot0(grant)`
用 `count < DEPTH` 当作假设	直接假设安全性	改成 `assert`，必要时增加真实输入前提
用 `disable iff(!rst_n)` 就认为复位值已被证明	禁用了复位期间检查	另写复位属性或复位启动流程
`cover property(done)` 认为所有请求都能完成	只证明至少存在一次完成	对每次合法请求写断言

16. 多时钟属性、X 值与复杂语法的限制

多时钟属性在 SystemVerilog 标准中有专门的采样与序列连接规则，不能简单将两种时钟事件放入单时钟模板。不同 EDA 引擎对多时钟交叉属性支持可能有限；参考 VC Formal User Guide §3.9 的具体限制。跨时钟同步器、握手与异步 FIFO 通常需分别定义每个时钟域的责任及跨时钟的数据一致性。

`$isunknown` 在四值仿真中很直观，但传统形式证明的二值布尔编码可能与仿真中的 X 传播语义不同；检查 RTL 中 X 传播风险更适合配合 VC Formal FXP 和明确的注入/观察模型。写有 `casex`、`casez`、`unique case` 的 RTL 时，还应结合综合与仿真对 X 的不同处理方式审查。

17. 完整示例：有界计数器及 VC Formal 工程

前文的短代码块用于解释单个语法；本节则给出可保存成文件的完整设计、检查器、编译列表和 Tcl。代码与 FPV 章的计数器案例保持一致。

17.1 被验证的计数器 RTL

规格：复位后计数为零；使能时从 0 计到 9 后回到 0；没有使能时保持。RTL 文件只包含设计，不把待验证结论写成环境假设。

保存为 案例工程/rtl/counter.sv：

```
module counter (
    input  logic       clk,
    input  logic       rst_n,
    input  logic       enable,
    output logic [3:0] count
);
  always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n)             count <= 4'd0;
    else if (enable) begin
      if (count == 4'd9)    count <= 4'd0;
      else                  count <= count + 4'd1;
    end
  end
endmodule
```

17.2 完整断言与绑定文件

history_valid 在复位后建立历史有效标志。ap_range 检查范围，ap_hold 检查保持，ap_wrap 检查回绕；两条 cover 检查最高计数值及回绕场景是否能够出现。末尾 bind 将检查器接入同名设计信号，不需要修改 RTL。

保存为 案例工程/formal/counter_props.sv：

```
module counter_props (
    input logic       clk, rst_n, enable,
    input logic [3:0] count
);
  logic history_valid;
  always_ff @(posedge clk or negedge rst_n)
    if (!rst_n) history_valid <= 1'b0;
    else        history_valid <= 1'b1;

  ap_range: assert property (@(posedge clk) disable iff (!rst_n)
      count <= 4'd9);

  ap_hold: assert property (@(posedge clk) disable iff (!rst_n)
      history_valid && $past(!enable) |-> count == $past(count));

  ap_wrap: assert property (@(posedge clk) disable iff (!rst_n)
      history_valid && $past(enable && count == 4'd9)
      |-> count == 4'd0);

  cp_reaches_nine: cover property (@(posedge clk) disable iff (!rst_n)
      count == 4'd9);
  cp_wrap: cover property (@(posedge clk) disable iff (!rst_n)
      count == 4'd9 ##1 count == 4'd0);
endmodule

bind counter counter_props u_counter_props (.*);
```

17.3 编译文件列表

先列 RTL，再列包含 `bind` 的属性文件；路径相对于运行 VC Formal 的 **案例工程/** 目录，而不是 Tcl 文件所在目录。

保存为 案例工程/filelist_counter.f：

```
./rtl/counter.sv
./formal/counter_props.sv
```

17.4 完整运行脚本

依次选择应用、读入设计与 SVA、定义时钟复位、保存初始状态、检查设置、列出属性并开始证明。初始化步骤需要在实际安装环境核对复位状态，不能把声明复位等同于已经完成复位。

保存为 案例工程/scripts/fpv_counter.tcl：

```
# Run from the example project root with: vcf -f scripts/fpv_counter.tcl -batch
set_fml_appmode FPV
read_file -top counter -sva -format sverilog -vcs {-f filelist_counter.f}
create_clock clk -period 100
create_reset rst_n -sense low
sim_run -stable
sim_save_reset
check_fv_setup
report_fv_setup -list
report_fv -list
set_fml_var fml_max_time 1H
set_fml_var fml_max_memory 8GB
check_fv -block
report_fv -verbose
```

运行命令（Bash）：

```
# 从包含“案例工程”目录的上一级目录开始。
cd 案例工程
mkdir -p logs
set -o pipefail
vcf -f scripts/fpv_counter.tcl -batch 2>&1 | tee logs/fpv_counter.log
```

这些命令应在本机具备相应 VC Formal 版本和许可证时执行。日志中的属性状态才是证明结论；命令退出、脚本结束或没有打印错误都不能代替属性结果检查。

结果阅读。在初始状态正确的前提下，正确版预期满足这些断言，并存在到达 9 和回绕的覆盖轨迹。这不是本次 VC Formal 的实际运行结果。该属性集合也不是全部计数器规格：例如每次普通递增的精确关系仍可另加属性，不能仅凭范围成立就认定全部功能正确。

18. 工程练习（建议自行运行与审查）

1. 将计数器回绕阈值从 9 改为 10，观察 `ap_range` 的反例；将阈值改回后重复证明。

2. 删除 `history_valid`，分析首个有效采样点 `$past` 规则的差异。
3. 为一个 2 输入仲裁器编写互斥、授权必须对应请求和竞争覆盖三类属性。
4. 为 ready/valid 通道编写背压保持、握手可达与连续握手覆盖。
5. 故意加入约束 `assume(!enable)`，观察覆盖目标为什么无法满足，并撤销该约束。

本章第 17 节已直接列出练习所需的 RTL、SVA、文件列表与完整运行脚本；独立源码文件仅用于方便编辑，不是阅读本章的前置要求。

19. 语法检索表

语法	类型	说明
`@(posedge clk)`	采样事件	时钟上升沿
`disable iff (cond)`	失效条件	条件为真时关闭正在运行的属性匹配
`##n` / `##[m:n]`	序列延迟	固定或范围延迟
`[*n]` / `[*m:n]`	连续重复	连续匹配指定次数
`[=n]`	非连续重复	指定次数事件出现
`[->n]`	goto 重复	第 n 次出现时匹配结束
`|->` / `|=>`	蕴含	重叠 / 非重叠
`and` / `or`	序列组合	双方均满足 / 任意一方满足
`intersect`	序列组合	区间起止匹配一致
`within`	序列组合	序列完整包含于另一序列区间
`throughout`	序列组合	布尔条件覆盖序列全部采样位置
`first_match`	匹配选择	第一次满足的候选匹配
`until` / `until_with`	属性时间关系	直到条件发生（含/不含终止采样）
`strong` / `weak`	匹配强度	终止义务强弱
`$past` / `$rose`	采样函数	过去值 / 上升事件
`assert` / `assume` / `cover`	属性角色	证明 / 环境前提 / 到达目标
`bind`	检查器注入	将外部 SVA 模块附加到 DUT

20. 参考与责任界定

本教程独立编写，对照 IEEE 1800 的常见语法概念以及用户所给 VC Formal User Guide X-2025.06-SP1 §3.7 ~ 3.9、附录 B。工具具体支持范围、强活性处理、采样行为和优化由正式安装版本决定。本文示例用于教学；没有声称在用户实际 VC Formal 授权环境中运行过全部代码。

FPV：功能属性验证应用完整指南

源码内嵌版。本章的完整案例以“保存为”标明文件路径，并在下方直接列出全文。代码框不带显示行号，可直接复制；**案例工程/** 只表示目录结构，不要求另找源码附件。包含占位符或保护性 `error` 的专用流程另行明确标为模板。

第 5 册 · 专用应用　Functional Property Verification。依据 User Guide 第 17 章（第 686 ~ 713 页），并结合第 3 ~ 8、18 ~ 20 章及 Quick Reference Guide 的 FPV 部分。本章直接列出设计、断言、文件列表与 Tcl；独立源码文件只作为便于编辑的副本。

1. 定位、能力与不适用范围

FPV 通过数学证明验证 RTL 和 SVA 所描述的设计规则。输入包括：待验证 RTL、用户断言、Assertion IP（AIP）断言、环境假设、时钟复位和初始化状态；输出为证明状态、反例、覆盖轨迹和分析报告。它适合局部协议、状态机、控制面、缓冲区、寄存器副作用、异常控制等，可在传统 testbench 完成前投入使用。

FPV 不会自动理解芯片规格，也不自动生成所有应该证明的规则。只有对**已经编写并启用**的属性进行了证明才有相应结论，且环境约束必须可信。若设计缺少软件配置顺序或外部协议模型，可能出现不实际的反例；若过度约束，则可能出现空洞证明。

2. 标准流程与输入输出

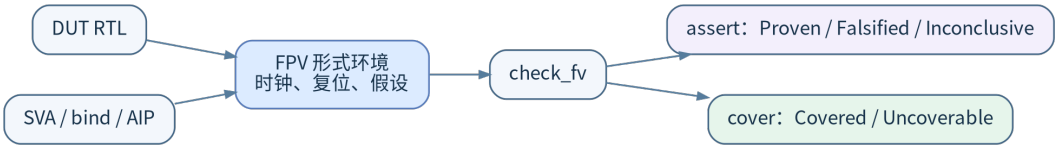


图 7: FPV 工作流程

阶段	任务	命令或文件	核查点
选择应用	设置 FPV	`set_fml_appmode FPV`	模式是否正确
编译	读入 RTL、属性和 bind 文件	`read_file` 或 `analyze/elaborate`	顶层、参数、属性数量

阶段	任务	命令或文件	核查点
初始化	时钟、复位、常量、初始状态	`create_clock`、`create_reset`、`sim_save_reset`	与真实启动一致
环境审查	检查黑盒和模型问题	`report_blackbox`、`check_fv_setup`	不存在关键环境缺陷
运行	执行验证	`check_fv -block`	任务真正结束
分析	输出证明状态和反例	`report_fv`、`view_trace`	性质与前提一致
回归	保存可复现结果	`save_session`、日志/manifest	版本与运行资源可追踪

3. 单时钟计数器：完整 RTL、SVA 与运行流程

本节直接给出全部依赖文件。按“保存为”的路径建立目录并保存代码，就能准备对应教学工程，不必再到附件寻找 RTL、断言或脚本。

3.1 被验证的计数器 RTL

规格：复位后计数为零；使能时从 0 计到 9 后回到 0；没有使能时保持。RTL 文件只包含设计，不把待验证结论写成环境假设。

保存为 案例工程/rtl/counter.sv：

```
module counter (
    input  logic       clk,
    input  logic       rst_n,
    input  logic       enable,
    output logic [3:0] count
);
  always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n)             count <= 4'd0;
    else if (enable) begin
      if (count == 4'd9)    count <= 4'd0;
      else                  count <= count + 4'd1;
    end
  end
endmodule
```

3.2 完整断言与绑定文件

`history_valid` 在复位后建立历史有效标志。`ap_range` 检查范围，`ap_hold` 检查保持，`ap_wrap` 检查回绕；两条 `cover` 检查最高计数值及回绕场景是否能够出现。末尾 `bind` 将检查器接入同名设计信号，不需要修改 RTL。

保存为 案例工程/formal/counter_props.sv：

```
module counter_props (
    input logic       clk, rst_n, enable,
    input logic [3:0] count
);
  logic history_valid;
  always_ff @(posedge clk or negedge rst_n)
    if (!rst_n) history_valid <= 1'b0;
    else        history_valid <= 1'b1;

  ap_range: assert property (@(posedge clk) disable iff (!rst_n)
      count <= 4'd9);

  ap_hold: assert property (@(posedge clk) disable iff (!rst_n)
      history_valid && $past(!enable) |-> count == $past(count));

  ap_wrap: assert property (@(posedge clk) disable iff (!rst_n)
      history_valid && $past(enable && count == 4'd9)
      |-> count == 4'd0);

  cp_reaches_nine: cover property (@(posedge clk) disable iff (!rst_n)
      count == 4'd9);
  cp_wrap: cover property (@(posedge clk) disable iff (!rst_n)
      count == 4'd9 ##1 count == 4'd0);
endmodule

bind counter counter_props u_counter_props (.*);
```

3.3 编译文件列表

先列 RTL，再列包含 bind 的属性文件；路径相对于运行 VC Formal 的 案例工程/ 目录，而不是 Tcl 文件所在目录。

保存为 案例工程/filelist_counter.f：

```
./rtl/counter.sv
./formal/counter_props.sv
```

3.4 完整运行脚本

依次选择应用、读入设计与 SVA、定义时钟复位、保存初始状态、检查设置、列出属性并开始证明。初始化步骤需要在实际安装环境核对复位状态，不能把声明复位等同于已经完成复位。

保存为 案例工程/scripts/fpv_counter.tcl：

```
# Run from the example project root with: vcf -f scripts/fpv_counter.tcl -batch
set_fml_appmode FPV
read_file -top counter -sva -format sverilog -vcs {-f filelist_counter.f}
create_clock clk -period 100
```

```
create_reset rst_n -sense low
sim_run -stable
sim_save_reset
check_fv_setup
report_fv_setup -list
report_fv -list
set_fml_var fml_max_time 1H
set_fml_var fml_max_memory 8GB
check_fv -block
report_fv -verbose
```

运行命令（Bash）：

```
# 从包含“案例工程”目录的上一级目录开始。
cd 案例工程
mkdir -p logs
set -o pipefail
vcf -f scripts/fpv_counter.tcl -batch 2>&1 | tee logs/fpv_counter.log
```

这些命令应在本机具备相应 VC Formal 版本和许可证时执行。日志中的属性状态才是证明结论；命令退出、脚本结束或没有打印错误都不能代替属性结果检查。

结果阅读。在初始状态正确的前提下，正确版预期满足这些断言，并存在到达 9 和回绕的覆盖轨迹。这不是本次 VC Formal 的实际运行结果。该属性集合也不是全部计数器规格：例如每次普通递增的精确关系仍可另加属性，不能仅凭范围成立就认定全部功能正确。

3.5 故意错误的 RTL

错误只在回绕阈值：把 9 改为 10。模块名仍为 `counter`，所以此文件与正确版必须分别编译。属性文件沿用本节已经完整列出的 `counter_props.sv`。

保存为 案例工程/rtl/counter_bad.sv：

```
// Intentionally buggy teaching variant: roll-over condition is 10 instead of 9.
// Do NOT include with rtl/counter.sv in the same compilation.
module counter (
    input  logic       clk,
    input  logic       rst_n,
    input  logic       enable,
    output logic [3:0] count
);
  always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n)              count <= 4'd0;
    else if (enable) begin
      if (count == 4'd10)    count <= 4'd0;
      else                   count <= count + 4'd1;
    end
  end
endmodule
```

3.6 错误版编译文件列表

只读入错误设计及同一份检查器，不同时包含两个同名设计。

保存为 案例工程/filelist_counter_bad.f：

```
./rtl/counter_bad.sv
./formal/counter_props.sv
```

3.7 错误版完整运行脚本

与正确版采用相同接口和初始化方法，不为了让错误版通过而增加输入限制。

保存为 案例工程/scripts/fpv_counter_bad.tcl：

```
# Intentionally failing counter example. Do not include the good counter at once.
set_fml_appmode FPV
read_file -top counter -sva -format sverilog -vcs {-f filelist_counter_bad.f}
create_clock clk -period 100
create_reset rst_n -sense low
sim_run -stable
sim_save_reset
check_fv_setup
check_fv -block
report_fv -verbose
```

运行命令（Bash）：

```
# 从包含“案例工程”目录的上一级目录开始。
cd 案例工程
mkdir -p logs
set -o pipefail
vcf -f scripts/fpv_counter_bad.tcl -batch 2>&1 | tee logs/fpv_counter_bad.log
```

这些命令应在本机具备相应 VC Formal 版本和许可证时执行。日志中的属性状态才是证明结论；命令退出、脚本结束或没有打印错误都不能代替属性结果检查。

错误版的预期反例是：在允许连续使能的输入下，计数从 9 进入 10，违反范围与回绕规则。查看反例时应区分时钟沿的采样值与非阻塞赋值后的寄存器值；不能把这里描述的预期现象伪装成工具日志。

4. 如何组织断言

建议把每条属性归入四个类别：初始化（复位值和复位后输出）；协议（有效应答、授权、稳定性）；内部安全性（计数范围、状态合法、资源互斥）；进展性（最大等待时间、错误响应）。每个类别不仅包含断言，还要有能触发关键规则的覆盖目标。可将大型属性拆成“输入合法 → 请求被接受 → 内部状态变化 → 输出响应”的小性质，降低定位和求解难度。

4.1 它真正验证了什么

```
ap_range: assert property (@(posedge clk) disable iff(!rst_n)
  count <= 4'd9);

cp_wrap: cover property (@(posedge clk) disable iff(!rst_n)
  count == 4'd9 ##1 count == 4'd0);
```

ap_range 证明占用值不超过上界；cp_wrap 搜索一次从 9 到 0 的回绕。两者不能互相替代。若设计从未能到达 9，ap_range 仍可能通过，但回绕规则并未真正被验证。对错误 RTL，前者应给出反例，后者是否能够覆盖取决于更新条件。

4.2 使用 bind 与外部属性模块

推荐把待验证 RTL 与检查器分离，方便调试和提交代码：

```
module interface_checker (
  input logic clk, rst_n,
  input logic valid, ready,
  input logic [7:0] data
);
  ap_stable: assert property (@(posedge clk) disable iff(!rst_n)
     valid && !ready |=> valid && $stable(data));
  cp_transfer: cover property (@(posedge clk) disable iff(!rst_n)
     valid && ready);
endmodule
```

在 formal/bind.sv 内写 bind <DUT_type> interface_checker ...，然后使绑定文件编译进入 -sva 的设计。必须确认属性端口与 DUT 的实际时序一致，不要仅凭属性能被编译就认为建模正确。

5. 常用 Tcl 命令逐项讲解

命令	作用	使用时机及注意事项
set_fml_appmode FPV	选择 FPV	在编译前执行；可用 get_fml_appmode 核对
read_file -top ... -sva ...	编译并展开含断言设计	源文件列表和模块顶层必须正确
analyze ...; elaborate ... -sva	将编译与展开拆开	不与 read_file 任意重复
aip_load -protocol "AHB AXI5"	加载受支持的协议断言 IP	需要协议适配及正确的 AIP 连接配置
create_clock	定义逻辑时钟	多域设计要分别核查
create_reset	定义复位极性	必须与实际 RTL 相符
set_constant	将稳定控制输入限制为给定值	确保该假设对目标应用合理
sim_run / sim_save_reset	建立并保存初始状态	应审查复位波形和寄存器值

命令	作用	使用时机及注意事项
`check_fv_setup`	检查形式模型和环境	在长时间证明前运行
`fvassert`	在 Tcl 创建或设定断言属性	复杂时序属性推荐 SVA 源文件
`fvassume`	添加或调整形式假设	必须给出规格来源
`fvcover`	添加覆盖目标	应覆盖关键协议触发条件
`fvdisable` / `fvenable`	关闭 / 启用属性	禁用不能隐瞒，需报告
`set_fml_var fml_vacuity_on true`	配置空洞性分析	仍需结合触发覆盖审查
`check_fv -block`	阻塞执行求解	适合批处理脚本
`report_fv -list/-verbose`	列表和详细报告	根据状态区分已证明、失败和未完成
`view_trace -property ...`	打开特定属性反例	对照 SVA 触发周期观察
`save_session -session ...`	保存工具工作会话	保留关联数据库以便恢复

`fvassert <name> -expr {<expression>}`、`fvassume <name> -expr {...}` 和 `fvcover <name> -expr {...}` 适合短属性或自动生成规则。Tcl 的 `{}`、`""` 会影响转义与 `$` 替换，例如使用 `$past` 时推荐外层花括号包围语句或直接写到 SV 中。

6. 四个逐步进阶的用例

用例 A：复位后状态正确

规格：复位退出后，状态机应进入空闲状态。首先明确同步或异步复位和断言采样时刻：

```
ap_reset_release: assert property (
  @(posedge clk) $rose(rst_n) |-> state == IDLE);
```

这段规则只适用于状态在第一次采样到复位高电平的时刻应该已经是 `IDLE` 的实现。若模块采用同步释放后的一个周期才进入有效状态，则必须调整响应周期。为复位前提单独建立初始化流程，不能只靠 `disable iff` 回避复位检查。

用例 B：握手停止期间数据不变

```
ap_stall_hold: assert property (@(posedge clk) disable iff(!rst_n)
  valid && !ready |=> valid && $stable(data));
cp_backpressure: cover property (@(posedge clk) disable iff(!rst_n)
  valid && !ready ##1 ready);
```

如 `ready` 为外部驱动，禁止假设 `ready` 始终为 1；否则背压规则可能很容易被证明，却没有真正触发的测试场景。

用例 C：深层计数器溢出

使用参数化深度为 10 的计数器，故意把最大值条件写错。预期安全性 `count <= 9` 会失败；反例提供每拍 `enable`，可从逐周期变化找到第一次非法值。修复后重新运行，与更早的报告比较应以 RTL 的校验值为前提。

用例 D：多端口仲裁

```
ap_mutex: assert property (@(posedge clk) disable iff(!rst_n)
  $onehot0(grant));
cp_contend: cover property (@(posedge clk) disable iff(!rst_n)
  req == 2'b11);
```

这是最基础的安全性和触发覆盖。若想证明“等待不超过 8 拍”，必须明确持续请求和仲裁器能否接收新请求，并对每一个端口分别写属性。状态空间复杂时，可先证明互斥与请求合法，再证明有界公平性。

6.5 FIFO 控制器：完整代码

这个例子只验证 FIFO 的占用计数、空满标志和请求接受规则，不包含存储阵列，因此不能据此宣称已经证明数据顺序正确。满状态允许同时读写；空状态不能读出，且本例没有空 FIFO 直通功能。

6.5.1 控制器 RTL

`pop_accept` 先排除空读；`push_accept` 在未满或本拍成功读出的条件下接受写入。实际更新依据接受信号，而不是原始 `push/pop` 请求。

保存为 案例工程/rtl/fifo_ctrl.sv：

```
module fifo_ctrl #(
    parameter int DEPTH = 4,
    parameter int CW = $clog2(DEPTH + 1)
) (
    input logic clk, rst_n,
    input logic push, pop,
    output logic full, empty,
    output logic push_accept, pop_accept,
    output logic [CW-1:0] occupancy
);
  assign full       = (occupancy == DEPTH);
  assign empty      = (occupancy == 0);
  assign pop_accept  = pop && !empty;
  assign push_accept = push && (!full || pop_accept);

  always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) occupancy <= '0;
    else case ({push_accept, pop_accept})
      2'b10: occupancy <= occupancy + 1'b1;
```

```
      2'b01: occupancy <= occupancy - 1'b1;
      default: occupancy <= occupancy;
    endcase
  end
endmodule
```

6.5.2 完整 SVA 与参数绑定

属性参数与 RTL 的 DEPTH/CW 对应；检查范围、标志一致性以及禁止空读、禁止无空间写入。覆盖目标辅助确认满状态与使用后回到空状态可达。

保存为 案例工程/formal/fifo_ctrl_props.sv:

```
module fifo_ctrl_props #(
  parameter int DEPTH = 4,
  parameter int CW = $clog2(DEPTH + 1)
) (
  input logic clk, rst_n, push, pop,
  input logic full, empty, push_accept, pop_accept,
  input logic [CW-1:0] occupancy
);
  ap_safe_count: assert property (@(posedge clk) disable iff (!rst_n)
    occupancy <= DEPTH);
  ap_flag_full: assert property (@(posedge clk) disable iff (!rst_n)
    full == (occupancy == DEPTH));
  ap_flag_empty: assert property (@(posedge clk) disable iff (!rst_n)
    empty == (occupancy == 0));
  ap_no_empty_pop: assert property (@(posedge clk) disable iff (!rst_n)
    pop_accept |-> !empty);
  ap_no_overflow_push: assert property (@(posedge clk) disable iff (!rst_n)
    push_accept |-> (!full || pop_accept));
  cp_full: cover property (@(posedge clk) disable iff (!rst_n)
    full);
  cp_empty_after_use: cover property (@(posedge clk) disable iff (!rst_n)
    !empty ##[1:5] empty);
endmodule

bind fifo_ctrl fifo_ctrl_props #(.DEPTH(DEPTH), .CW(CW)) u_fifo_ctrl_props (.*);
```

6.5.3 文件列表

保存为 案例工程/filelist_fifo.f:

```
./rtl/fifo_ctrl.sv
./formal/fifo_ctrl_props.sv
```

6.5.4 完整脚本

保存为 案例工程/scripts/fpv_fifo.tcl:

```
set_fml_appmode FPV
read_file -top fifo_ctrl -sva -format sverilog -vcs {-f filelist_fifo.f}
create_clock clk -period 100
create_reset rst_n -sense low
sim_run -stable
sim_save_reset
check_fv_setup
check_fv -block
report_fv -verbose
```

运行命令（Bash）：

```
# 从包含“案例工程”目录的上一级目录开始。
cd 案例工程
mkdir -p logs
set -o pipefail
vcf -f scripts/fpv_fifo.tcl -batch 2>&1 | tee logs/fpv_fifo.log
```

这些命令应在本机具备相应 VC Formal 版本和许可证时执行。日志中的属性状态才是证明结论；命令退出、脚本结束或没有打印错误都不能代替属性结果检查。

7. 如何分析结果

`Proven`：属性在该模型和假设下成立。需要查触发是否可达与约束是否合理。`Falsified`：工具给出违反路径，首先检查输入是否符合环境规格与时钟复位是否正确。`Inconclusive`：目前未完成，不能当作通过。`Covered`：存在一条满足覆盖目标的路径。`Uncoverable`：该目标在当前模型不可达，需要区分真实死逻辑与过度约束。详见 验证工程与结果判读。

8. 优化证明效率而不损害结论

1. 缩小模块验证范围，必要时划分不同 `fvtask`；不要先大面积假设内部值。
2. 将复杂的一条断言拆成可解释的若干不变量与阶段性结果。
3. 分析影响锥，确认算术、RAM、时钟域和不必要的外围逻辑是否进入证明。
4. 再考虑 `set_fml_var fml_effort high`、资源限制、`set_abstractions`、`snip_driver` 等进阶优化；新抽象可能引入虚假反例。
5. 对长期回归使用符合条件的 `fvlearn_config -local_dir ...`，并区分继承的缓存与新运行获得的真实结果。
6. 使用 `report_fml_engines`、`report_fml_jobs`、`report_fml_hosts` 定位瓶颈。

9. 常见故障与排查

现象	优先检查	不应做的事
所有断言都轻松通过	触发覆盖、空洞性和假设	直接宣布完整验证通过
许多断言在第 0 拍失败	初始化状态与 `$past`	假设所有寄存器为预期值

现象	优先检查	不应做的事
证明几乎没有进展	影响锥、参数和 RAM	无审查地增加内部假设
只有有界深度结论	是否真正 `Proven`	把展开深度当作无限证明
Verdi 中找不到信号	优化、层级、会话版本	在不知影响的情况下关掉全部优化
频繁得到非法请求反例	环境模型与协议规范	把所有 DUT 失败行为都排除

10. 进阶主题

FPV 可以与 AIP、覆盖质量评估（FCA）、FTA 的形式测试平台突变分析、Proof Architect 的组合式证明方法等配合。有关无法快速证明的属性，阅读 调试与证明性能；有关交付报告，阅读 覆盖与验证验收。FPV 的物理规模限制随属性影响锥、状态空间及引擎而变化，没有一个对所有设计都有效的固定门数门槛。

11. 参考定位

User Guide §17（FPV，686 页起），§3（SVA/属性，66 页起），§4（初始化，115 页起），§6 ~ 7（运行与调试，183 ~ 312 页），§18 ~ 20（性能、组合式证明与验收）。Quick Reference Guide 的 FPV 页面提供命令速览，但不是全部参数说明。

AEP：自动提取属性应用完整指南

源码内嵌版。本章的完整案例以“保存为”标明文件路径，并在下方直接列出全文。代码框不带显示行号，可直接复制；**案例工程/** 只表示目录结构，不要求另找源码附件。包含占位符或保护性 `error` 的专用流程另行明确标为模板。

第 6 册 · 专用应用　Automatically Extracted Properties。依据 User Guide 第 10 章（第 362 ~ 385 页）与 Quick Reference Guide 的 AEP 部分。

1. AEP 的角色与应用价值

AEP 不需要用户先手工写出每一条 SVA，就能从 RTL 结构和编码方式中自动推导一批易引发功能问题的条件，并将其交给形式引擎检查。适用于新接入 IP、第三方 RTL、复杂状态机、溢出与数组访问风险排查，以及验证项目启动阶段的快速质量筛查。

AEP **不等于自动证明全部功能规格**。它擅长发现通用的实现错误（越界、算术溢出、状态机问题、三态冲突、一些不完整分支等），但不会知道客户要求某个控制寄存器在第几个周期复位，也不会自动理解外部协议的最终响应期限。经 AEP 筛查后仍需补充 FPV 业务属性。

2. AEP 的典型检查项目

AEP 类型	`-aep` 选择值（按 User Guide）	要检查的危险行为	例子
数组越界	`bounds_check`	索引超出声明范围	深度为 2 的数组访问索引 3
算术溢出	`arith_oflow`	运算结果超过目标表达能力	较窄寄存器接收宽加法结果
移位溢出	`shift_oflow`	移位导致有效位全部移出	1 位左操作数移位 2 位
类型转换溢出	`cast_oflow`	转换后的有效位损失	带符号操作转换到窄寄存器
悬空三态总线	`floating_bus`	所有三态驱动均未启用	所有 `oe` 均为 0
多驱动三态总线	`multi_driver`	多个驱动同时有效	`oe0` 与 `oe1` 同时为 1

AEP 类型	`-aep` 选择值（按 User Guide）	要检查的危险行为	例子
分支覆盖完整性	`full_case`	case/if 组合未完整覆盖	没有 default 的分支遗漏
并行分支互斥	`parallel_case`	多分支同时匹配	重叠编码引起优先级问题
置位/复位问题	`set_reset`	相关控制状态及赋值组合危险	置位与复位同时发生
未知值赋值	`x_assign`	显式 X 进入逻辑	`assign out = 1'bx`
FSM 死锁	`fsm_deadlock`	状态无法继续转移	没有离开等待状态的路径
FSM 活锁	`fsm_livelock`	在循环中反复转移却不完成	两状态无限循环
FSM 相关额外检查	`fsm_unionlock`	与 FSM 联合锁定相关的规则	详见版本帮助

表格中的实际生成条件、适用语言和局限需以 UG §10.1.1（第 363 ~ 375 页）为准，不是所有检查都支持全部 Verilog、SystemVerilog 与 VHDL 语法。例如 User Guide 对 `cast_oflow` 的 VHDL 支持有单独限制。

3. 初学者第一个 AEP 案例：数组越界

3.1 完整设计源码

数组只有两个元素，但两位 `index` 能表示 0、1、2、3。这里故意保留未检查的索引访问；数组内容是否已初始化与本例的索引越界问题应分开分析。

保存为 案例工程/rtl/array_demo.sv：

```
// Deliberately has an index range wider than the 2-element memory.
module array_demo (
  input  logic       clk,
  input  logic [1:0] index,
  output logic [7:0] data_out
);
  logic [7:0] mem [0:1];
  always_ff @(posedge clk)
    data_out <= mem[index];
endmodule
```

如果接口规格确实允许任意两位索引，就应当由 RTL 拒绝非法访问。只有在接口规范明确禁止索引 2、3 时，才可以建立相应环境假设，不能为了消除反例任意限制输入。

3.2 完整文件列表

保存为 案例工程/filelist_aep.f：

```
./rtl/array_demo.sv
```

4. 标准运行步骤

4.1 最小数组检查脚本

先选择 bounds_check，再检查属性是否实际生成。没有生成属性不等同于数组安全。

保存为 案例工程/scripts/aep_bounds.tcl：

```
set_fml_appmode AEP
read_file -top array_demo -aep bounds_check -format sverilog -vcs {-f
↪  filelist_aep.f}
create_clock clk -period 100
check_fv_setup
report_fv -list
check_fv -block
report_fv -verbose
```

运行命令（Bash）：

```
# 从包含“案例工程”目录的上一级目录开始。
cd 案例工程
mkdir -p logs
set -o pipefail
vcf -f scripts/aep_bounds.tcl -batch 2>&1 | tee logs/aep_bounds.log
```

这些命令应在本机具备相应 VC Formal 版本和许可证时执行。日志中的属性状态才是证明结论；命令退出、脚本结束或没有打印错误都不能代替属性结果检查。

真实工程有复位的，应加上 create_reset 和必要的初始化序列；上述 array_demo 刻意只有时钟，突出数组检查的概念。若工具对该语言、数组形式或优化方式没有生成相应属性，先检查 -aep 语法与对象是否符合其抽取前提。

4.2 多种 AEP 组合

```
read_file -top my_top -aep fsm_deadlock+fsm_livelock \
  -format sverilog -vcs {-f filelist.f}
```

这里的 + 表示组合多个检查选择值，是指南中提供的一种常见写法；-aep all 并不意味着对每个设计都推荐一开始就开启所有检查。建议先对高优先级逻辑运行小集合，再逐步扩展，以便降低误报排查成本。

4.3 VHDL 的相关配置

```
set_app_var fml_enable_vhdl_aep true
set_app_var fml_enable_vhdl_bus_check true
set_app_var fml_enable_vhdl_cov true
```

这些应用变量来自该版本的快速参考，启用前确认当前分析模式确实为 VHDL 且变量在安装版本可用。VHDL 的语言结构与 SystemVerilog 不同，不能假定每一种 AEP 的覆盖条件完全相同。

5. 状态机检查：死锁、活锁与公平性

FSM 死锁一般指在给定约束下进入无法合理继续的状态；活锁指能够不停转移却始终不进入完成或期望状态。是否一定算设计缺陷，取决于环境前提：一个等待读取外设的状态如果允许外设永远不响应，可能是系统规格缺失，而非状态机 RTL 必然有错。

```
read_file -top controller -aep fsm_deadlock+fsm_livelock \
  -format sverilog -vcs {-f controller.f}
aep_generate -type fsm_fairness
# 可把候选公平性配置保存为脚本后审查
aep_generate -type fsm_fairness -tcl fairness.tcl
```

生成公平性相关条件时应由系统工程师审查其物理可实现性。例如“不停请求外部总线”并不能保证对方必然授权，除非系统规格明确了调度公平性。

5.1 控制某种状态的自动生成

```
aep_config -dont_generate -type fsm_livelock {state_reg::state_name}
```

该指令适用于已确认特定状态的自动生成规则不适当的情况；必须给出关闭原因和关联设计评审。示例 `state_reg::state_name` 只是占位对象，实际名称应从生成属性和设计状态列表取得。不要以大量关闭属性的方式解决真问题。

6. AEP 的命令作用与注意事项

指令	功能	注意
`set_fml_appmode AEP`	进入自动属性应用	编译前设置
`read_file ... -aep ...`	指定要从设计抽取的检查类型	类型和语言需要对应
`elaborate top -aep ...`	分步编译流程中生成 AEP	与 `analyze` 配对
`aep_generate -type fsm_fairness`	为 FSM 相关检查生成候选公平性设置	必须人工审查合理性
`aep_config -dont_generate ...`	对指定状态或检查关闭自动生成	需保存关闭理由
`fvdisable` / `fvenable`	调整已生成属性是否参加求解	不等同于修改 RTL
`fvwaive` / `fvunwaive`	将属性纳入/移出豁免	保留变更依据

指令	功能	注意
`check_fv`	求解启用的自动属性	按 `-block` 或报告确认运行完成
`report_fv -verbose`	检查详细结果	分析反例与真正危险性
`save_waiver_file -file aep.el`	保存豁免清单	提交审查版本
`read_waiver_file -elfiles aep.el`	导入豁免	注意 RTL 或属性名变化

7. 常见问题与处理方案

问题 A：数组越界属性失败。查索引值、数组实际维度、索引来源、是否为非法但允许输入；若设计应该防御非法访问，则修复输入检查和返回行为。**问题 B：`full_case` 告警**。检查组合分支是否覆盖所有合法编码；若某些编码应视为非法，先明确可达性，不能仅靠注释 `full_case` 掩盖。**问题 C：状态机活锁告警**。查看反例循环，确认是否需要外设公平假设。**问题 D：属性数为零**。检查 AEP 开关、RTL 类型和提取范围，不应推断“RTL 不存在问题”。

用例：审查豁免

```
fvwaive <property_name>
save_waiver_file -file reviewed_aep.el
```

建议豁免记录独立写明编号、原始现象、为何不是设计缺陷、所依据的系统规格、最后审查日期与责任人。`fvwaive` 只是结果管理，不会使设计行为变正确。

8. 推荐的 AEP + FPV 双阶段工作方法

先在整段 RTL 范围运行 AEP 发现基础实现风险；其次针对关键模块编写 FPV 属性以覆盖具体的时序与功能规格。AEP 发现的真缺陷，应补充成可长期回归的手工断言（如果合适）。AEP 发现的假阳性，应检查真实前提并保留说明；不可简单添加 `assume` 让它消失。

对于大型 SoC，可以先针对地址解码、寄存器文件索引、FIFO 状态机、宽窄运算及三态总线使用 AEP。对于自动生成属性无法证明的情况，可单独分组并提升资源，而不是影响所有检查的分析时间。

9. 官方参考

User Guide §10：第 362 ~ 385 页；§3.5 属性 class 与 type；§7 反例与状态；Quick Reference Guide 的 AEP 章节。本文检查类型的简述为独立归纳，并非逐页复制官方表格。

FXP：形式未知值传播验证完整指南

源码内嵌版。本章的完整案例以“保存为”标明文件路径，并在下方直接列出全文。代码框不带显示行号，可直接复制；**案例工程/** 只表示目录结构，不要求另找源码附件。包含占位符或保护性 `error` 的专用流程另行明确标为模板。

第 7 册 · 专用应用　Formal X-Propagation Verification。依据 User Guide 第 12 章（第 444 ~ 457 页）、Quick Reference Guide 的 FXP 页面。原有笔记中有较多全设计信号扫描与调试命令，本版将其整理为有风险说明的进阶用法。

1. 为什么未知值传播值得单独验证

RTL 仿真中的 X 可能表示：寄存器尚未复位、未驱动、赋值了 X、模块未定义、数组越界、选择条件不确定等。真实电路并不存在一种独立的“X 电压值”；它表示形式或仿真层面对确定性的缺失。在安全控制、启动流程、电源域切换、复位释放和错误路径中，一个无法确定的控制值可能在时序上被传播或放大。

FXP 的核心目标是寻找：**指定的未知值来源是否可能影响不应该出现未知值的观察信号**。它通过生成 X 注入点与观察点、建立检查并使用形式引擎寻找违反条件的轨迹。普通 FPV 的二值断言不能自动等同于这种 X 传播分析。



图 8: FXP 注入源和观察点

2. 注入点与观察点的概念

注入点是允许引入未知值的位置，例如未初始化寄存器、显式 X 赋值、未驱动信号、黑盒输出或人为指定的关键节点。**观察点**是要求未知值不应到达的位置，例如顶层功能输出、关键状态机控制端、黑盒输入和被指定为需要稳定可判定值的寄存器。

形式检查回答的是：在当前复位和输入条件下，能否找到一条路径，让注入的不确定性传播到观察点？它不是统计 X 出现概率，也不等于电路可靠性故障率分析。

3. FXP 自动生成检查的类型

UG §12.3（第 447 ~ 451 页）介绍了下列常见类型：

类型名	来源或目标	意义
`uninit`	未初始化寄存器	未复位状态可能携带未知值
`xassign`	显式 X 赋值	设计源代码主动产生不确定值
`undriven`	未驱动信号	网络没有确定驱动
`undef`	未定义运算	例如某些除零场景
`oba`	越界访问	数组等操作超出合法范围
`bbout`（按该版本帮助核对）	黑盒输出	未知结果可能进入下游
`bbin`	黑盒输入	观察未知值是否送入模块
`in` / `out`	顶层输入/输出	自定义源或观察目标
`snip`	经过截断的网络	被抽象网络引入不确定性
`abs`	抽象对象	抽象造成额外未知来源或观察点
`unresin` / `unresout`	未解析模块输入/输出	未解析实例两侧的传播

类型字符串以具体安装版本 `help fxp_generate` 和 UG 表 19 为准。不同来源给出的缩写有差异，不能仅凭表格猜测选项。`fxp_generate` 无 `-type` 时已包含若干默认类别；明确选择不同类别后，自动生成的属性数可能发生变化。

4. 标准脚本与运行顺序

```
set_fml_appmode FXP
read_file -top soc_top -format sverilog -vcs {-f filelist.f}
create_clock clk -period 100
create_reset rst_n -sense low
set_constant test_mode -value 1'b0
sim_run -stable
sim_save_reset
check_fv_setup
fxp_generate
report_fv -list
check_fv -block
report_fv -verbose
```

`fxp_generate` 使用默认类型自动提取检查。若没有复位的原型模块，应删除不适用的 `create_reset`，不能输入一个不存在的复位信号。任何被 `set_constant` 固定的控制输入都应有真实工程依据，否则可能人为切断未知值传播路径。

4.1 显式选择未初始化寄存器类型

```
fxp_generate -name uninitialized_checks -type {uninit}
```

只关注某类来源有利于快速建立第一批可解释的结果。UG 中还给出了将某个范围及若干类型一起配置的方式；例如只针对某个子模块，可传入对应的 scope 列表（需要真实实例路径）。

5. 自定义 X 注入、去除与断言

命令	作用	工程注意
`fxp_generate ...`	自动生成 X 检查	分类范围与默认类型需确认
`fxp_assume -injectx <signal>`	指定信号作为 X 注入来源	时钟与复位不允许当作该命令的注入源
`fxp_assume -nox <signal>`	在指定条件下认为目标不携带未知值	必须有硬件/协议依据
`fxp_assert <signal>`	指定目标信号不允许发生 X 传播	首先明确哪些运行状态必须已知
`fxp_compute_rootcause <property>`	尝试计算属性的未知来源原因	需要存在可用的分析结果
`fxp_report_rootcause`	报告来源与传播信息	按报告定位 RTL/层级

```
# 假设 input_data 确实是不受控外部输入，调试时人为注入 X
fxp_assume -injectx input_data

# 关键控制输出不允许成为未知
fxp_assert -name no_x_on_commit commit_valid
```

`fxp_assume -injectx` 的 `-reset`、`-oba`、`-xassign`、`-undef`、`-condition`、`-scope` 有类型与范围限制。例如 UG §12.3.2 提到 `-oba`、`-xassign` 和 `-undef` 需要相应 scope，并且对时钟/复位注入有限制。`fxp_assume -nox` 的条件参数在 PDF 的语法行存在排印不严谨之处，实际脚本应在本机通过 `fxp_assume -help` 校验，不能机械复制不完整的占位语法。

5.1 什么时候允许 `-nox`

只有当硬件结构、外部协议或正式初始化流程**保证该信号在指定阶段已知**时，才允许消除其 X 来源。例如某寄存器由上电复位明确赋值，且复位时序确实执行；或者某输入在完成训练后才被采样，且存在可证明的有效标志。如果将所有未知数据都用 `-nox` 约束掉，FXP 就失去实际作用。

6. 教学用例 A：未复位的数据寄存器

```
module uninit_demo (
  input  logic clk, rst_n, write,
  input  logic [7:0] in_data,
  output logic [7:0] out_data
);
  logic [7:0] cache;
  always_ff @(posedge clk) begin
    if (write)
```

```
      cache <= in_data;
  end
  always_comb out_data = cache;
endmodule
```

cache 不受 rst_n 复位，且复位后在第一次写入之前可以作为任意初值输出。若 out_data 必须始终有效，FXP 应提示风险。修复方式有两类：一是复位 cache，二是在 cache 内容有效前通过 valid 等硬件逻辑阻止下游采样，或输出一个确定值。**仅添加形式假设让 write 在第一拍发生，不一定等同于真实硬件有此保证**。

6.1 更合理的有效性控制

```
logic initialized;
always_ff @(posedge clk or negedge rst_n) begin
  if (!rst_n) initialized <= 1'b0;
  else if (write) initialized <= 1'b1;
end
assign out_data = initialized ? cache : 8'h00;
```

当 cache 尚未被写入时，输出为确定值；写入后使用缓存数据。FXP 仍要检查选择条件以及其他传播分支是否可能携带未知值。

6.2 本节完整工程文件

上面的片段展示问题原理。以下是同一类未初始化缓存案例的完整文件，故意保留复位端口却不复位缓存，用来说明“端口存在”并不意味着缓存有已知初值。

6.2.1 被验证的完整 RTL

保存为 案例工程/rtl/uninit_demo.sv：

```
// Purposefully uninitialized cache to illustrate FXP, not recommended RTL.
module uninit_demo (
  input logic clk, rst_n, write,
  input logic [7:0] in_data,
  output logic [7:0] out_data
);
  logic [7:0] cache;
  always_ff @(posedge clk)
    if (write) cache <= in_data;
  assign out_data = cache;
endmodule
```

6.2.2 完整文件列表

保存为 案例工程/filelist_fxp.f：

```
./rtl/uninit_demo.sv
```

6.2.3 完整 FXP 脚本

`fxp_generate` 选择未初始化对象，`fxp_assert` 将输出作为必须已知的观察位置。输入 `write` 没有被假设为始终有效，因此必须考虑尚未写入的路径。

保存为 案例工程/scripts/fxp_uninit.tcl：

```
set_fml_appmode FXP
read_file -top uninit_demo -format sverilog -vcs {-f filelist_fxp.f}
create_clock clk -period 100
create_reset rst_n -sense low
sim_run -stable
sim_save_reset
fxp_generate -name uninitialized_checks -type {uninit}
fxp_assert -name output_must_be_known out_data
check_fv -block
report_fv -verbose
```

运行命令（Bash）：

```
# 从包含“案例工程”目录的上一级目录开始。
cd 案例工程
mkdir -p logs
set -o pipefail
vcf -f scripts/fxp_uninit.tcl -batch 2>&1 | tee logs/fxp_uninit.log
```

这些命令应在本机具备相应 VC Formal 版本和许可证时执行。日志中的属性状态才是证明结论；命令退出、脚本结束或没有打印错误都不能代替属性结果检查。

预期观察是：未发生有效写入时，缓存的不确定性可能到达输出。分析失败应查看工具实际选择的注入点和反例，不应通过把 `write` 固定为 1 来掩盖初始化缺陷。这里没有真实 FXP 运行报告。

7. 教学用例 B：显式 X 赋值与越界

```
always_comb begin
  unique case (opcode)
    2'b00: result = a + b;
    2'b01: result = a ^ b;
    default: result = 'x;
  endcase
end
```

对未定义操作码，显式 X 可能被后级直接使用；`-type {xassign}` 可以帮助定位。若协议规定未支持操作码必须返回错误码或忽略写入，则直接赋 X 不是完整实现，应修改 RTL 和相应 SVA。如果设计规则明确规定非法操作码永远不会输入，则需要有可审查的上游保证，不能只对当前检查添加无限制输入假设。

对数组 `mem[index]` 的非法访问则属于 `oba` 等相关检查。在一些 RTL 里，越界索引导致的 X 传播既可被 AEP 发现，也可通过 FXP 分析其实际影响的观察点，两者结论不同、相互补充。

8. 教学用例 C：有意屏蔽不确定数据

```
assign packet_out = valid ? raw_packet : '0;
```

如果 valid 的生成完全可靠且只有 valid=1 时 raw_packet 才被使用，则未初始化的 raw_packet 未必造成危险输出。FXP 应在考虑 valid 的实际来源和复位行为后判断其是否能够影响 packet_out；不要只凭 raw_packet 未复位就断定必须增加复位网络。对于大面积数据寄存器，**有效位优先**的方案往往比强行复位所有触发器更合理，但前提是证明有效位本身可靠，且其控制可完全屏蔽未知数据。

9. 调试：由下游症状回溯到上游来源

```
check_fv -block
report_fv -list
view_trace -property <failed_property>
fxp_compute_rootcause <failed_property>
fxp_report_rootcause
```

定位时先找观察点在何时受影响，然后从该周期往前追踪经过的条件选择、多路复用器、寄存器和复位逻辑。重点确认是否存在：复位未覆盖、输出在有效位之前被采样、黑盒建模不当、未知模式输入未处理、未驱动网络或隐含时钟门控。必须把“真正产生不确定性”和“最终使不确定性可见”分开分析。



图 9: FXP 调试排查流程

10. 设计规模很大时怎么做

先针对关键输出与重要控制节点建立一批观察点；其次根据报告只针对相关模块开启额外的注入类型；最后扩大到全模块或全芯片。全信号扫描往往生成大量属性，可能包含没有工程意义的中间节点和普通数据寄存器，因此不推荐把 get_signals 的全部对象直接无差别传给 fxp_assert，尤其不推荐在没有容量预算和报告分类时执行。若需批量生成对象列表，应该过滤时钟、复位、扫描测试、非功能占位和已知允许变化的信号，并在最终报告中清晰列出排除规则。

10.1 与复位覆盖工作的关系

FXP 可帮助识别“实际运行时需要已知，但没有完整初始化”的重要状态，但**不能单独替代复位规格验证**。建议同时使用 FPV 证明复位之后关键控制寄存器为规定的常量。这样发现的 X 传播问题可以分成：状态未初始化、控制路径泄漏、协议保护失效、或仿真 X 语义不同。

11. 常见问题

现象	可能原因	改进方向
复位后一拍输出为 X	数据寄存器未复位或有效位错误	修改 RTL 或协议保护
某黑盒输出引起所有下游失败	模型过于自由	提供正确的黑盒约束或可综合模型
所有属性都通过但已知存在 X 问题	未将关键输出列为观察点或注入范围不足	核对 `fxp_generate` 类型及属性数量
生成属性太多，内存不足	扫描范围过宽	按重要模块、输出和原因分批
`-injectx` 报错	选中时钟/复位或不支持对象	查该命令对象与 scope 限制
人工 `-nox` 后全部通过	可能屏蔽真实传播	审查硬件依据，保留原始对照结果

12. FXP 与 AEP、FPV 的分工

- AEP 可自动检查“数组是否越界、表达式是否存在潜在不安全结构”；
- FXP 检查“潜在未知值是否影响重要观察点”；
- FPV 检查“满足输入规范时，关键输出值或控制状态是否符合功能规格”。

例如越界读导致数据可能不确定，三个应用依次能够定位越界原因、传播风险、以及实际功能规则失败。联合使用更全面，但三者仍依赖正确的模型和约束。

13. 官方依据与版本提醒

User Guide 第 12 章（444 ~ 457 页），Quick Reference Guide FXP 部分。`set_blackbox` 的开关应以安装版本帮助为准，部分旧笔记中存在 `-design` 与 `-designs` 混用；不要把未核对的选项直接写入生产脚本。源码形式验证与真实芯片的亚稳态、模拟电压和物理故障不是同一个模型。

CC：形式连通性检查完整指南

源码内嵌版。本章的完整案例以“保存为”标明文件路径，并在下方直接列出全文。代码框不带显示行号，可直接复制；**案例工程/** 只表示目录结构，不要求另找源码附件。包含占位符或保护性 `error` 的专用流程另行明确标为模板。

第 8 册 · 专用应用　Connectivity Checking。依据 User Guide 第 13 章（第 458 ~ 516 页）及 Quick Reference Guide CC 部分。

1. 这项应用解决的问题

大规模 SoC 有大量中断线、复位线、测试模式信号、控制总线、调试线和模块间的数据连接。人工核对顶层 RTL 不仅费时，也难以覆盖条件选择、多级复用和逻辑变换。CC 通过预先声明源、目的、使能及延迟条件，验证设计的结构或功能上是否存在符合要求的传播关系。

结构连通性关注从源到目的的候选有向路径是否存在；**功能连通性**进一步检查在规定使能、初始状态和时序条件下，目的值是否满足规格。结构上有路径并不能证明功能上始终正确；功能检查还需具体考虑控制条件。

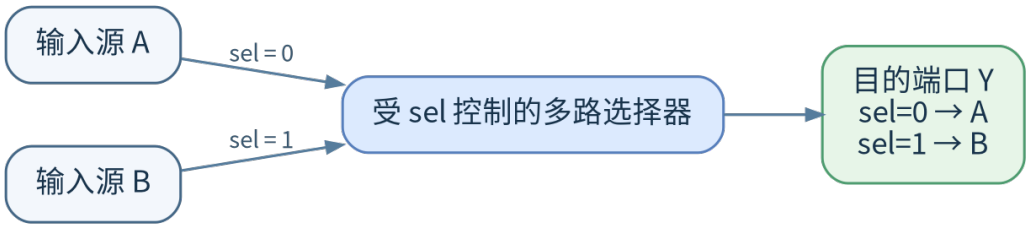


图 10: CC 的源、选择控制与目的端

2. 连通性规格包含哪些信息

User Guide §13.2 将一个基本连通性要求分解为：名称、源 `src`、目的 `dest`、使能表达式、路径延迟 `path_delay`、使能保持时间 `enable_hold`、使能延迟 `enable_delay`、起始偏移 `offset_delay` 和时钟。常用连接的延迟为 0，并不代表全部 SoC 连接都零延迟。

字段	要解决的具体问题	示例说明
name	这一连接叫什么	`cc_irq0`
src	信号从哪里来	`irq_in[0]`
dest	到哪里去	`irq_out[0]`
enable	何时应该生效	`sel == 0`
path_delay	源到目的要经过几个时钟	0 或固定延迟
enable_hold	使能需要保持的周期范围	与数据到达时机有关
enable_delay	使能相对源信号何时有效	可为 0 或负偏移
offset_delay	距离初始化的检查起点	排除合法启动窗口
clock	延迟检查采用哪个时钟	`clk`

不能用“检查有线路连接”概括含使能的连通性：例如一个由 `sel` 控制的多路选择器，在 `sel=0` 时源 `a` 应到 `y`，在 `sel=1` 时源 `b` 应到 `y`，这属于两条具有不同使能的规格。

3. 最小实例：多路选择器

3.1 完整多路选择器源码

此处没有寄存器，输出完全由当前 `sel/a/b` 决定。

保存为 案例工程/rtl/mux2.sv：

```
module mux2 (
  input logic a, b, sel,
  output logic y
);
  assign y = sel ? b : a;
endmodule
```

对应两条连接规格：当 `sel=0` 时 `a` 与 `y` 相同；当 `sel=1` 时 `b` 与 `y` 相同。可以在 Tcl 中创建：

```
set_fml_appmode CC
read_file -top mux2 -format sverilog -vcs {./rtl/mux2.sv}
add_cc -src a -dest y -enable {!sel}
add_cc -src b -dest y -enable {sel}
check_fv -block
report_fv -list
```

`add_cc` 的 `-enable` 语法应以当前安装帮助为准；如果本版本对复杂表达式格式有限制，可改成 CSV 规格导入。**该脚本是展示连接语义的教学骨架，运行前须按安装版本补充完整的编译和检查配置。**

4. CSV 导入：大规模 SoC 的推荐方法

CSV 可以由规格表维护，从模块接口说明或端口表生成。依据 UG 第 461 ~ 464 页，默认字段顺序从属性名、源、目的、使能、路径延迟开始。一个简单 CSV 如下：

4.1 完整 CSV 连接规格

本例使用名称、源、目的、使能、路径延迟五列。第一行是说明性注释；是否接受这一注释形式仍需按安装版本核对。两条连接覆盖两个选择条件。

保存为 案例工程/cc_connections.csv：

```
// format: name,src,dest,enable,path_delay
cc_mux_a,a,y,!sel,0
cc_mux_b,b,y,sel,0
```

4.2 CSV 方式的完整运行脚本

直接读入上节 RTL 和本节 CSV；不需要隐藏的 `filelist.f`。先查看 `report_load_cc`，确认两条连接均成功加载，再分析形式结果。

保存为 案例工程/scripts/cc_mux.tcl：

```
set_fml_appmode CC
read_file -top mux2 -format sverilog -vcs {./rtl/mux2.sv}
load_cc -format csv cc_connections.csv
report_load_cc
check_fv -block
report_fv -verbose
```

运行命令（Bash）：

```
# 从包含“案例工程”目录的上一级目录开始。
cd 案例工程
mkdir -p logs
set -o pipefail
vcf -f scripts/cc_mux.tcl -batch 2>&1 | tee logs/cc_mux.log
```

这些命令应在本机具备相应 VC Formal 版本和许可证时执行。日志中的属性状态才是证明结论；命令退出、脚本结束或没有打印错误都不能代替属性结果检查。

对于跨组织的大规模规格，建议保留完整的九个核心字段并明确表头与默认值。`load_cc -default_param_order` 对应默认列顺序，`load_cc_set_param` 可定义自定义列对应关系，如 `load_cc_set_param csv_src "%2%"`；如果使用自定义格式必须随规格表一起提交，防止 CSV 列移动造成检查错位。

4.3 延迟连接的限制

CC 支持与时钟关联的连接延迟；路径中可能有寄存器，也可能由使能控制时序组合。验证之前需要指定准确时钟并检查 `path_delay` 的解释。不要把源和目的之间的**物理布线延迟**误认为 `path_delay=1`；这里描述的是形式模型中按采样时钟计算的功能延迟。多时钟跨域连接不能简单用一条普通单时钟延迟规格代替 CDC 分析。

5. 源目的别名与设计层级

实际 SoC 规格名称通常与 RTL 层级名不同，可以使用别名处理：

```
set_cc_alias CPU_IRQ u_top.u_cpu.irq_in
report_cc_alias
# 必要时删除已失效的别名
delete_cc_alias CPU_IRQ
```

别名使规格表与层级变动解耦，但必须建立审查规则：别名应对应单一、明确的对象或合法总线片段；层级优化后应重新检查别名的解析结果。

6. 专属命令与报告

命令	作用	主要核查点
`set_fml_appmode CC`	选择连通性检查	模式正确
`add_cc -src ... -dest ...`	添加单条连接规格	信号、方向、条件
`load_cc -format csv ...`	批量导入连接表	CSV 默认列或自定义列
`load_cc_set_param`	指定 CSV 的列含义	格式版本必须固定
`report_load_cc`	报告导入及结构检查	丢失信号、缺路径、无效规格
`report_fv -formatCC ...`	输出 CC 专属格式	具体 `csv/tcl/path` 选项版本对应
`list_path -src ... -dest ...`	查询网络之间路径	只代表连接搜索结果
`view_schematic -prop ...`	在原理图中查看检查关联	对照条件和路径
`generate_cc ...`	自动辅助提取连接	自动提取结果需人工确认规格
`compute_cc_cov`	计算相关连通性覆盖	需正确打开覆盖模式
`save_cc_cov_results`	保存覆盖分析结果	可追溯到已启用规则

7. 三种常见用例

用例一：复位信号分发。检查顶层复位能否经过相关复位控制逻辑到达每个子模块的对应复位端；如经过同步复位器，不要错误地要求零拍直接等值。**用例二：中断选择器**。按每个路由配置值检查中断来源与目的的条件连接，另外独立检查未使能中断不应错误出现在输出端。**用例三：测试模式控**

制。检查正常模式 `test_mode=0` 时功能信号未被测试逻辑截断，同时要避免把受硬件控制的测试状态误假设成永久关闭。

8. 失败分类与排查

结构不连通：检查层级、拼写、位选择、网表优化、黑盒以及真正是否有从源到目的的组合/时序路径。**结构有路径但功能失败**：检查控制使能、复位时序、源目的位宽、数据翻转关系、被插入的反相器或优先级。**不可适用或无意义的连接**：确认目的是否可观察、是否被固定常量、规则是否超出所支持的延迟/时钟形式。

不会因为报告中某条连接显示 `Connected` 就证明整个 SoC 的功能行为。CC 只覆盖已经列出的连接规则，规格完整性需要接口所有者审核。

9. 对大型项目的建议

按接口类别分 CSV 文件（时钟、复位、中断、扫描、调试、寄存器地址控制等），为每行保留规格编号及责任模块。批处理时针对真实的错误分类输出 `report_load_cc` 与 `report_fv`。将结构不连通与功能不符合分开处理；以保留全部输入列表的方式生成导出报告，便于发现某个连接根本没有被加载的情况。

10. 参考

User Guide §13（458 ~ 516 页），尤其 §13.2（460 页起）、CSV 格式说明（461 ~ 465 页）、连接延迟与覆盖章节；Quick Reference Guide CC 章节。低功耗场景另见 FLP_LPC.md。

FCA：形式覆盖率分析完整指南

源码内嵌版。本章的完整案例以“保存为”标明文件路径，并在下方直接列出全文。代码框不带显示行号，可直接复制；**案例工程/** 只表示目录结构，不要求另找源码附件。包含占位符或保护性 `error` 的专用流程另行明确标为模板。

第 9 册 · 专用应用　Formal Coverage Analyzer。依据 User Guide 第 11 章（第 386 ~ 443 页），Quick Reference Guide FCA 部分。**重要：该版本 User Guide 使用 Tcl 模式 `COV`，而应用名称缩写为 FCA**。

1. FCA 与 FPV 的区别

FPV 证明某一条属性是否成立；FCA 侧重理解覆盖目标的可达性，帮助分析仿真中未覆盖的代码和功能点是否在现有约束下真正不可能触发。典型情况：随机回归得到某分支从未执行，是因为测试不够、环境限制过严，还是这个分支从设计初始状态根本不可到达？FCA 可以在适当的逻辑模型中研究可达性，并为覆盖率排除提出依据。

FCA 的结果与 VCS 的覆盖数据通常相配合，常涉及 `simv.vdb`、排除文件 `.el` 和 `save_covdb` 产生的覆盖数据库。**不是所有未覆盖都应该豁免**，覆盖目标在形式模型下不可到达，也不代表目标一定不对应产品真实行为；模型和假设可能过度限制。

2. 支持的主要覆盖对象

覆盖类型	检查内容	典型缺陷或用途
Line	行覆盖目标是否可能执行	死代码或条件永不满足
Branch	if/case 分支是否可能发生	分支遗漏、错误优先级
Condition	布尔条件组合是否可到达	测试缺失或过度约束
Toggle	位信号 0↔1 翻转可达性	模式控制/死信号
FSM	状态及状态转移是否可达	死状态和隐藏转移
Functional coverage（视版本与模型）	覆盖组定义的目标	需确认模型、绑定与有效条件

不同覆盖目标的生成与排除规则并不完全相同。某些类型还需要额外变量启用，例如 `set_fml_var fml_cov_enable_branch_cov true`。

3. 第一个 FCA 用例：输入条件与代码分支可达性

3.1 完整设计源码

本例没有限制 mode，因此四种输入组合都允许出现。分析其可达性不是预先宣布某个分支不可到达。

保存为 案例工程/rtl/coverage_demo.sv：

```
// FCA exercise: branch conditions and reachability.
// mode is intentionally unconstrained, unless the verification environment
// supplies an explicitly reviewed operating-mode assumption.
module coverage_demo (
  input  logic       clk,
  input  logic       rst_n,
  input  logic [1:0] mode,
  output logic [1:0] result
);
  always_comb begin
    result = 2'd0;
    case (mode)
      2'd0: result = 2'd1;
      2'd1: result = 2'd2;
      2'd2: result = 2'd3;
      default: result = 2'd0;
    endcase
  end
endmodule
```

如果真实系统规范要求 mode[1] 恒为零，另外两个模式在该约束下才不可达；必须记录约束出处，并区分真实限制与为了提高覆盖数字而新增的假设。组合分支能否保留并产生检查，还受编译优化和覆盖类型配置影响。

3.2 完整文件列表

保存为 案例工程/filelist_fca.f：

```
./rtl/coverage_demo.sv
```

4. 主要流程：完整 Tcl

4.1 完整运行脚本

此版本按 User Guide 使用 COV 模式。脚本保留检查设置和结果报告；没有在未审查结果时自动把所有未完成目标导出为排除项。

保存为 案例工程/scripts/fca_coverage.tcl：

```
# Formal Coverage Analyzer uses COV appmode in X-2025.06-SP1 User Guide.
```

```
# Run from example project root; treat result as tool-dependent.
set_fml_appmode COV
read_file -top coverage_demo -format sverilog -cov all -vcs {-f filelist_fca.f}
create_clock clk -period 100
create_reset rst_n -sense low
sim_run -stable
sim_save_reset
check_fv_setup
check_fv -block
report_fv -verbose
```

运行命令（Bash）：

```
# 从包含“案例工程”目录的上一级目录开始。
cd 案例工程
mkdir -p logs
set -o pipefail
vcf -f scripts/fca_coverage.tcl -batch 2>&1 | tee logs/fca_coverage.log
```

这些命令应在本机具备相应 VC Formal 版本和许可证时执行。日志中的属性状态才是证明结论；命令退出、脚本结束或没有打印错误都不能代替属性结果检查。

需要导出经过审查的不可达候选时，再按实际会话执行：

```
save_cov_exclusion -file formal_unreachable.el
```

`-cov all` 的覆盖类型仍须核对版本，某些类型有单独的启用变量。应先确认生成了哪些目标、目标数量以及实际完成状态，再解释覆盖率或排除文件。

5. 从仿真数据库开始

```
set_fml_appmode COV
read_covdb -cov_input simv.vdb -cov_dut tb.dut
read_file -top dut -format sverilog -cov all -vcs {-f filelist.f}
create_clock clk -period 100
create_reset rst_n -sense low
sim_run -stable
sim_save_reset
check_fv -block
save_covdb -status covered -name formal_result.vdb
save_cov_exclusion -file candidate_exclusion.el
```

`-cov_dut` 必须是仿真数据库中的真实 DUT 实例路径。若层级不匹配，生成的排除记录可能关联错误。UG 第 395 页说明，无仿真数据库时的排除对象与有仿真数据库时的层级来源不同；最终在覆盖可视化工具中导入 `.el` 之前需要确认层级。

6. 命令分类

命令	作用
`set_fml_appmode COV`	选择 User Guide 认可的覆盖应用 Tcl 模式
`read_covdb -cov_input ...`	导入仿真覆盖数据库
`read_file ... -cov ...`	为指定类型创建覆盖目标
`check_fv`	对目标做形式检查
`report_fv -verbose`	显示覆盖检查状态
`save_cov_exclusion -file ...`	导出候选不可达排除说明
`save_covdb -status covered -name ...`	保存覆盖状态结果
`view_coverage`	打开覆盖查看界面
`cov_config -auto_scale`	配置自动分配覆盖分析任务
`compute_cov`	启动有关覆盖计算流程
`cov_config -line_task/-tgl_task ...`	调整各类型任务比例

7. Auto Scale 及资源策略

当覆盖目标数量巨大时，FCA 可自动划分多个覆盖类型和模块任务。先运行一个小范围测试估计内存与每类目标的数量，再考虑 `cov_config -auto_scale`、`-line_task`、`-fsm_task`、`-tgl_task` 等参数。不要在不了解网格队列和许可证预算的情况下照搬快速指南中几十或上百个 worker 的示例。汇总结果时，应说明每一类型完成、不可达和未完成的数量。

8. 排除文件需要审查

“不可达” 至少可能来自三种情形：设计逻辑确实使其不可达；复位/环境约束使其不可达；模型存在缺失使其无法触发。前者可能是冗余 RTL，也可能是特定配置下合法不可达；后两者则不能简单纳入永久覆盖排除。建议每条排除记录带上规格版本、环境限制和审批人。

一个完整的报告应区分仿真原始覆盖率、形式分析发现的不可达目标、审查后允许排除的目标、仍需增加激励的可达目标。不能把形式分析结果当成单纯提高覆盖率百分比的手段。

9. 常见排查

现象	检查事项
无覆盖属性	`-cov` 类型、模式、文件展开、分支额外开关
导出的排除文件和仿真层级不一致	`-cov_dut`、顶层名、编译优化
许多目标显示不可达	复位、环境常量、黑盒与约束
计算耗时非常长	按模块、覆盖类型拆分任务；使用 Auto Scale
`.el` 无法被查看器识别	数据库版本和导入路径

10. 命名差异和官方参考

X-2025.06-SP1 版本的 Quick Reference Guide 使用 `set_fml_appmode FCA`，但 User Guide §2（65 页）及 §11.2.2（395 页）使用 `set_fml_appmode COV`。本手册选择 User Guide 的 `COV` 作为示例；安装环境如存在别名或不同版本行为，以 `set_fml_appmode -help` 和 `get_fml_appmode` 确认为准。资料：User Guide 第 11 章及 Quick Reference Guide FCA 页。

SEQ：顺序等价检查完整指南

源码内嵌版。本章的完整案例以“保存为”标明文件路径，并在下方直接列出全文。代码框不带显示行号，可直接复制；**案例工程/** 只表示目录结构，不要求另找源码附件。包含占位符或保护性 `error` 的专用流程另行明确标为模板。

第 10 册 · 专用应用　Sequential Equivalence Checking。依据 User Guide 第 14 章（第 517 ~ 611 页），Quick Reference Guide SEQ 部分。

1. 为什么不能只比较组合逻辑

RTL 优化常常改变状态寄存器数量、编码、流水线或时钟门控。如果参考版本和实现版本内部结构不同，但对外部可观察输入输出仍符合约定，则二者可能功能等价。SEQ 比较两个顺序系统：参考模型（Specification）和实现模型（Implementation），通过寻找状态对应关系、输入关系和输出关系来检查其等价性。

SEQ 可用于 RTL 重构、状态机重编码、寄存器重定时、时钟门控优化以及某些微架构变化。需要明确：**等价的定义离不开观测点、初始状态、输入条件与时间对应关系**。如果实现比参考多一拍输出延迟，未经处理的逐周期相等检查可能失败，而根据允许的事务延迟重写比较规则后则可能成立。

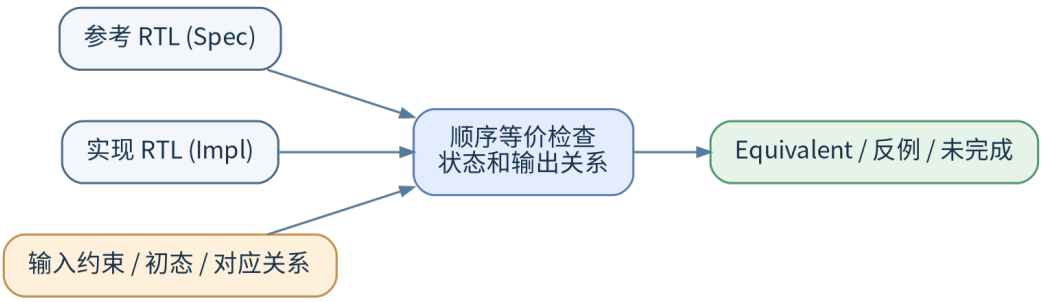


图 11: SEQ 两个版本的比较

2. 初学者例子：两种计数器写法

两边都是四位模 16 计数器；不是 FPV 章的模 10 计数器。接口相同，代码组织不同。

2.1 参考实现完整源码

使能时加一，不使能时通过未赋值保持状态。

保存为 案例工程/rtl/counter_spec.sv：

```
module counter_spec(input logic clk, rst_n, en, output logic [3:0] q);
  always_ff @(posedge clk or negedge rst_n)
    if (!rst_n) q <= 4'd0;
    else if (en) q <= q + 4'd1;
endmodule
```

2.2 待比较实现完整源码

将使能扩展为四位加数；为零时相当于保持，为一时相当于递增。

保存为 案例工程/rtl/counter_impl.sv：

```
module counter_impl(input logic clk, rst_n, en, output logic [3:0] q);
  always_ff @(posedge clk or negedge rst_n)
    if (!rst_n) q <= 4'd0;
    else        q <= q + {3'b000,en};
endmodule
```

2.3 故意不等价版本

同样使用 counter_impl 作为模块名，但使能时加二。正确版与错误版只能选一个加入本次比较。

保存为 案例工程/rtl/counter_impl_bad.sv：

```
// 故意错误版本：模块名仍为 counter_impl；不要与正确版本同时编译。
module counter_impl(input logic clk, rst_n, en, output logic [3:0] q);
  always_ff @(posedge clk or negedge rst_n)
    if (!rst_n) q <= 4'd0;
    else        q <= q + (en ? 4'd2 : 4'd0);
endmodule
```

在合法初始化和对应关系下，前两种写法预期等价，第三种则可在使能后产生差异。预期结果不代替实际的工具证明。

3. 编译两个独立设计库并运行

3.1 正确实现的完整 SEQ 脚本

参考设计放入 spec 库，待比较设计放入 impl 库。此脚本直接列出 RTL 路径，没有额外未展示的文件列表。map_by_name 后必须查看对应结果，不能认为名字相同就完成了初始化审查。

保存为 案例工程/scripts/seq_counter.tcl：

```
set_fml_appmode SEQ
analyze -format sverilog -library spec -vcs {./rtl/counter_spec.sv}
analyze -format sverilog -library impl -vcs {./rtl/counter_impl.sv}
elaborate_seq -spectop counter_spec -impltop counter_impl
create_clock clk -period 100
create_reset rst_n -sense low
map_by_name
report_seq_mappings
check_fv_setup
check_fv -block
report_fv -list
```

3.2 错误实现的完整 SEQ 脚本

只替换待比较侧的源文件，参考侧和环境条件不变。

保存为 案例工程/scripts/seq_counter_bad.tcl：

```
set_fml_appmode SEQ
analyze -format sverilog -library spec -vcs {./rtl/counter_spec.sv}
analyze -format sverilog -library impl -vcs {./rtl/counter_impl_bad.sv}
elaborate_seq -spectop counter_spec -impltop counter_impl
create_clock clk -period 100
create_reset rst_n -sense low
map_by_name
report_seq_mappings
check_fv_setup
check_fv -block
report_fv -list
```

运行命令（Bash）：

```
# 从包含“案例工程”目录的上一级目录开始。
cd 案例工程
mkdir -p logs
set -o pipefail
vcf -f scripts/seq_counter.tcl -batch 2>&1 | tee logs/seq_counter.log
vcf -f scripts/seq_counter_bad.tcl -batch 2>&1 | tee logs/seq_counter_bad.log
```

这些命令应在本机具备相应 VC Formal 版本和许可证时执行。日志中的属性状态才是证明结论；命令退出、脚本结束或没有打印错误都不能代替属性结果检查。

这里保留原有教学脚本的初始化框架；实际项目应检查两个设计的复位状态和初始状态关系，不能把 `create_reset` 或名称对应当作“任意初态自动一致”。如需额外复位激励，应按 SEQ 的本地运行流程补充并记录。

4. 对应关系是什么

匹配可基于相同名称的输入、输出和部分状态，常用 `map_by_name`，其结果可通过 `report_seq_mappings` 审查；更复杂时可用 `seqmap`、`sequnmap` 或读入人工编写的匹配文件。不能仅因为“映射出来很多对象”就认为匹配一定正确；应特别检查复位极性、寄存器位宽、符号扩展、被优化掉的状态和不同流水线阶段的关系。

4.1 未初始化位的影响

若参考版和实现版的寄存器初态均任意，两者不一定要求从任意**独立**初态开始都立刻一致：合理的等价证明需要定义可接受的初始状态关系。将所有未初始化位置零的 `seq_config -map_uninit -map_x zero` 是该版本示例中的一种选择，但只在硬件上电模型允许时使用，否则可能忽略真实启动差异。

5. 顺序优化类型与对应关系

改动	主要挑战	需要关注的比较条件
组合表达式重写	表达式结构变化	相同输入下输出是否同拍一致
状态机重新编码	状态变量命名与编码变化	状态关系、输出一致性
加入时钟门控	等效使能与采样行为	门控使能关闭时是否保持
寄存器重定时	状态位置与延迟变化	正确建立跨状态对应
流水线深度改变	输入到输出周期可能变化	确认规格是否允许变化
微架构重构	内部状态数量可能不同	匹配可观察事务行为

6. 主流程命令说明

命令	作用
`set_fml_appmode SEQ`	切换顺序等价检查
`analyze -library spec/impl`	分别编译两个版本
`elaborate_seq`	创建双设计比较环境
`seq_config`	指定检查策略和特殊匹配行为
`map_by_name`	基于名称建立候选对应
`seqmap` / `sequnmap`	人工调整比较关系
`seq_read_mapping_file`	导入外部匹配文件
`report_seq_mappings`	查看已采用的对应关系
`check_fv`	执行等价检查
`report_proofs`	汇总证明对象
`set_debug` / `seq_debug_report`	对失败证据开展专门分析
`resume_seq`	恢复可继续的等价证明过程

7. 如何解释不等价结果

首先判断是否为真实对外行为差异，然后才检查内部状态。推荐按如下顺序：确定失败的输出或检查点；核对两设计的输入在同一采样时刻是否一致；检查复位起始状态；检查 X 与未初始化值策略；检查流水线延迟或时钟门控；最后追踪内部状态分歧最早发生的位置。

三种不同结论不能混同：一是明确找到了两者输出行为不一致的反例；二是当前资源下无法证明；三是当前对应关系或抽象造成局部失败。若初始状态和外部协议前提建错，需要修复比较环境而不是简单修改 RTL。

8. 高级优化策略

对于寄存器重定时、多时钟门控和大型算术逻辑，可考虑 `seq_config -recipe orch_generic_medium`、`set_fml_var fml_effort high`、`set_seq_abstractions ...` 和分组证明。抽象可能缩小计算量，也可能引入新反例，需要再验证其可实现性。

对复杂关系建议分步骤：证明复位和普通模式的基础等价，确认输入输出映射，再处理新增加的内部状态及特殊操作模式。不要将所有复杂证明一次交给自动命名对应而忽略潜在误配。

9. 教学练习

1. 证明 `en ? q+1 : q` 与 `q+en` 在指定位宽下是否一致；注意表达式位宽。
2. 修改实现版，使输出晚 1 拍，观察直接逐拍比较为什么失败。
3. 在实现版增加一个无功能影响的寄存器，分析映射报告和求解结果。
4. 把初始状态改为未定义，比较是否仍成立，并说明真正的硬件上电规格。

10. 参考

User Guide §14（517 ~ 611 页），尤其设计编译、映射与调试部分；§18 的复杂性分析；Quick Reference Guide SEQ 章节。对比较多周期事务而非逐拍行为的需求，也可对照 DPV.md。

FRV：形式寄存器验证完整指南

源码内嵌版。本章的完整案例以“保存为”标明文件路径，并在下方直接列出全文。代码框不带显示行号，可直接复制；**案例工程/** 只表示目录结构，不要求另找源码附件。包含占位符或保护性 `error` 的专用流程另行明确标为模板。

第 11 册 · 专用应用　Formal Register Verification。依据 User Guide 第 16 章（第 619 ~ 685 页）、Quick Reference Guide FRV 部分。

1. 典型目标和输入资料

SoC 中寄存器字段数量多，容易出现复位值错误、读写权限错误、写一清零行为错误、保留位处理错误、地址别名或总线包装接口问题。FRV 通过寄存器说明文件（IP-XACT XML、RALF 等）提取字段规则，产生并运行形式检查；其价值在于减少重复手工编写针对每个寄存器位的测试与断言。

输入至少包括真实 RTL、寄存器规格文件、总线协议适配及形式环境。特别注意：寄存器文件包含的**理想行为**必须与设计真实的事务接口含义一致；仅有 XML 不会使工具自动猜到哪一条总线读写应对应哪个寄存器。

2. 核心概念

寄存器属性	预期行为	应验证的例子
Reset value	复位后默认值	配置字段复位为零
RW	可读可写	写入已接受后读回一致
RO	只读	总线写入不能改变状态
WO	只写	读取行为应符合规格定义
W1C	写 1 清零	写 1 清对应位，写 0 不清
W1S	写 1 置位	写 1 置位，其他位不变
Reserved	保留位	写入是否被忽略、读取返回什么
Volatile	硬件可以自主更新	不要求所有周期保持软件写入值
Side effect	读取或写入有副作用	读清除、中断状态操作

字段策略由寄存器规范决定。不要把没有发生读写握手的周期当作已接受事务，也不能把硬件会自动变化的计数器字段视作永远保持的软件配置值。

3. 寄存器规格与 RTL 的对照

例如有 CTRL（地址 0x00，位 0 使能，RW）、STATUS（地址 0x04，位 0 完成，W1C）。典型教学实现是：接受对 CTRL 地址的写入后更新使能位；STATUS.done 由硬件事件置位、软件向对应位写 1 清除；异步复位全部清零。对应检查至少覆盖默认值、RW 正常写入、W1C 保持和清零，以及无效地址的处理。

FRV 的自动检查可以缩小手工工作量，但对多写入源冲突、硬件置位和软件清零同周期优先级等复杂语义，通常仍需要补充明确的 SVA。假定硬件事件优先级高于 W1C，需要实现与规格一致的条件判断。

3.1 完整教学实现与手写属性

以下把上述规格写成可以独立保存的 RTL 和 SVA。接口不是 APB/AXI：wr_en=1 的上升沿立即接受写入，读取是由 rd_en 控制的组合输出；无字节使能，无等待周期。这样可以先理解 RW、W1C 与硬件置位优先级，再接入真实总线。

3.1.1 寄存器 RTL

保存为 案例工程/rtl/reg_block_demo.sv：

```
// 单周期教学寄存器接口，不是 APB/AXI 适配器。
// 上升沿 wr_en=1 即接受写入；rd_en=1 时组合读出。
module reg_block_demo (
  input  logic        clk, rst_n,
  input  logic        wr_en, rd_en,
  input  logic [7:0]  addr,
  input  logic [31:0] wdata,
  input  logic        hw_done,
  output logic [31:0] rdata,
  output logic        ctrl_enable, status_done
);
  localparam logic [7:0] CTRL_ADDR   = 8'h00;
  localparam logic [7:0] STATUS_ADDR = 8'h04;

  always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
      ctrl_enable <= 1'b0;
      status_done <= 1'b0;
    end else begin
      // CTRL[0] 为 RW，其他位忽略写入。
      if (wr_en && addr == CTRL_ADDR)
        ctrl_enable <= wdata[0];

      // 硬件置位优先于软件写一清零；写零保持原状态。
      if (hw_done)
        status_done <= 1'b1;
      else if (wr_en && addr == STATUS_ADDR && wdata[0])
```

```
        status_done <= 1'b0;
    end
  end

  always_comb begin
    rdata = 32'd0;
    if (rd_en) begin
      case (addr)
        CTRL_ADDR:   rdata = {31'd0, ctrl_enable};
        STATUS_ADDR: rdata = {31'd0, status_done};
        default:     rdata = 32'd0;
      endcase
    end
  end
endmodule
```

3.1.2 完整检查器

W1C 检查只有在上一拍没有硬件事件时才要求清零；如果硬件事件和软件写清除同时发生，本例明确要求硬件置位优先。

保存为 案例工程/formal/reg_block_demo_props.sv：

```
module reg_block_demo_props (
  input logic clk, rst_n, wr_en, rd_en, hw_done,
  input logic [7:0] addr,
  input logic [31:0] wdata, rdata,
  input logic ctrl_enable, status_done
);
  logic history_valid;
  always_ff @(posedge clk or negedge rst_n)
    if (!rst_n) history_valid <= 1'b0;
    else        history_valid <= 1'b1;

  // 当前采样值应等于上一拍已接受写事务的结果。
  ap_rw_write: assert property (@(posedge clk) disable iff (!rst_n)
    history_valid && $past(wr_en && addr == 8'h00)
      |-> ctrl_enable == $past(wdata[0]));
  ap_rw_hold: assert property (@(posedge clk) disable iff (!rst_n)
    history_valid && !$past(wr_en && addr == 8'h00)
      |-> ctrl_enable == $past(ctrl_enable));
  ap_hw_set_priority: assert property (@(posedge clk) disable iff (!rst_n)
    history_valid && $past(hw_done) |-> status_done);
  ap_w1c: assert property (@(posedge clk) disable iff (!rst_n)
    history_valid && $past(!hw_done && wr_en && addr == 8'h04 && wdata[0])
      |-> !status_done);
  ap_status_hold: assert property (@(posedge clk) disable iff (!rst_n)
    history_valid && $past(!hw_done && !(wr_en && addr == 8'h04 && wdata[0]))
      |-> status_done == $past(status_done));
```

```
  ap_read_ctrl: assert property (@(posedge clk) disable iff (!rst_n)
    rd_en && addr == 8'h00 |-> rdata == {31'd0, ctrl_enable});
  ap_read_status: assert property (@(posedge clk) disable iff (!rst_n)
    rd_en && addr == 8'h04 |-> rdata == {31'd0, status_done});
  ap_read_default: assert property (@(posedge clk) disable iff (!rst_n)
    (!rd_en || (addr != 8'h00 && addr != 8'h04)) |-> rdata == 32'd0);

  // 不使用 disable iff，单独说明采样到复位时的后继状态。
  ap_reset: assert property (@(posedge clk)
    !rst_n |=> !ctrl_enable && !status_done);
  cp_hw_sw_conflict: cover property (@(posedge clk) disable iff (!rst_n)
    hw_done && wr_en && addr == 8'h04 && wdata[0]);
endmodule

bind reg_block_demo reg_block_demo_props u_reg_props (.*);
```

3.1.3 文件列表

保存为 案例工程/filelist_fpv_register_demo.f：

```
./rtl/reg_block_demo.sv
./formal/reg_block_demo_props.sv
```

3.1.4 手写属性的 FPV 运行脚本

这是 FRV 专章中的配套 FPV 练习，不调用 `frv_load`，不会生成 FRV 自动检查器。

保存为 案例工程/scripts/fpv_register_demo.tcl：

```
# 从案例工程目录执行；结果必须以本机 VC Formal 日志为准。
set_fml_appmode FPV
read_file -top reg_block_demo -sva -format sverilog -vcs {-f
 ↪  filelist_fpv_register_demo.f}
create_clock clk -period 100
create_reset rst_n -sense low
# 先核验实际复位状态，再保存初始状态。
sim_run -stable
sim_save_reset
check_fv_setup
report_fv -list
check_fv -block
report_fv -verbose
```

运行命令（Bash）：

```
# 从包含“案例工程”目录的上一级目录开始。
cd 案例工程
mkdir -p logs
set -o pipefail
vcf -f scripts/fpv_register_demo.tcl -batch 2>&1 | tee logs/fpv_register_demo.log
```

这些命令应在本机具备相应 VC Formal 版本和许可证时执行。日志中的属性状态才是证明结论；命令退出、脚本结束或没有打印错误都不能代替属性结果检查。

检查范围。本节源码足以说明并验证这个简化接口下的寄存器规则，但不能替代 FRV 对 IP-XACT/RALF 的加载、总线适配和自动生成属性。原生 FRV 工程仍须由本节之后的规格输入及接口桥接组成。

4. 一个最小脚本模板

```
set_fml_appmode FRV
# 二选一：实际工程提供寄存器描述文件
frv_load -ral registers.ralf -top block:REG_BLOCK -auto_load
# frv_load -ipxact registers.xml -auto_load
read_file -top reg_block -sva -format sverilog \
  -vcs {-f filelist_reg.f} bind_frv.sv
create_clock clk -period 100
create_reset rst_n -sense low
sim_run -stable
sim_save_reset
check_fv_setup
report_fv -class register -list
check_fv -block
frv_report
report_fv -class register -verbose
```

`bind_frv.sv` 必须实现与实际总线的接口对应关系，不能把此占位文件视作工具自动生成的所有环境组件。在上述 `-top block:REG_BLOCK` 中，`REG_BLOCK` 是 RALF 定义的地址块名称，**不是随意从 RTL 顶层抄来的实例名**；如果使用其他层级，可指定 `comp:`、`mmap:` 或 `block:`，详见 UG 第 626 ~ 630 页。IP-XACT 和 RALF 的可支持字段、命名空间及访问类型以该版本 User Guide 16.4 ~ 16.11 为准。

4.1 原生 FRV 模板完整源码

模板也直接列在文内。它保留防误执行的 `error`；RALF/IP-XACT、生成检查器接口以及真实总线绑定并未伪造成已经存在的文件。补齐输入并审查后才能启用相应命令。

保存为 案例工程/templates/frv_ralf.tcl：

```
# FRV：RALF/IP-XACT 寄存器检查模板。绝不能原样执行。
# 替换 RTL 顶层、RALF、总线桥接与复位时序，并按 User Guide §16 审核。
error " 此文件是 FRV 教学模板：请先替换设计、寄存器定义、接口桥接和复位序列"
set_fml_appmode FRV
# frv_load -ral /project/csr/regs.ralf -top block:REG_BLOCK -auto_load
# read_file -top reg_top -sva -format sverilog -vcs {-f filelist_reg.f} bind_frv.sv
# create_clock clk -period 100
# create_reset rst_n -sense low
# sim_run -stable
# sim_save_reset
```

```
# check_fv -block
# frv_report
```

5. frv_load 与检查器配置

```
frv_load -ipxact chip_regs.xml -auto_load
frv_load -ral chip_regs.ralf -top block:REG_BLOCK -auto_load
frv_load -parameter {VOLATILE=1}
frv_load -parameter {COVER_READ=1}
```

这些示例展示输入和参数能力，并不表示一个工程同时加载两份描述文件后能自动完成去重。UG 第 630 页给出若干用于检查器的参数示例；对易变字段、读写覆盖目标的启用与禁用，应由规格责任人决定。`frv_config` 可调整行为，`frv_report` 用于生成专用报告。使用前通过帮助确认 `frv_load` 所支持的实际格式和字段。

5.1 FRV 检查器和总线桥接的实际组成

UG 第 627 页说明，`frv_load` 可以生成专用的 SVA 检查器文件（默认 `frv.sv`），并可用 `-auto_load` 自动装入生成的检查器、绑定文件及库文件。对于常见的 AMBA 总线，安装目录 `$VC_STATIC_HOME/packages/Frv/` 还包含 APB、AHB、AXI 的预制适配模块，以及用于简单非 AMBA 接口的通用适配模块。

典型路径（以实际安装为准）	用途
`snps_frv_apb_wrapper.sv`	APB 总线寄存器访问抽象
`snps_frv_ahb_wrapper.sv`	AHB 总线寄存器访问抽象
`snps_frv_axi_wrapper.sv`	AXI 总线寄存器访问抽象
`snps_frv_gen_wrapper.sv`	一般寄存器读写接口示例

桥接模块主要负责把总线请求、响应、地址和有效数据准确提供给属性检查器。**提供预制适配模块并不意味着可以免除总线配置检查**：例如 AXI 的 `ready/valid` 接受时刻、超时和响应对应关系仍需核实。若 IP 自定义了特殊握手，需要编写自己的适配器并检查覆盖目标是否生成。

5.2 关于 -parameter / -parameters 的说明

同一版 UG 第 626 页的语法框将选项写成 `-parameters`，随后第 630 页示例写成 `frv_load -parameter {...}`。本手册保留第 630 页的示例，不推断两者均被 Tcl 接受；工程中务必使用 `frv_load -help` 确认版本可用拼写。

6. 结果分析的方法

如果复位值属性失败：检查该寄存器是不是在真实复位域内、复位序列是否正确、字段是否含上电未知值。若读回失败：先确定读请求是否被接受以及返回数据何时有效，再检查寄存器地址、字节

使能与读数据对齐。若 W1C 失败：检查写入掩码、硬件事件与软件写清除的优先级。若很多属性都没被生成：核对寄存器描述路径与映射，不要直接把“没有失败”理解为“寄存器全部正确”。

7. 与 UVM RAL、仿真和 FPGA/SoC 寄存器校验的配合

软件寄存器模型适合生成仿真激励和软件驱动期望值；FRV 可以对寄存器行为进行更系统的逻辑证明。两者应复用同一权威寄存器描述，避免手工维护两份不一致的默认值。对异步硬件更新、跨时钟寄存器和只在某个模式下可写的寄存器，还应有专用接口前提和 SVA 约束。

8. 工程清单

- 确认 IP-XACT/RALF 字段地址、宽度、复位和访问类型均与规格一致。
- 确认总线读写接受点、字节使能、错误响应、超时与复位行为。
- 确认已经报告并审查 FRV 自动生成属性的数量及类别。
- 对 W1C/W1S/volatile/自清除等字段增加必要的手工检查。
- 对无法证明、豁免和未支持的字段单独汇总，而不是将其遗漏。

9. 参考资料

User Guide §16（619 ~ 685 页）及附录中的相关设计查询命令；工具安装目录 `<VC_STATIC_INSTALL>/doc/vcst/examples/FRV` 可寻找官方示例（若随本机安装）。

FTA：形式测试平台分析完整指南

源码内嵌版。本章的完整案例以“保存为”标明文件路径，并在下方直接列出全文。代码框不带显示行号，可直接复制；**案例工程/** 只表示目录结构，不要求另找源码附件。包含占位符或保护性 `error` 的专用流程另行明确标为模板。

第 12 册 · 专用应用　Formal Testbench Analyzer。依据 User Guide 第 21 章（第 914 ~ 960 页），Quick Reference Guide FTA 部分，并参照 User Guide 第 20 章的形式验证验收流程。

1. FTA 解决的不是 RTL 正确性，而是属性环境的质量

形式验证通常给出很多 `Proven`，但这些证明是否足够覆盖真实设计风险？某个功能错误是否会被当前断言检测到？FTA 使用**故障/RTL 修改（mutation）注入**的方法分析形式测试平台的检错能力。如果注入一个功能变化后仍没有任何断言检测到异常，说明当前检查集合可能存在薄弱位置。它是一项验证质量评估方法，而不是再运行一次普通 FPV。

例如，RTL 中 `grant[0]` 的组合条件被修改。如果已有的互斥属性仍全部通过，但缺少“请求必须得到相应授权”的规则，那么 FTA 可能发现这一类变更不易被现有属性检测。



图 12: FTA 故障注入与属性检出

2. 工作过程

1. 在 FPV 模式编译 DUT 和属性，建立正确的时钟、复位与环境前提。
2. 审查现有属性状态，确认值得评估的断言范围，区分断言与覆盖目标。
3. 根据当前软件版本配置可用的故障类型和注入策略。
4. 切换到 FTA，运行激活、传播与检测分析。

5. 汇总可被检出的故障、无法检出的故障以及尚无结论的故障。
6. 对未检测区域补充有意义的设计属性，再运行 FTA 和 FPV 回归。

3. 不能直接从 `-fmode FTA` 启动

User Guide 第 65 页明确提醒：该版本通常从 FPV 模式建立工作区，再使用 `set_fml_appmode FTA` 切换，`vcf -fmode FTA` 不属于支持的直接启动方式。快速指南有独立的 `FTA` 命令行，但并不意味着可以跳过准备好的 FPV 环境。

4. 示范 Tcl：多阶段流程的轮廓

```
# 第一阶段：FPV 环境和属性分析
set_fml_appmode FPV
read_file -top core -sva -format sverilog -vcs {-f filelist.f}
create_clock clk -period 100
create_reset rst_n -sense low
sim_run -stable
sim_save_reset
check_fv_setup
check_fv -block
report_fv -list

# 第二阶段：FTA（故障设置根据具体版本和设计调整）
set_fml_appmode FTA
fta_init
configure_fta_props
compute_fta -par_task FPV
fta_report
```

这个脚本描述学习顺序，**不是一个对任何 FTA 许可证和故障模型都立即可运行的完整正式工程**。尤其 `fta_init`、`configure_fta_props` 和 `compute_fta` 的参数需与当前会话的 FPV task、故障数据库及流程类型对应。阅读 User Guide 第 923 ~ 941 页完整流程再部署生产项目。

4.1 带防误执行保护的完整模板

这里完整展示模板，不要求打开附件。`error` 明确阻止把缺少已验证属性集合或故障配置的脚本当成真实 FTA 分析。

保存为 案例工程/templates/fta_mutation.tcl：

```
# FTA：Formal Testbench Analyzer。须基于真实的 FPV 工程与属性集合。
# 不能仅用 vcf -fmode FTA 直接启动，详见 UG §21 和手册 FTA.md。
error " 此文件是 FTA 教学模板：请先完成真实 FPV 环境、属性集合和故障策略"
# set_fml_appmode FPV
# read_file -top dut -sva -format sverilog -vcs {-f filelist.f}
# create_clock clk -period 100
# create_reset rst_n -sense low
```

```
# sim_run -stable
# sim_save_reset
# check_fv -block
# set_fml_appmode FTA
# fta_init
# configure_fta_props ...      ;# 真实参数必须通过 help 确认
# compute_fta -par_task FPV
# fta_report ...               ;# 依据项目需要设置过滤与实例
```

5. 故障分类与结果含义

场景	工程解释	后续建议
故障被属性检出	当前断言对该类改变敏感	检查检出的属性是否符合预期
故障未检出	可能是死代码、影响不可达、属性缺失或环境屏蔽	先审查可达性和影响路径
注入无法激活	故障点在现有环境下不被触发	检查约束及故障定义
结果未完成	当前证明资源不足	提升策略或缩小任务
被审查豁免	有外部充分证据认定不需要该项	记录依据并保留原结果

检出率与业务规格覆盖完整程度相关，但不能直接等同于“芯片功能正确的百分比”。

6. 关键命令及用途

命令	用途
`fta_init`	初始化 FTA 所需数据结构和分析
`configure_fta_props`	选择待评估的属性集合和条件
`compute_fta -par_task FPV`	基于已准备的 FPV task 开展 FTA 分析
`fta_report -instance ...`	按实例报告 FTA 状态
`get_fta_faults`	查询所考虑的注入故障
`get_activated_props`	对故障查询有激活关系的属性
`cluster_fta_faults`	将故障进行分组以方便分析
`report_fta_fault_clusters`	输出故障分组结果
`export_fault`	导出选中的故障信息
`debug_fta`	启动或准备专用调试
`read_fta_waiver`	导入经过审查的豁免

7. 两个评估示例

例一：FIFO 缺少溢出断言。只写了 FIFO 空闲和复位属性，注入“满时仍接受写入”的 RTL 改变，可能不被检出。补充 `occupancy <= DEPTH`、接受写入时机等断言，重新比较检测结果。**例二：状**

态机错误状态转换。只检查状态编码合法性，状态从 WAIT 错误跳到 IDLE 仍可能保持编码合法，却违反协议。FTA 可以促使增加状态转移规则与完成覆盖。

7.1 用完整源码观察断言是否能发现设计变化

下面采用计数器的正确版与错误版进行手工对照，帮助理解 “改变设计后，原有属性是否敏感”。这一练习使用 FPV；它不是 FTA 自动生成故障或计算检出率的替代品。为便于单独阅读本章，相关文件再次全文列出。

7.1.1 被验证的计数器 RTL

规格：复位后计数为零；使能时从 0 计到 9 后回到 0；没有使能时保持。RTL 文件只包含设计，不把待验证结论写成环境假设。

保存为 案例工程/rtl/counter.sv：

```
module counter (
    input  logic       clk,
    input  logic       rst_n,
    input  logic       enable,
    output logic [3:0] count
);
  always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n)             count <= 4'd0;
    else if (enable) begin
      if (count == 4'd9)    count <= 4'd0;
      else                  count <= count + 4'd1;
    end
  end
endmodule
```

7.1.2 完整断言与绑定文件

history_valid 在复位后建立历史有效标志。ap_range 检查范围，ap_hold 检查保持，ap_wrap 检查回绕；两条 cover 检查最高计数值及回绕场景是否能够出现。末尾 bind 将检查器接入同名设计信号，不需要修改 RTL。

保存为 案例工程/formal/counter_props.sv：

```
module counter_props (
    input logic       clk, rst_n, enable,
    input logic [3:0] count
);
  logic history_valid;
  always_ff @(posedge clk or negedge rst_n)
    if (!rst_n) history_valid <= 1'b0;
    else        history_valid <= 1'b1;

  ap_range: assert property (@(posedge clk) disable iff (!rst_n)
```

```
      count <= 4'd9);

  ap_hold: assert property (@(posedge clk) disable iff (!rst_n)
      history_valid && $past(!enable) |-> count == $past(count));

  ap_wrap: assert property (@(posedge clk) disable iff (!rst_n)
      history_valid && $past(enable && count == 4'd9)
      |-> count == 4'd0);

  cp_reaches_nine: cover property (@(posedge clk) disable iff (!rst_n)
      count == 4'd9);
  cp_wrap: cover property (@(posedge clk) disable iff (!rst_n)
      count == 4'd9 ##1 count == 4'd0);
endmodule

bind counter counter_props u_counter_props (.*);
```

7.1.3 编译文件列表

先列 RTL，再列包含 bind 的属性文件；路径相对于运行 VC Formal 的 **案例工程/** 目录，而不是 Tcl 文件所在目录。

保存为 案例工程/filelist_counter.f：

```
./rtl/counter.sv
./formal/counter_props.sv
```

7.1.4 完整运行脚本

依次选择应用、读入设计与 SVA、定义时钟复位、保存初始状态、检查设置、列出属性并开始证明。初始化步骤需要在实际安装环境核对复位状态，不能把声明复位等同于已经完成复位。

保存为 案例工程/scripts/fpv_counter.tcl：

```
# Run from the example project root with: vcf -f scripts/fpv_counter.tcl -batch
set_fml_appmode FPV
read_file -top counter -sva -format sverilog -vcs {-f filelist_counter.f}
create_clock clk -period 100
create_reset rst_n -sense low
sim_run -stable
sim_save_reset
check_fv_setup
report_fv_setup -list
report_fv -list
set_fml_var fml_max_time 1H
set_fml_var fml_max_memory 8GB
check_fv -block
report_fv -verbose
```

运行命令（Bash）：

```
# 从包含“案例工程”目录的上一级目录开始。
cd 案例工程
mkdir -p logs
set -o pipefail
vcf -f scripts/fpv_counter.tcl -batch 2>&1 | tee logs/fpv_counter.log
```

这些命令应在本机具备相应 VC Formal 版本和许可证时执行。日志中的属性状态才是证明结论；命令退出、脚本结束或没有打印错误都不能代替属性结果检查。

结果阅读。在初始状态正确的前提下，正确版预期满足这些断言，并存在到达 9 和回绕的覆盖轨迹。这不是本次 VC Formal 的实际运行结果。该属性集合也不是全部计数器规格：例如每次普通递增的精确关系仍可另加属性，不能仅凭范围成立就认定全部功能正确。

7.1.5 故意错误的 RTL

错误只在回绕阈值：把 9 改为 10。模块名仍为 `counter`，所以此文件与正确版必须分别编译。属性文件沿用本节已经完整列出的 `counter_props.sv`。

保存为 案例工程/rtl/counter_bad.sv：

```
// Intentionally buggy teaching variant: roll-over condition is 10 instead of 9.
// Do NOT include with rtl/counter.sv in the same compilation.
module counter (
    input  logic       clk,
    input  logic       rst_n,
    input  logic       enable,
    output logic [3:0] count
);
  always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n)             count <= 4'd0;
    else if (enable) begin
      if (count == 4'd10)   count <= 4'd0;
      else                  count <= count + 4'd1;
    end
  end
endmodule
```

7.1.6 错误版编译文件列表

只读入错误设计及同一份检查器，不同时包含两个同名设计。

保存为 案例工程/filelist_counter_bad.f：

```
./rtl/counter_bad.sv
./formal/counter_props.sv
```

7.1.7 错误版完整运行脚本

与正确版采用相同接口和初始化方法，不为了让错误版通过而增加输入限制。

保存为 案例工程/scripts/fpv_counter_bad.tcl：

```
# Intentionally failing counter example. Do not include the good counter at once.
set_fml_appmode FPV
read_file -top counter -sva -format sverilog -vcs {-f filelist_counter_bad.f}
create_clock clk -period 100
create_reset rst_n -sense low
sim_run -stable
sim_save_reset
check_fv_setup
check_fv -block
report_fv -verbose
```

运行命令（Bash）：

```
# 从包含“案例工程”目录的上一级目录开始。
cd 案例工程
mkdir -p logs
set -o pipefail
vcf -f scripts/fpv_counter_bad.tcl -batch 2>&1 | tee logs/fpv_counter_bad.log
```

这些命令应在本机具备相应 VC Formal 版本和许可证时执行。日志中的属性状态才是证明结论；命令退出、脚本结束或没有打印错误都不能代替属性结果检查。

错误版的预期反例是：在允许连续使能的输入下，计数从 9 进入 10，违反范围与回绕规则。查看反例时应区分时钟沿的采样值与非阻塞赋值后的寄存器值；不能把这里描述的预期现象伪装成工具日志。

8. FTA 与故障仿真的区别

FTA 使用形式分析寻找能够触发和传播注入差异的逻辑证据；故障仿真通常依赖已准备的激励序列。两者都可用来研究测试平台质量，但对状态空间、收敛条件、故障模型和资源需求的依赖不同。FTA 的故障并不等同于制造测试的 stuck-at / transition ATPG 故障模型。

9. 参考

User Guide 第 21 章，尤其多阶段运行、故障注入、结果调试与性能部分；User Guide 第 20 章的最终验证质量评估方法。

FuSa：功能安全形式分析完整指南

源码内嵌版。本章的完整案例以“保存为”标明文件路径，并在下方直接列出全文。代码框不带显示行号，可直接复制；**案例工程/** 只表示目录结构，不要求另找源码附件。包含占位符或保护性 `error` 的专用流程另行明确标为模板。

第 13 册 · 专用应用　Functional Safety。依据 User Guide 第 23 章（第 1009 ~ 1039 页），Quick Reference Guide FuSa 部分。与 FTA 的目的不同：FuSa 对故障的可控性、可观察性、检测路径等进行安全分析，输出可以参与功能安全分析报告的数据。

1. 场景与关键概念

面向汽车、工业控制或其他要求功能安全的芯片，故障不仅要判断“是否存在”，还需分析在规定运行状态下可能带来什么影响，是否可以通过硬件检测机制观察到。FuSa 与故障数据库和 Z01X 等工具协同，可按故障可控性、可观察性、检测能力分类，为 FMEDA（失效模式、影响及诊断分析）提供支持。

可控性：是否可能在允许的输入/状态下触发相关故障影响；**可观察性**：故障差异是否可能传播到指定观察点；**可检测性**：预定诊断机制是否能检测到故障。三者相关但不同。不能把“故障可观察”直接写成“已经被安全机制检测”。

2. 前置要求和故障输入

据 UG 第 1015 ~ 1016 页，准备 FuSa 前应先具备无关键设置问题的 FPV 环境；故障定义可以来自 Standard Fault Format（SFF）或统一故障数据库（FDB），还需提供观察点、检测点及有关常量前提。工具版本和故障数据库版本应保持兼容。

典型资料包括：RTL、参数配置、故障清单、正确的时钟复位与初始化、合法运行模式的定义、观察/检测节点列表。若观察点设错，即使故障在真实系统中危险也可能被错误分类。

3. 四类分析

分析类别	目标	可能的误解
结构分析（Structural）	初步查看故障与观察/检测点的结构关系	有结构路径不代表一定可传播

分析类别	目标	可能的误解
可控性分析（Control）	检查是否能激活相应故障效应	不可控可能受环境约束影响
可观察性分析（Observe）	研究故障效应是否到达观察端	到达观察端不代表诊断会报警
可检测性分析（Detection）	判断指定检测机制是否响应	取决于指定检测点和时序规则

UG 23.1.1 给出结构、可控性、可观察性及检测分析。不同模式的输出状态不应混合汇总，尤其不能把“结构排除”当成已经对完整工作状态进行了所有逻辑分析。

4. 标准脚本结构

```
set_fml_appmode FUSA
# 先配置故障清单，后读入设计
fusa_config -sff faults.sff
# 或者结合 FDB 设置，具体数据库名与路径按项目确定
# fusa_config -fdb_campaign my_campaign
read_file -top safety_top -format sverilog -vcs {-f filelist.f}
create_clock clk -period 100
create_reset rst_n -sense low
sim_run -stable
sim_save_reset
fusa_observation -add safety_output
fusa_detection -add alarm
fusa_generate
set_fml_var fusa_run_mode structural
check_fv -block
fusa_report
```

在这个教学骨架中，`faults.sff`、`safety_output`、`alarm` 是必须由实际故障清单和 RTL 替换的对象。正式项目应先在 FPV 环境完成 `check_fv_setup -block` 等准备核查，并了解 UG 关于正式 FuSa 重复执行这些检查的性能提示。

4.1 原生 FuSa 模板完整源码

真实 SFF/FDB 文件和系统安全故障定义属于必要输入，不能用下面的教学异或掩码代替这些输入来声称完成 FuSa。模板默认停止执行。

保存为 案例工程/templates/fusa_safety.tcl：

```
# FuSa：功能安全分析，需要 SFF/FDB、真实的安全检测点和失效定义。
error " 此文件是 FuSa 教学模板：请先提供 SFF/FDB、观察点、检测点、RTL 和安全场景"
# set_fml_appmode FUSA
# fusa_config -sff /project/faults/campaign.sff
# read_file -top safety_top -format sverilog -vcs {-f filelist.f}
# create_clock clk -period 100
# create_reset rst_n -sense low
# sim_run -stable
```

```
# sim_save_reset
# fusa_observation -add ...      ;# 参照 help/UG §23
# fusa_detection -add ...        ;# 参照 help/UG §23
# fusa_generate
# set_fml_var fusa_run_mode structural
# check_fv -block
# fusa_report
```

5. 逐步提高分析精度

```
set_fml_var fusa_run_mode structural
check_fv -block
set_fml_var fusa_run_mode control
check_fv -block
set_fml_var fusa_run_mode observe
check_fv -block
```

执行不同分析阶段时，需要使用每阶段生成的故障类别并保存结果。某些版本或项目中还需专用的检测配置，不能仅凭这三条命令推断完整 FMEDA 已经生成。阅读该安装版本的 `fusa_config -help`、`fusa_generate -help`、`fusa_report -help`。

6. 核心命令说明

命令	功能	重点
`fusa_config -sff ...`	导入标准故障描述	故障实例路径匹配
`fusa_config -fdb ...`	配置统一故障数据库	项目名称与服务器路径
`fusa_observation -add ...`	指定观测节点	与 FMEDA 分析目标对应
`fusa_detection -add ...`	指定诊断检测节点	区分系统输出和安全报警
`fusa_blackbox ...`	针对分析进行适当黑盒设置	应评估故障排除的影响
`fusa_generate`	创建功能安全分析对象	输入的故障清单必须有效
`set_fml_var fusa_run_mode ...`	选择分析阶段	结构、控制、观察等按需运行
`fusa_report_total_faults`	汇总故障数量	注意分母是否包含排除故障
`fusa_report -all_failure_modes`	按模式输出故障结果	区分状态类别
`fusa_save` / `fusa_save -sff ...`	保存分类结果	供后续流程复用

7. 案例：带奇偶校验的安全寄存器

一个重要配置寄存器使用奇偶校验位，写入时更新校验；读取/使用时，如果校验与数据不一致则拉高 `alarm`。可以指定寄存器数据寄存器及校验位中的故障为分析对象，把 `alarm` 作为检测点，关键

输出作为观察点。分析时区分：故障是否真正改变了数据；改变是否到达输出；奇偶校验是否在规定时延内报警。即使一个故障未影响输出，也不能把其他故障的检测结果套用在它身上。

7.1 完整代码：观察值、报警与漏检场景

为把“输出受影响”与“报警检测到”分开，以下只在读取路径增加教学用异或掩码。数据寄存器和校验寄存器正常更新；`fault_mask` 改变读出的数据位，`fault_parity` 改变读出校验位。这个模型既不是晶体管故障模型，也不是故障数据库文件。

7.1.1 完整 RTL

保存为 案例工程/rtl/safety_reg_demo.sv：

```
// 教学故障模型：用异或掩码改变读取路径的数据/校验位。
// 不是 SFF/FDB 注入器，也不代表真实硬件故障的全部类型。
module safety_reg_demo (
  input  logic       clk, rst_n, write_en,
  input  logic [7:0] write_data,
  input  logic [7:0] fault_mask,
  input  logic       fault_parity,
  output logic [7:0] safety_output,
  output logic       alarm
);
  logic [7:0] stored_data;
  logic       stored_parity;

  always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
      stored_data   <= 8'd0;
      stored_parity <= 1'b0;
    end else if (write_en) begin
      stored_data   <= write_data;
      stored_parity <= ^write_data;
    end
  end

  assign safety_output = stored_data ^ fault_mask;
  assign alarm = (^safety_output) ^ stored_parity ^ fault_parity;
endmodule
```

7.1.2 完整 SVA

除了单比特错误报警，还覆盖两位数据同时翻转但不报警的情况。这有助于识别奇偶校验的能力限制，不能把单个场景的结果推广为全部故障检出率。

保存为 案例工程/formal/safety_reg_demo_props.sv：

```
module safety_reg_demo_props (
  input logic clk, rst_n,
```

```
  input logic [7:0] fault_mask,
  input logic fault_parity, alarm
);
  ap_no_fault_no_alarm: assert property (@(posedge clk) disable iff (!rst_n)
    fault_mask == 8'd0 && !fault_parity |-> !alarm);
  ap_single_bit_detected: assert property (@(posedge clk) disable iff (!rst_n)
    $onehot(fault_mask) && !fault_parity |-> alarm);
  ap_parity_bit_detected: assert property (@(posedge clk) disable iff (!rst_n)
    fault_mask == 8'd0 && fault_parity |-> alarm);
  // 两位数据同时翻转不改变奇偶性，用覆盖目标寻找漏检场景。
  cp_two_bit_undetected: cover property (@(posedge clk) disable iff (!rst_n)
    fault_mask == 8'h03 && !fault_parity && !alarm);
endmodule

bind safety_reg_demo safety_reg_demo_props u_safety_props (
  .clk(clk), .rst_n(rst_n), .fault_mask(fault_mask),
  .fault_parity(fault_parity), .alarm(alarm)
);
```

7.1.3 文件列表

保存为 案例工程/filelist_fpv_safety_demo.f：

```
./rtl/safety_reg_demo.sv
./formal/safety_reg_demo_props.sv
```

7.1.4 配套 FPV 脚本

保存为 案例工程/scripts/fpv_safety_demo.tcl：

```
# 从案例工程目录执行；结果必须以本机 VC Formal 日志为准。
set_fml_appmode FPV
read_file -top safety_reg_demo -sva -format sverilog -vcs {-f
 ↪  filelist_fpv_safety_demo.f}
create_clock clk -period 100
create_reset rst_n -sense low
# 先核验实际复位状态，再保存初始状态。
sim_run -stable
sim_save_reset
check_fv_setup
report_fv -list
check_fv -block
report_fv -verbose
```

运行命令（Bash）：

```
# 从包含“案例工程”目录的上一级目录开始。
cd 案例工程
mkdir -p logs
set -o pipefail
vcf -f scripts/fpv_safety_demo.tcl -batch 2>&1 | tee logs/fpv_safety_demo.log
```

这些命令应在本机具备相应 VC Formal 版本和许可证时执行。日志中的属性状态才是证明结论；命令退出、脚本结束或没有打印错误都不能代替属性结果检查。

在该教学模型中，单个数据位或单独校验位改变预期会报警；两位数据同时翻转的覆盖目标预期可达。只有将真实故障清单、观察点、检测点和系统失效定义接入原生 FuSa 流程后，才能解释该流程的实际故障分类。

8. 与 ISO 26262 / FMEDA 的关系

FuSa 分析结果可作为硬件随机故障指标评估的一部分，但不能仅凭工具中的一个百分比就认定达到某 ASIL 等级。功能安全还涉及故障模型、故障率来源、诊断间隔、残余风险、独立性、系统级失效影响和安全机制完整性。所有故障分类以及能够被排除的对象应由功能安全负责人审查。

9. 常见错误

- 故障数据与 RTL 层级不一致，使部分故障没有真正进入分析。
- 未先校验时钟复位及模式常量，造成大量虚假的不可控。
- 观察点或检测点过少，使报告过于乐观或不完整。
- 将可观察与已检测、未完成与不可达混同。
- 把结构分析结果视作所有故障的最终证明。

10. 官方资料

User Guide §23（1009 ~ 1039 页）、§4（初始化）、§6（结果控制）；如涉及数据库交互，应同时参照 Z01X / FDB 的相同软件版本文档。

FSV：形式安全信息流验证完整指南

源码内嵌版。本章的完整案例以“保存为”标明文件路径，并在下方直接列出全文。代码框不带显示行号，可直接复制；**案例工程/** 只表示目录结构，不要求另找源码附件。包含占位符或保护性 `error` 的专用流程另行明确标为模板。

第 14 册 · 专用应用　Formal Security Verification。依据 User Guide 第 22 章（第 961 ~ 1008 页）及 Quick Reference Guide FSV 部分。

1. 目标：检查本不应出现的信息传播

数字系统可能包含安全域与普通域。安全域中的密钥、认证状态、受保护计数器等信息不能在未授权条件下影响普通域输出；普通域输入也不能未经允许改变关键安全状态。FSV 用形式化的来源、目的及传播条件描述这些安全信息流要求。

与一般 CC 不同，FSV 不只是检查“需要的连接是否存在”，还要检查**禁止的信息传播是否可能发生**。示例：安全密钥不允许直接或间接影响公开的调试端口；普通外设不能未经权限检查更新可信启动状态。可见信息流分析并不自动涵盖时间侧信道、功耗侧信道以及模拟泄漏。



图 13: FSV 安全域隔离

2. 什么是信息流与非干扰

假设存在高安全级输入 `secret` 和低安全级输出 `public_out`。一种非干扰（noninterference）性质要求：当普通级输入和合法初态相同时，改变 `secret` 不得改变 `public_out`，除非系统允许某种已定义的解密或公开处理。它是一种关于两种输入情况的关系，而不只是单条波形上某个值等于零。

FSV 通过源和目的定义生成信息流属性，并在形式环境中检查合法传播范围。对于含授权条件、故障检测条件或某些模式允许公开的数据，必须建模对应条件，不能只无条件禁止全部传播。

3. 典型命令结构

```
set_fml_appmode FSV
read_file -top security_top -format sverilog -vcs {-f filelist.f}
create_clock clk -period 100
create_reset rst_n -sense low
sim_run -stable
sim_save_reset
check_fv_setup
fsv_generate -src secret_key -dest debug_out
check_fv -block
fsv_report
report_fv -list
```

脚本中信号名和复位过程是教学占位，真正的形式模型必须包含完整的普通输入、授权模式及初态。`fsv_generate` 的实际语法支持一个或多个源、一个或多个目的，以及 `-src_condition`、`-dest_condition`、`-exclude`、`-name`、`-zone` 等选项（见 User Guide 第 966 页）。

3.1 面向真实系统的完整模板

此模板用于替换成真实安全域层级；第 5 节另有完整的小型设计与运行脚本。这里保留保护性 `error`，不会把尚未审查的系统安全策略自动启用。

保存为 案例工程/templates/fsv_secure_flow.tcl：

```
# FSV：信息流安全。请先完成 source、destination 与授权策略审查。
error " 此文件是 FSV 教学模板：必须指定真正的安全源与公开输出"
# set_fml_appmode FSV
# read_file -top secure_top -format sverilog -vcs {-f filelist.f}
# create_clock clk -period 100
# create_reset rst_n -sense low
# sim_run -stable
# sim_save_reset
# fsv_generate -src u_core.secret -dest u_dbg.out
# check_fv -block
# fsv_report
```

4. 可控的允许传播

有些设计需要在合法授权后输出秘密信息的某种变换结果，例如密钥派生操作的授权输出。此时不能简单禁止所有相关数据依赖，而应制定允许传播的安全策略。FSV 的 `-src_condition`、`-dest_condition` 等选项可帮助表达仅对特定工作条件分析的信息传播。任何白名单范围都需要安全审查，避免授权信号可由不可信输入随意置位。

5. 案例：保护调试输出的完整工程

策略是：无论调试是否开启，公开输出都不得依赖 secret_key。允许调试输出普通状态，但不允许输出秘密数据。本节不用“调试永久关闭”这样的假设掩盖泄漏。

5.1 正确版本 RTL

保存为 案例工程/rtl/debug_guard.sv：

```
// 安全策略：公开调试输出永远不得依赖 secret_key。
module debug_guard (
  input  logic       clk, rst_n, debug_enable,
  input  logic [7:0] secret_key, normal_status,
  output logic [7:0] debug_out
);
  always_comb begin
    if (debug_enable) debug_out = normal_status;
    else              debug_out = 8'h00;
  end
endmodule
```

5.2 故意泄漏的完整 RTL

两份文件模块名相同，必须分别读入。坏版只把普通状态替换为秘密数据，让读者能看清被检查的差异。

保存为 案例工程/rtl/debug_guard_bad.sv：

```
// 故意泄漏版本；模块名与正确版相同，必须分开编译。
module debug_guard (
  input  logic       clk, rst_n, debug_enable,
  input  logic [7:0] secret_key, normal_status,
  output logic [7:0] debug_out
);
  always_comb begin
    if (debug_enable) debug_out = secret_key;
    else              debug_out = 8'h00;
  end
endmodule
```

5.3 正确版文件列表

保存为 案例工程/filelist_fsv_good.f：

```
./rtl/debug_guard.sv
```

5.4 正确版完整 FSV 脚本

本例有明确的源、目的与实际 RTL，不再只使用 `security_top/filelist.f` 占位名称。仍须核对本机 `fsv_generate` 的接口及对象解析结果。

保存为 案例工程/scripts/fsv_debug_good.tcl：

```
# 读取一个版本；先检查源/目的是否成功解析，不能把零属性当作通过。
set_fml_appmode FSV
read_file -top debug_guard -format sverilog -vcs {-f filelist_fsv_good.f}
create_clock clk -period 100
create_reset rst_n -sense low
sim_run -stable
sim_save_reset
check_fv_setup
fsv_generate -src secret_key -dest debug_out
report_fv -list
check_fv -block
fsv_report
report_fv -verbose
```

5.5 错误版文件列表

保存为 案例工程/filelist_fsv_bad.f：

```
./rtl/debug_guard_bad.sv
```

5.6 错误版完整 FSV 脚本

保存为 案例工程/scripts/fsv_debug_bad.tcl：

```
# 读取一个版本；先检查源/目的是否成功解析，不能把零属性当作通过。
set_fml_appmode FSV
read_file -top debug_guard -format sverilog -vcs {-f filelist_fsv_bad.f}
create_clock clk -period 100
create_reset rst_n -sense low
sim_run -stable
sim_save_reset
check_fv_setup
fsv_generate -src secret_key -dest debug_out
report_fv -list
check_fv -block
fsv_report
report_fv -verbose
```

运行命令（Bash）：

```
# 从包含“案例工程”目录的上一级目录开始。
cd 案例工程
mkdir -p logs
set -o pipefail
```

```
vcf -f scripts/fsv_debug_good.tcl -batch 2>&1 | tee logs/fsv_debug_good.log
vcf -f scripts/fsv_debug_bad.tcl -batch 2>&1 | tee logs/fsv_debug_bad.log
```

这些命令应在本机具备相应 VC Formal 版本和许可证时执行。日志中的属性状态才是证明结论；命令退出、脚本结束或没有打印错误都不能代替属性结果检查。

结果核查。正确版源端不参与输出计算，错误版在调试开启时有明确依赖。工具可能对未使用的源做优化，必须先确认属性生成、源目的解析及报告分类；零属性或对象找不到不能当作通过。示例结果仍待本机 VC Formal 执行确认。

6. 信息流安全与普通功能断言相互补充

FPV 可证明 `debug_enable==0` 时 `debug_out==0`，但这不一定足以证明“只要安全源变化，任何普通输出都不发生可识别的变化”。尤其如果信息通过状态机控制、地址、错误响应或其他共享资源间接传播，普通值检查可能漏掉依赖关系。FSV 帮助发现这些关联，但它也需要明确的观察范围和安全模型。

7. 黑盒与抽象的风险

`fsv_blackbox <instance>` 用于安全验证中指定特殊黑盒对象；它的抽象行为会影响信息流路径和可证明性。把加密模块全部黑盒化后，若未正确描述安全输出与普通输出的可见关系，结果可能既不保守也不完整。建议先建立最小原型，比较带/不带黑盒的属性数量与失效情况，再决定大型 SoC 的建模策略。

8. 重要命令

命令	作用
`set_fml_appmode FSV`	选择安全信息流应用
`fsv_generate -src ... -dest ...`	生成保护指定路径的属性
`fsv_generate ... -src_condition ...`	限定源侧条件
`fsv_generate ... -dest_condition ...`	限定目的侧条件
`fsv_blackbox`	为安全分析提供特殊黑盒配置
`fsv_report`	专门输出安全分析结果
`report_fv`	查询底层形式检查状态
`view_trace -property ...`	查看不应出现传播的反例

9. 常见错误及处理

- 忽略普通域的间接数据依赖，只检查显式赋值。
- 未考虑复位或切换安全模式期间的合法输入，导致假阳性。
- 把未知授权条件设置为恒真，从而屏蔽真实泄漏。

- 把“没有找到信息传播反例”当成“已经没有所有侧信道风险”。
- 将安全策略与实现混在同一套强制约束中，无法审查真正的安全责任。

10. 官方参考

User Guide §22 (961~1008 页), 尤其 §22.2 的源目的定义和 `fsv_generate`; Quick Reference Guide FSV 部分。

DPV：数据路径验证与 C/C++—RTL 事务等价完整指南

源码内嵌版。本章的完整案例以“保存为”标明文件路径，并在下方直接列出全文。代码框不带显示行号，可直接复制；**案例工程/** 只表示目录结构，不要求另找源码附件。包含占位符或保护性 `error` 的专用流程另行明确标为模板。

第 15 册 · 专用应用　Data Path Validation。依据 User Guide 第 25 章（第 1066 ~ 1080 页）和 Quick Reference Guide DPV 部分；深度用法需参考单独出版的 VC Formal Data Path Validation User Guide（该文件未包含在此次上传材料中）。

1. 解决的问题与 SEQ 的区别

高性能加法器、乘法器、DSP、编码器和浮点运算单元通常先建立不含硬件周期细节的 C/C++ 算法参考模型，后将其实现为多级流水 RTL。DPV 使用 HECTOR 技术对照高层事务模型和硬件实现的结果，目标是证明相同事务输入产生正确的输出结果，即便两边的执行阶段不同。

SEQ 一般比较两个顺序设计及其相关状态；**DPV** 强调事务级数据变换的对照，例如参考模型在一个函数中完成计算，RTL 则经过 4 个周期才输出。它并不把两个模型天真地逐拍对齐，而是需要声明输入与输出的**事务阶段关系**。

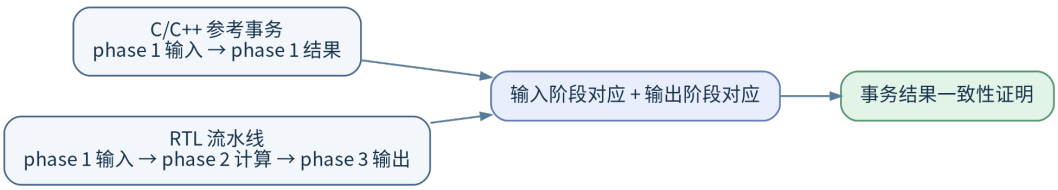


图 14: DPV 的事务阶段对照

2. 典型应用示例：8 位加法器

参考 C++ 函数计算 `uint8_t y = (uint8_t)(a+b)`；RTL 可能有两拍流水线：第 1 拍接收 `a,b`，第 2 拍计算，第 3 拍向输出寄存器提交。若直接比较参考函数返回值与 RTL 每拍 `out`，即使算法正确也可能失败，因为没有处理有效标志和延迟。

需要定义：哪个事件接受一个输入事务；何时读取参考模型参数；RTL 在哪个 phase 输出结果；是

否允许同时在流水线中有多个事务；操作是模 256 加法还是需要保留进位位。输入位宽、C++ 整数提升和有符号规则都必须统一。

3. C++ 事务接口的结构

User Guide 第 1067 页示例提到 `Hector::registerInput`、`Hector::registerOutput` 以及 `Hector::beginCapture`、`Hector::endCapture` 用于定义事务输入输出及捕获范围。实际完整 C++ 包装代码需要适配安装提供的 HECTOR 头文件与编译前端；**本教程不会凭空编写一个未经验证的完整 HECTOR API 程序并声称可执行**。下列普通 C++ 函数表达数学规格，可以作为普通 C++ 代码编译，但它不是 HECTOR wrapper：

```
#include <cstdint>
uint8_t add_mod256(uint8_t a, uint8_t b) {
    return static_cast<uint8_t>(a + b);
}
```

编写正式 DPV C++ spec 时还应处理整型符号、移位、未定义行为、数组访问和宿主编译器的优化规则，否则会出现“高层规范自身就不精确”的问题。

3.1 完整 C++ 源文件与穷举自测

下面两个文件可以用普通 C++ 编译器构建，用于先确认参考函数的模 256 运算。它们不包含、也不伪装成 HECTOR 的接口包装；运行成功不等同于 DPV 已经证明 C++ 与 RTL 等价。

3.1.1 参考函数完整源码

保存为 案例工程/cpp/add_mod256.cc：

```
#include <cstdint>

// 普通 C++ 参考函数；此文件没有调用 HECTOR 接口。
std::uint8_t add_mod256(std::uint8_t a, std::uint8_t b) {
    const unsigned sum = static_cast<unsigned>(a)
                       + static_cast<unsigned>(b);
    return static_cast<std::uint8_t>(sum & 0xffu);
}
```

3.1.2 完整自测程序

枚举 256×256 组输入，遇到不一致返回非零退出状态；检查的是参考函数，不是 RTL。

保存为 案例工程/cpp/add_mod256_selftest.cc：

```
#include <cstdint>
#include <iostream>

std::uint8_t add_mod256(std::uint8_t a, std::uint8_t b);
```

```cpp
int main() {
    unsigned tested = 0;
    for (unsigned a = 0; a < 256; ++a) {
        for (unsigned b = 0; b < 256; ++b) {
            const auto actual = add_mod256(
                static_cast<std::uint8_t>(a),
                static_cast<std::uint8_t>(b));
            const unsigned expected = (a + b) % 256u;
            if (static_cast<unsigned>(actual) != expected) {
                std::cerr << "Mismatch: a=" << a << " b=" << b << '\n';
                return 1;
            }
            ++tested;
        }
    }
    std::cout << "PASS: " << tested << " C++ input pairs\n";
    return 0;
}
```

普通 C++ 自测命令（从 案例工程/ 目录执行）：

```
mkdir -p build
g++ -std=c++11 -Wall -Wextra -Werror \
  cpp/add_mod256.cc cpp/add_mod256_selftest.cc \
  -o build/add_mod256_selftest
./build/add_mod256_selftest
```

4. 创建双设计环境

User Guide §25.3 给出基本过程：创建设计、分析源代码、可选展开、编译数据流图。下面分别展示这两个阶段的命令（具体接口及参数仍需按现有工程修改）：

```
set_fml_appmode DPV

proc compile_spec {} {
  create_design -name spec -top hector_wrapper
  cppan model.cc
  compile_design spec
}

proc compile_impl {} {
  create_design -name impl -top alu_pipe -clock clk -reset rst_n -negReset
  vcs -sverilog alu_pipe.sv
  compile_design impl
}

compile_spec
compile_impl
```

`create_design` 将 C++ 包装函数与 RTL 顶层分别定义成可比较的设计。`cppan` 分析

C++，vcs/vlogan/vhdlan 处理 RTL，compile_design 构造工具所需的内部结构。不能把 compile_design 与普通 FPV 的 read_file 混为一谈。

4.1 软件版本差异：C++11 前端开关

上传的快速参考稿写了 set_hector_comp_use_new_flow true，但 User Guide 第 1069 页给出的 Tcl 示例是：

```
set _hector_comp_use_new_flow true
```

两者不是同一条 Tcl 命令，且 User Guide 指出 C++11 前端可能需要额外许可证。因此本教程以 User Guide 形式提示版本差异，正式脚本应在实际安装环境核验支持的变量及其配置方式，避免照抄未经核实的快速笔记。

4.2 原生 DPV 流程模板完整源码

此模板仍需要正式 HECTOR 包装函数和由实际捕获过程确定的阶段关系。不能把普通 add_mod256 函数直接重命名为 hector_wrapper 后宣称完成对接。

保存为 案例工程/templates/dpv_hector.tcl：

```
# DPV：C/C++ 与 RTL 计算结果对照。需要 HECTOR 事务包装与阶段定义。
error " 此文件是 DPV 教学模板：补齐真实 HECTOR C++ wrapper 和 spec/impl 阶段定义"
# set_fml_appmode DPV
# set _hector_comp_use_new_flow true ;# 此写法见 UG；须复核本地安装帮助
# create_design -name spec -top hector_wrapper
# cppan /project/ref/model.cc
# compile_design spec
# create_design -name impl -top alu_pipe -clock clk -reset rst_n -negReset
# vcs -sverilog -f filelist_impl.f
# compile_design impl
# proc ual {} {
#   map_by_name -inputs -specphase 1 -implphase 1
#   map_by_name -outputs -specphase 1 -implphase 3
# }
# set_user_assumes_lemmas_procedure ual
# report_dpv_mappings
# check_fv
# proofwait
```

5. 通过阶段对应建立事务关系

```
proc ual {} {
  map_by_name -inputs  -specphase 1 -implphase 1
  map_by_name -outputs -specphase 1 -implphase 3
}
set_user_assumes_lemmas_procedure "ual"
```

示例将参考模型输入第 1 阶段与 RTL 第 1 阶段相匹配，同时将参考模型输出第 1 阶段与 RTL 第 3 阶段相匹配。两边所使用的相位编号由实际 C++ wrapper/RTL 捕获和阶段定义决定；“第 3 阶段”只是这个例子的合法约定，不能机械用于所有流水线。

`map_by_name` 根据名称批量形成输入假设及输出对照关系；名称不同的信号需要显式指定。处理加法器输出时还要确保 C++ 与 SystemVerilog 的截断规则、运算宽度及溢出处理相同。

5.1 完整 RTL 与独立的时序检查

以下 RTL 有输入采集、求和、结果提交三个寄存器阶段。由于并发 SVA 在上升沿使用更新前的采样值，本例用接受输入后的第 3 个后继采样沿检查输出。这个 SVA 采样间隔不等于 HECTOR 阶段编号，不能由 `##3` 直接推导 DPV 的 `-implphase`。

5.1.1 完整流水线 RTL

保存为 案例工程/rtl/alu_pipe_demo.sv：

```
// 三个寄存器阶段：输入采集、求和、结果提交。
// 不含 ready/stall；in_valid=1 的每个上升沿都接受一项输入。
module alu_pipe_demo (
  input  logic       clk, rst_n, in_valid,
  input  logic [7:0] a, b,
  output logic       out_valid,
  output logic [7:0] out
);
  logic [7:0] a_q, b_q, sum_q;
  logic       valid_q, sum_valid;

  always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
      a_q       <= 8'd0;
      b_q       <= 8'd0;
      sum_q     <= 8'd0;
      out       <= 8'd0;
      valid_q   <= 1'b0;
      sum_valid <= 1'b0;
      out_valid <= 1'b0;
    end else begin
      valid_q   <= in_valid;
      sum_valid <= valid_q;
      out_valid <= sum_valid;
      if (in_valid) begin
        a_q <= a;
        b_q <= b;
      end
      if (valid_q)   sum_q <= a_q + b_q;
      if (sum_valid) out   <= sum_q;
    end
  end
```

```
endmodule
```

5.1.2 完整 SVA

属性局部变量为每个并发事务保存自己的八位预期值；另一条属性检查有效标志的延迟关系。本例不含停止、反压或冲刷。

保存为 案例工程/formal/alu_pipe_demo_props.sv：

```
module alu_pipe_demo_props (
  input logic clk, rst_n, in_valid, out_valid,
  input logic [7:0] a, b, out
);
  // expected 只属于当前这一次属性匹配，连续事务不会共用一个值。
  property p_add_result;
    logic [7:0] expected;
    @(posedge clk) disable iff (!rst_n)
      (in_valid, expected = a + b)
      |-> ##3 (out_valid && out == expected);
  endproperty
  ap_add_result: assert property (p_add_result);

  logic [2:0] history_valid;
  always_ff @(posedge clk or negedge rst_n)
    if (!rst_n) history_valid <= 3'b000;
    else history_valid <= {history_valid[1:0], 1'b1};

  ap_valid_delay: assert property (@(posedge clk) disable iff (!rst_n)
    history_valid[2] |-> out_valid == $past(in_valid, 3));
  cp_consecutive_results: cover property (@(posedge clk) disable iff (!rst_n)
    out_valid ##1 out_valid ##1 out_valid);
endmodule

bind alu_pipe_demo alu_pipe_demo_props u_alu_props (.*);
```

5.1.3 文件列表

保存为 案例工程/filelist_fpv_alu_demo.f：

```
./rtl/alu_pipe_demo.sv
./formal/alu_pipe_demo_props.sv
```

5.1.4 配套 FPV 脚本

这是检查 RTL 事务行为的独立练习，不会自动构成 C++—RTL 的 DPV 证明。

保存为 案例工程/scripts/fpv_alu_demo.tcl：

```
# 从案例工程目录执行；结果必须以本机 VC Formal 日志为准。
set_fml_appmode FPV
```

```
read_file -top alu_pipe_demo -sva -format sverilog -vcs {-f filelist_fpv_alu_demo.f}
create_clock clk -period 100
create_reset rst_n -sense low
# 先核验实际复位状态，再保存初始状态。
sim_run -stable
sim_save_reset
check_fv_setup
report_fv -list
check_fv -block
report_fv -verbose
```

运行命令（Bash）：

```
# 从包含“案例工程”目录的上一级目录开始。
cd 案例工程
mkdir -p logs
set -o pipefail
vcf -f scripts/fpv_alu_demo.tcl -batch 2>&1 | tee logs/fpv_alu_demo.log
```

这些命令应在本机具备相应 VC Formal 版本和许可证时执行。日志中的属性状态才是证明结论；命令退出、脚本结束或没有打印错误都不能代替属性结果检查。

6. 证明与报告

```
# 在属性已经创建、设计已经编译的前提下运行
check_fv
proofwait
listproofs
listassumes
listlemmas
listcovers
```

对于特定证明对象，DPV 也支持 `solveNB proofName`、`solveNB_init proofName` 等命令；在工程中需要先明确 proof 的名称。调试可以查看对应的反例及 `simcex`，但 C++ 参考模型和 RTL 的相位关系必须正确才能解释差异。

7. 完整可验证事务的六个核对项

1. 输入采集的条件是什么，例如 `in_valid && in_ready`。
2. 每个输入事务是否有唯一的 ID 或输出顺序保证。
3. 参考模型与 RTL 对同一个事务使用完全一致的参数。
4. 输出可比较时的有效标志和阶段编号是否正确。
5. 输出整数截断、饱和、舍入和异常处理是否一致。
6. 复位、气泡、停顿、反压、流水线冲刷等情况是否定义在两边的行为中。

若第 1 阶段输入有效但后续遇到 `stall`，事务完成时间可能不是固定三拍，那么应进一步建模阶段条件或使用其他机制，不能持续使用不成立的固定延迟前提。

8. 常用命令及作用

指令	作用
`create_design`	建立 spec 或 impl 设计以及顶层接口信息
`cppan` / `scdtan`	分析高层语言模型（具体支持参照安装文档）
`vcs` / `vlogan` / `vhdlan`	分析 RTL
`compile_design`	将设计编译为可比较的内部模型
`set_user_assumes_lemmas_procedure`	指定生成输入关系与输出比较规则的 Tcl 过程
`map_by_name -inputs/-outputs`	按命名对应自动创建比较关系
`report_dpv_mappings`	查看输入输出对应关系
`set_hector_case_splitting_procedure`	为困难数据路径选择分情况策略
`solveNB` / `proofwait`	启动与等待证明任务
`listproofs` / `listlemmas`	查询当前证明对象和结论
`simcex`	复现及分析比较失败的事务输入

9. 数据路径证明困难时的策略

分清是**规格定义错误**（C++ 的有符号性/未定义行为）、**阶段设置错误**（错误的输入输出相位）、**RTL 功能错误**，还是**求解规模困难**（乘法/除法/浮点运算）。先用较小位宽和少量操作码验证模型，再推广到完整位宽。必要时按指令类别切分证明目标，或利用 DPV 特有的重写与求解脚本；引擎调优应晚于正确的事务接口设置。

10. 官方参考

User Guide §25（1066 ~ 1080 页）、Quick Reference Guide DPV 部分。更深入的 HECTOR 包装函数、内部 phase、定制求解策略应以单独的 VC Formal Data Path Validation User Guide 为准，不能由通用 UG 的十余页简介替代。

FLP / LPC：低功耗连通性形式验证完整指南

源码内嵌版。本章的完整案例以“保存为”标明文件路径，并在下方直接列出全文。代码框不带显示行号，可直接复制；**案例工程/** 只表示目录结构，不要求另找源码附件。包含占位符或保护性 `error` 的专用流程另行明确标为模板。

第 16 册 · 专用应用　Formal Low Power / Low Power Connectivity Check。依据 User Guide 第 24 章（第 1040 ~ 1065 页）及 Quick Reference Guide 的 Low Power Connectivity Check 部分。

1. 核心问题

低功耗 SoC 通过不同电源域、电源关断、隔离单元、保持寄存器和电源控制器降低功耗。在普通 RTL 连通性检查中，信号可能看起来已经正确相连，但电源域关断后仍可能把未知或无效数据送到始终通电的逻辑；隔离控制接反，可能导致不同电源状态下的功能不符合规格。

FLP CC 将 RTL 与 UPF（Unified Power Format，IEEE 1801）电源意图结合，验证在相应电源状态及使能条件下，指定源与目的的连通性规则是否成立。它不只是检查 UPF 文件能否成功解析，还要分析电源意图对 RTL 的功能影响。



图 15: 低功耗域和隔离输出

2. 重要对象

对象	解释	例子
Power domain	电源控制范围	可关断图像处理域
Supply net / set	电源网络与电压状态	`VDD_CORE`

对象	解释	例子
Isolation	电源关断时对跨域输出进行限制	下游锁定为 0
Retention	保存关断前需要恢复的状态	睡眠前的配置位
Power state table	组合描述电源合法状态	工作、待机、关断
Power controller	控制电源和隔离时序	关断前先使能隔离

要求明确：何时隔离、生效电平是什么、钳位值为何、是否需要恢复保存状态。普通 CC 的组合连接模型不足以单独表达所有电源关闭的效果。

3. 该版本的工作模式

Quick Reference Guide 的低功耗流程不是直接输入 `set_fml_appmode FLP`，而是：

```
set_fml_appmode CC
fvlp_instrument -type "iso pnm"
set_fml_var fvlp_enable_pst_constraint true
read_file -top lp_top -format sverilog -vcs {-f filelist.f}
read_upf power_intent.upf
create_clock clk -period 100
create_reset rst_n -sense low
```

`fvlp_instrument` 启用低功耗语义插桩，`-type` 可根据设计需要选择 `iso`、`pnm`、`ret` 等，具体支持类型参照 UG 1043 页。插入低功耗支持、读入 RTL/UPF 的顺序需要与安装版本示例一致。`read_upf` 中的路径必须指向该工程真实 UPF 文件。

3.1 原生低功耗检查模板完整源码

UPF、电源状态及隔离策略必须来自实际项目。本模板的省略对象属于明确的待配置输入，不是已提供的完整低功耗工程。

保存为 案例工程/templates/flp_low_power.tcl：

```
# 低功耗 CC：必须有 RTL、UPF、电源状态表、真正的电源域信号。
# 不支持普通 CC 的 delayed connectivity check（UG 第 1041 页）。
error " 此文件是 FLP 教学模板：须先提供 UPF、电源控制与隔离策略"
# set_fml_appmode CC
# fvlp_instrument -type "iso pnm"
# set_fml_var fvlp_enable_pst_constraint true
# read_file -top lp_top -format sverilog -vcs {-f filelist.f}
# read_upf /project/upf/power_intent.upf
# create_clock clk -period 100
# create_reset rst_n -sense low
# add_cc -src ... -dest ... -lpa_type ... ;# 填写真实端点与类型
# check_fv -block
# report_fv -verbose
```

4. 最小系统案例

上游处理单元运行于可关断电源域 PD_A，下游控制单元在始终通电的 PD_AON。上游输出 data_o 经隔离单元到下游 data_i。规格：当 PD_A 供电正常且隔离关闭时，data_i 应等于 data_o；当 PD_A 关断时，data_i 应被钳位为零。

检查分两部分：功能连接和隔离钳位。前者可使用带电源条件的 add_cc；后者可使用 flp_check_clamp_value 等专门命令。具体命令需提供符合本版本要求的 -lpa_type、-location、-clamp_value、-enable 和层级对象。

```
# 仅为表达对象的示意；根据 UPF 名称和命令帮助填写合法类型
# add_cc -src u_a.data_o -dest u_aon.data_i -lpa_type <type>
# flp_check_clamp_value -lpa_type <type> -location <self|parent> \
#    -clamp_value 0 -enable iso_en u_aon.data_i
```

以上包含 <...> 的两行是示例占位内容，不得未经替换直接运行。

4.1 完整源码：先理解逻辑隔离关系

为了不把 UPF 环境和普通逻辑混在一起，以下单独给出理想隔离控制的 RTL 与 SVA。这里的 source_power_good 只是布尔输入，不会使仿真器或形式工具真的把某个电源域关断，也没有建立掉电后的未知值模型。

4.1.1 完整逻辑模型

源电源不正常时，理想控制模型立即使能隔离；真实系统的电源控制时序、跨时钟处理和故障行为必须另行验证。

保存为 案例工程/rtl/lp_isolation_demo.sv：

```
// 仅表示理想的逻辑隔离关系，不模拟电源关断产生的未知值。
module lp_isolation_demo (
  input  logic       clk, rst_n,
  input  logic       source_power_good, isolation_request,
  input  logic [7:0] source_data,
  output logic       isolation_enable,
  output logic [7:0] destination_data
);
  // 理想控制模型：源电源不正常时强制隔离；真实 PMC 必须另行验证。
  assign isolation_enable = !source_power_good || isolation_request;
  assign destination_data = isolation_enable ? 8'h00 : source_data;
endmodule
```

4.1.2 完整 SVA

保存为 案例工程/formal/lp_isolation_demo_props.sv：

```
module lp_isolation_demo_props (
  input logic clk, rst_n,
```

```
  input logic source_power_good, isolation_request, isolation_enable,
  input logic [7:0] source_data, destination_data
);
  ap_isolation_clamp: assert property (@(posedge clk) disable iff (!rst_n)
    isolation_enable |-> destination_data == 8'h00);
  ap_power_off_clamp: assert property (@(posedge clk) disable iff (!rst_n)
    !source_power_good |-> destination_data == 8'h00);
  ap_normal_connection: assert property (@(posedge clk) disable iff (!rst_n)
    source_power_good && !isolation_request
      |-> destination_data == source_data);
  cp_off_on_sequence: cover property (@(posedge clk) disable iff (!rst_n)
    !source_power_good ##1 source_power_good && !isolation_request);
endmodule

bind lp_isolation_demo lp_isolation_demo_props u_lp_props (.*);
```

4.1.3 文件列表

保存为 案例工程/filelist_fpv_lp_logic_demo.f：

```
./rtl/lp_isolation_demo.sv
./formal/lp_isolation_demo_props.sv
```

4.1.4 普通 FPV 练习脚本

保存为 案例工程/scripts/fpv_lp_logic_demo.tcl：

```
# 从案例工程目录执行；结果必须以本机 VC Formal 日志为准。
set_fml_appmode FPV
read_file -top lp_isolation_demo -sva -format sverilog -vcs {-f
 ↪  filelist_fpv_lp_logic_demo.f}
create_clock clk -period 100
create_reset rst_n -sense low
# 先核验实际复位状态，再保存初始状态。
sim_run -stable
sim_save_reset
check_fv_setup
report_fv -list
check_fv -block
report_fv -verbose
```

运行命令（Bash）：

```
# 从包含“案例工程”目录的上一级目录开始。
cd 案例工程
mkdir -p logs
set -o pipefail
vcf -f scripts/fpv_lp_logic_demo.tcl -batch 2>&1 | tee logs/fpv_lp_logic_demo.log
```

这些命令应在本机具备相应 VC Formal 版本和许可证时执行。日志中的属性状态才是证明结论；命令退出、脚本结束或没有打印错误都不能代替属性结果检查。

与原生 FLP 的区别。上述练习说明隔离钳位和正常连接的布尔关系，并不验证 UPF 插入位置、电源网络、保持单元或电源关断行为。真正的 FLP 检查还需要第 3 节列出的 UPF 和工具低功耗配置，不能用本节 FPV 结果替代。

5. UPF 可信性与 PMC

UPF 应由电源架构负责人确认电源域划分、电源状态、隔离和保持策略；必要时还需为真实电源管理控制器（PMC）建立环境模型。UG §24.2 提到可为不存在于形式 RTL 的 PMC 建模其外部行为，不能把这种模型误当作 RTL 自身需要被证明的功能。错误地假设所有电源域永久打开，将直接屏蔽很多低功耗缺陷。

6. FLP CC 和普通 CC 的重要区别

User Guide 第 1041 页明确说明：FLP CC 不支持连通性延迟检查。因此不能直接把普通 CC 中的 `path_delay` 用例复制到低功耗检查环境。对于寄存器延迟、时序握手或跨时钟的低功耗交互，需要以另一套适当的功能属性或等价流程验证，而非宣称 FLP CC 已证明它们。

7. 主要命令与用途

命令	用途
`set_fml_appmode CC`	在 CC 模式启动低功耗检查
`fvlp_instrument -type ...`	启用隔离/电源网络/保持等插桩
`read_upf`	读入电源意图并建立相关模型
`get_fvlp_free_supply_nets`	查询没有充分控制的电源网络
`fvlp_set_supply_aon`	将指定电源声明为始终打开
`fvlp_set_supply_onoff_net`	连接电源控制信号与供电网络
`add_cc ... -lpa_type ...`	定义带低功耗类型的连接规则
`flp_check_clamp_value`	检查预期隔离钳位
`flp_cc_vacuity_gen_split`	辅助分析条件不可满足造成的空洞结果
`flp_cc_vacuity_report`	查看有关空洞报告
`fvlp_cc_print_path`	输出低功耗连接的追踪路径

8. 三个需要分别检查的场景

正常通电：隔离关闭时功能数据能够到达下游；**关断阶段：**隔离在失去电源前启动、钳位值正确并持续；**恢复阶段：**电源回到合法状态后，对隔离解除、复位和保持恢复的顺序进行验证。第三阶段常涉及跨周期时序，可能需要 FPV 与电源控制器的状态属性配合。

9. 调试方法

从失败的低功耗连接属性开始，检查 UPF 的源目的供电网络、`iso_en` 及电源状态，再查看波形或 Power Browser。特别注意电源状态表是否包含不实际的组合、隔离使能是否被形式环境错误固定、以及低功耗插桩是否真正参与了编译。不能仅因为正常供电情况下路径连接正确就结束验证。

10. 参考

User Guide §24（1040~1065 页），Quick Reference Guide Low Power Connectivity Checking 部分；涉及 UPF 语法时还应参考项目采用的 IEEE 1801 版本与电源意图开发规范。

反例调试、证明性能与复杂性控制

第 17 册 · 进阶流程　主要依据 User Guide 第 6、7、8、18、19 章（第 183 ~ 344、714 ~ 835 页）以及 Verdi 调试说明（附录 33）。本文将“模型真实缺陷”和“证明尚未结束”严格区分。

1. 为什么形式验证会困难

大设计的原始状态空间往往呈指数增长。真正决定计算规模的不是 RTL 总行数，而是某一条属性需要考虑的状态、时钟、存储结构、算术逻辑、输入自由度和时间深度。极大规模的 SoC 中，一个受局部状态机约束的输出属性可能很容易，而一个 64 位除法运算或需要跨上千周期的罕见序列可能十分困难。

这意味着：**先理解问题与所需观察状态，再投入更多机器资源**。

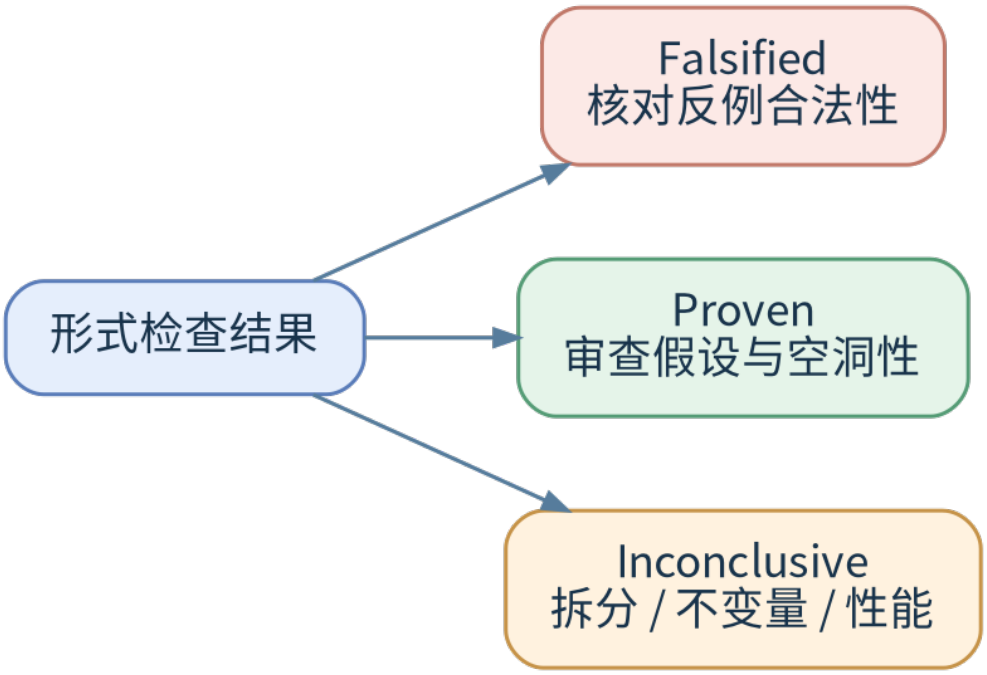


图 16: 失败、空洞与未完成的分流

2. 报告状态决定不同的调试路径

状态	首先需要回答的问题	典型措施
Falsified	是否是真正可实现的反例	查看波形、复位、合法输入和抽象
Proven	规则是否完整且非空洞	`cover` 触发、审查假设及性质范围
Inconclusive	计算难点在哪里	复杂性报告、拆分、抽象、策略调整
Uncoverable	目标为什么无法到达	审查环境约束与初始状态
No properties	属性是否编译和启用	检查 `-sva`、`bind`、模式和任务

3. 失败属性：逐周期回溯

```
report_fv -list
report_fv -verbose
view_trace -property <actual_property_name>
```

波形中应同时查看触发输入、内部状态、相关有效标志、输出及复位；不能只截取失败周期最后一帧。回溯时先从断言后件不满足的位置定位，然后反推输入是否启动了前件检查。许多“错误”属于 SVA 错把 `|=>` 写成 `|->`、忽略非阻塞赋值采样、没有在 `$past` 前验证历史有效等问题。

3.1 错误类型诊断表

典型反例	建议核查
复位刚释放就失败	复位模型、SVA 失效条件、前一拍采样
外部输入给出非法值	协议是否允许；合法约束是否缺失
某内部寄存器突然跳变	是否黑盒、抽象或部分初始状态任意
只有一条复杂断言失败	拆成数条检查以确定错误具体阶段
重试之后“失败消失”	模型或约束是否变化，是否仅换了证明任务

4. 约束检查：最应优先防止的错误

形式环境常见问题是**过度约束**：为了让属性收敛，把本应由 DUT 产生的某个输出/内部状态假设为正确；或者把关键请求置为恒零，使断言从不触发。另一类是**约束不足**：允许非法输入状态，导致反例不符合系统协议。

对每个 assume 保存三个信息：自然语言前提、规格或接口出处、可以用于反驳不合理假设的覆盖目标。跨时钟约束尤其要区别输入保持、边沿采样和真实时钟独立性。若存在多组假设，可分组关闭并比较覆盖目标的可达性。

5. 查找状态空间问题

```
report_fv_complexity
```

```
report_fml_engines
report_fml_jobs
report_fml_hosts
```

复杂性报告帮助识别大影响锥和特殊数据结构。面对大型存储器，可以考虑：将具体数据证明缩减为一个或少量符号地址、抽象未直接参与控制的 RAM 内容、将读写控制与数据正确性分别验证；但必须清楚抽象是否覆盖真实冲突规则和初始化语义。面对大型算术逻辑，可能需要拆分操作码、证明进位或截断的局部不变量。

6. 改善证明性能的合理顺序

1. 首先排除环境错误和空洞的属性。
2. 查找关键的状态和输入，拆分长时序规则。
3. 增加正确的辅助不变量，而不是任意假设结论。
4. 按功能建立独立任务 `fvtask`，先单独证明基础性质。
5. 选择合适的引擎努力程度与运行时间，例如 `set_fml_var fml_effort high`。
6. 对仍存在规模问题的属性评估抽象/分解；出现新反例时回原始设计验证其可行性。
7. 根据主机内存和许可证数配置网格，并对比实际加速效果。

7. `fvtask`：按性质组织工作

```
fvtask -create reset_task
fvtask -create datapath_task
# 选择工作任务
fvtask reset_task
get_fvtask
```

不同应用对复制任务或传递属性集合的选项支持可能不同，实际可使用 `fvtask new_task -copy source_task` 或查看 `help fvtask`。为了便于回归，不应在不记录的情况下频繁切换任务并把不同任务的结果拼成一份“同一配置完成”的报告。

8. 抽象及裁剪驱动器

```
# 进阶操作，仅在确认目标和影响后使用
snip_driver u_dut.internal_net
set_abstractions -construct {mult=16 count=4}
report_abstractions
```

裁剪部分网络的驱动或用抽象代替复杂数据结构后，原设计的行为集合可能改变。这可能加速求解，但也可能出现物理上不可能的反例或把某些条件与设计脱离。任何用于最终证明的抽象都需要解释其正确性及覆盖范围。

9. 困难证明的辅助分析

指令	说明
`compute_reduced_constraints -property ...`	分析对某条证明真正相关的约束子集
`report_reduced_constraints`	输出与证明有关的约束信息
`compute_formal_core -property ...`	分析证明关注的关键逻辑部分
`report_formal_core`	输出核心逻辑关系
`compute_proof_assist -property ...`	获取部分证明辅助信息
`report_proof_assist`	输出辅助结果
`fvlearn_config -local_dir ...`	按版本配置回归加速数据

这些是深入调优工具，而不是新手应默认启用的选项；应先明确每次调优对结果是否有实质影响。

10. Proof Architect：组合式证明思路

当一个大型属性难以一次性证明时，可以选择辅助不变量、局部切分和多阶段验证。组合式证明（Compositional Verification）思路是：将大问题分解成较容易的子目标，在逻辑上确保子目标联合能支持原结论。Proof Architect 是 VC Formal 提供的相关能力之一，详见 User Guide 第 19 章。

正确分解需要保证：局部证明使用的假设可由其他模块实际保证；模块间不会形成未经处理的循环假设；最终合成结论对应原始设计。这不是“把大模块拆成几份后，每份单独通过就自动保证整体”。

11. 有规律地建立证明性能实验

每次实验应只改变一到两个有解释的因素。保存：属性数量、最长/最难属性、证明状态分布、CPU 时间、总运行时间、峰值内存、worker 数、工具版本、输入哈希值。面对证明效率提升，要区分是确实证明了更多属性，还是把一部分属性禁用或过度约束造成的虚假改善。

12. 官方参考

User Guide 第 6 ~ 8、18、19 章，尤其第 714 页起的复杂性分析和第 788 页起的组合式证明；Quick Reference Guide 通用性能部分。

覆盖率评估、验证质量与结果验收

第 18 册 · 工程交付　主要依据 User Guide 第 20 章（第 836 ~ 913 页）、第 11 章（FCA）、第 21 章（FTA）及通用运行/调试章节。

1. 为什么不能只报告“全部属性通过”

形式验证的前提之一是必须有人编写有意义的属性。若一个仲裁器只检查“不存在两路同时授权”，这条性质即使被完整证明，也无法证明“有请求时必须给予授权”。因此验证验收需同时审查三个层面：属性是否正确、属性是否涵盖规格、证明是否完成。

1.1 四种互补的检查手段

方法	回答的问题	局限性
断言证明（FPV）	已编写规则是否永不违反	无法覆盖未编写的规格
触发/行为覆盖（SVA cover）	关键事件是否可达	一条路径可达不代表全部情况正确
代码覆盖与可达性（FCA）	未覆盖代码是否真的不可达	模型与约束可能过度限制
属性敏感性（FTA）	注入错误能否被断言检测	故障集合有限、求解可能未完成

结合这些方法可以发现验证薄弱区域，但不能把它们的百分比相加为“验证完成度”。

2. 建立规格—属性对应表

规格项	类型	属性名称	输入前提	覆盖目标	证明状态	评审状态
FIFO 上溢保护	安全性	`AP_FIFO_DEPTH`	`push` 为请求	`CP_FIFO_FULL`	待运行	待评审
请求应答最多 3 拍	有界响应	`AP_REQ_ACK`	合法请求被受理	`CP_REQ_ACCEPT`	待运行	待评审
复位后中断清零	初始化	`AP_RST_IRQ`	正确复位时序	`CP_RESET_RELEASE`	待运行	待评审

这是交付模板，不是某次工具运行产生的真实状态。真实报告不应把未运行的规则写成 Proven。当规格中某项只能通过仿真、人工审查或系统测试验证时，要明确标注验证方式，而非制造一个看似覆盖它的空洞断言。

3. 验收需要保留哪些文件

```
release/
├── manifest.yaml          # 设计与工具版本、校验和、资源配置
├── requirements.csv       # 规格与属性对应
├── assumptions.md         # 环境约束的来源和证明影响
├── properties.sv          # 提交时实际编译的 SVA
├── run.tcl                # 运行与初始化脚本
├── report_fv.txt          # 原始工具输出
├── summary.csv            # 属性状态汇总
├── waivers.md             # 豁免理由与审批
└── known_limits.md        # 未完成目标与后续验证计划
```

使用工具本身的 save_session 可保存调试数据，但不能替代可读的人类审核记录。运行结果需保留对应的源码版本和相关中间配置，避免以后找不到相同的证明环境。

4. 对“Proven”的独立复核

每条关键属性应检查：前件是否在某条合法路径上发生；disable iff 是否意外关闭主要检查阶段；假设有没有直接限制被验证的 DUT 内部状态；证明对象是否因黑盒化而改变；目标时钟和复位是否正确。建议将首次建立的属性在仿真中加入一两个故意错误版本，确认它确实能够检测到预期缺陷。

5. 不完成或豁免的处理要求

情况	是否允许直接归类“通过”	必须补充的信息
Falsified	否	反例分析、RTL/SVA/约束归因
Inconclusive	否	当前资源、证明难点、替代验证方案
Disabled	否	禁用依据及是否存在未覆盖规格
Waived	不能自动等同于通过	责任人、规格依据、审核和到期条件
Uncoverable	不是功能证明	约束可达性及覆盖目标定义

6. 组合使用 FCA 与 FTA

FCA 可优先帮助识别没有在正常运行配置下被触发的代码或状态；FTA 可进一步研究即便某区域可达，目前的断言是否能够检测改变。比如一个状态机的两个分支都可达，但错误地交换两路输出后当前断言仍不能发现，这显示需要更有针对性的输出行为属性。

7. 质量评审问题

1. 验证的 DUT 顶层和参数是否就是准备交付的版本?
2. 所有必检属性数量与规格表是否一致?
3. 属性是否均具有时钟、复位和输入前提说明?
4. 有无过度约束或从未触发的关键断言?
5. 所有失败与未完成是否已分类处理?
6. 自动生成的属性、覆盖目标及豁免是否经过审核?
7. 更改初始化脚本或工具版本后是否重新运行?
8. 有无保留完整的报告、运行命令及可重现文件?

8. 自动化结果摘要

如果需要脚本自动解析 `report_fv`，应优先采用该工具版本明确支持的机器可读输出，避免仅靠模糊匹配报告文本中的 `pass` 或 `fail`；结果里可能同时包含不同应用下的覆盖和断言状态。示例工程所附 `scripts/validate_sources.py` 只检查文件结构及代码示例，不是 VC Formal 报告解析器。

9. 参考

User Guide §20（形式验收/Signoff），§11（FCA），§21（FTA）。本章阐述通用工程验收方法，不应被解读为 Synopsys 或某行业标准的强制验收条件。

术语表、应用选择与官方资料定位

第 19 册 · 附录索引 源资料：用户提供的 `VC_Formal_UG.pdf` 与 `VC_Formal_Quick_Reference_Guide.pdf`，均为 **X-2025.06-SP1，2025 年 9 月**。UG 页码与 PDF 页码一致。

1. 术语表

英文或命令	中文名称	准确理解
Formal verification	形式验证	对数字设计数学模型进行属性或等价性检查
DUT	待验证设计	被检查的 RTL/网表模块或系统
RTL	寄存器传输级描述	主要使用 Verilog/SystemVerilog/VHDL 编写的数字硬件模型
SVA	SystemVerilog 断言	描述布尔与时序规则的语言结构
Property	属性	可由工具管理的 assert、assume 或 cover 目标
Assertion	断言	DUT 必须满足的规定行为
Assumption	假设	对外部输入与初始条件等的合法性限制
Cover	覆盖目标	搜索某事件/状态是否能够发生
Counterexample	反例	可在当前模型下违反属性的输入/状态路径
Witness	见证	使某目标或性质成立的示范路径
Vacuity	空洞成立	属性成立但关键前提可能从未触发
COI	影响锥	能影响某个目标的逻辑和状态集合
BMC	有界模型检查	展开有限深度搜索违反属性的路径
PDR / IC3	属性引导可达性	用可达集合和归纳不变量等方法完成证明

英文或命令	中文名称	准确理解
Induction	数学归纳证明	基础情形与递推情形共同建立全时域性质
Abstraction	抽象	简化设计模型，可能扩大允许行为集合
Proof	证明	在指定模型和前提下建立属性的正确性
Inconclusive	未完成	当前资源或策略尚无明确结论
Noninterference	非干扰	高安全级输入不应影响未授权低安全级输出
Transactional equivalence	事务等价	按事务输入输出阶段对应比较两个实现
Waiver	豁免	经过审查的结果管理，不改变电路正确性
FMEDA	失效模式、影响及诊断分析	对故障后果、诊断能力等的系统分析
UPF	统一电源格式	描述电源域、隔离、保持与供电状态的文件
IP-XACT	IP 结构和寄存器描述规范	供工具交换接口与寄存器规范的数据格式
RALF	寄存器抽象层格式	寄存器定义及访问属性的数据格式

2. 遇到问题先选哪个应用

你的问题	推荐优先应用	常用补充应用
“这个状态机会不会进入非法状态?”	FPV	AEP
“第三方 RTL 有无明显结构性危险?”	AEP	FPV
“这个中断线是否按配置接到目的端口?”	CC	FPV
“为什么这个信号在启动后可能为 X?”	FXP	AEP / FPV
“重写的 RTL 是否行为一致?”	SEQ	FPV
“寄存器默认值、W1C 行为是否正确?”	FRV	FPV
“未覆盖分支是测试不够还是不可达?”	FCA	FPV
“现有断言能否发现故意注入的 RTL 错误?”	FTA	FPV
“安全故障能否被诊断发现?”	FuSa	故障仿真
“密钥是否可能影响普通调试输出?”	FSV	FPV
“C++ 算法和多拍流水线 RTL 是否一致?”	DPV	SEQ
“电源关断后隔离是否正确?”	FLP / LPC	FPV / CC

3. 官方 UG 章节与页码

UG 章	英文主题	页码	本教程对应文件
1	About VC Formal	31–52	形式验证
2	Setting up VC Formal Design Environment	53–65	平台与环境
3	Managing Constraints and Properties	66–114	SVA
4	Compiling Designs and Verifying Formal Setup	115–170	工程与结果
5	Regression Mode Accelerator	171–182	调试与性能
6	Performing and Configuring Checks	183–246	工程与结果
7	Debugging Results	247–312	调试与性能
8–9	Work Environment; Navigator	313–361	平台与环境
10	Automatically Extracted Properties	362–385	AEP
11	Formal Coverage Analyzer	386–443	FCA
12	Formal X-Propagation	444–457	FXP
13	Connectivity Checking	458–516	CC
14	Sequential Equivalence Checking	517–611	SEQ
15	Supported Assertion IPs	612–618	FPV
16	Formal Register Verification	619–685	FRV
17	Formal Property Verification	686–713	FPV
18–19	Convergence; Proof Architect	714–835	调试与性能
20	Obtaining Formal Signoff	836–913	覆盖与验收
21	Formal Testbench Analyzer	914–960	FTA
22	Security Verification	961–1008	FSV
23	Functional Safety	1009–1039	FuSa

UG 章	英文主题	页码	本教程对应文件
24	Formal Low Power	1040–1065	FLP/LPC
25	Data Path Validation	1066–1080	DPV
26–33	查询命令、SVA、示例、变量与调试附录	1081–1168	速查指南

4. 确认过的重要版本差异

1. FCA 是应用名称；该版本 User Guide 使用 `set_fml_appmode COV`，Quick Reference Guide 写为 `FCA`。
2. FTA 一般先在 FPV 会话准备，再通过 `set_fml_appmode FTA` 切换；User Guide 第 65 页说明不能从 `vcf -fmode FTA` 直接启动。
3. DPV 的新 C++ 前端变量写法在两份参考资料中不同，User Guide 第 1069 页给出 `set_hector_comp_use_new_flow true`，需要核验安装环境。
4. 普通 CC 的连接延迟能力不能照搬到 FLP CC：User Guide 第 1041 页明确指出 FLP CC 不支持延迟连接检查。
5. 用户原始笔记包含部分具备本地部署假设的命令，整理后的正式手册在这些位置增加了“教学模板/版本核验/输入文件必要条件”的说明。

5. 文档范围与版权说明

本文为根据用户提供的资料、公开硬件验证概念和作者独立说明编写的原创中文学习材料。它没有声称完整翻译 Synopsys 的 1,168 页官方文档，也没有代替受许可证保护的命令参考书。两份 PDF 是 Synopsys 所有的受保护材料，请在许可证约定范围内保管与使用；本次交付不重新打包原始 PDF。具体命令以实际安装版本的 `-help`、相关专用参考指南和厂商支持为准。

VC Formal 快速参考指南（中文整理版）

源码内嵌版。本章的完整案例以“保存为”标明文件路径，并在下方直接列出全文。代码框不带显示行号，可直接复制；**案例工程/** 只表示目录结构，不要求另找源码附件。包含占位符或保护性 `error` 的专用流程另行明确标为模板。

版本核对提示（2026-10-17 修订）：本文是便于检索的速查资料；更完整解释见 总目录及各应用专章。上传的 User Guide 与 Quick Reference Guide 在 FCA 应用模式（`COV/FCA`）及 DPV 新流程参数上有不一致之处，本版优先参考 User Guide 的完整流程，并已在相应位置标注。凡是以尖括号、`[...]`、`spec|impl` 表示的占位写法，均不能直接当作 Tcl 运行。本文命令未经授权环境真实运行验证。

- 来源：VC_Formal_Quick_Reference_Guide.pdf
- 版本：X-2025.06-SP1，2025-09
- 说明：本文保留命令名与参数格式，中文部分为整理和翻译，方便直接对照 Tcl 使用。

1. VC Formal 简介

VC Formal 是面向 RTL 和网表的形式验证平台。它不依赖随机仿真，而是把设计、约束和属性转成可求解的逻辑问题，在更大的状态空间里发现潜在缺陷。

它特别适合发现以下问题：- 边界条件错误 - 协议违规 - 死锁和活锁 - 漏连、错连、位宽不匹配 - 时钟/复位假设错误 - 深层角落场景

1.1 常见应用

- FPV：功能属性验证，验证用户定义属性和 Assertion IP 属性。
- FRV：寄存器验证，根据 IP-XACT 或 RALF 自动生成并验证检查。
- FCA：形式覆盖率分析，辅助仿真覆盖收敛与形式证明收敛。
- FSV：形式安全验证，确保安全域与非安全域之间不会发生意外数据传播。
- CC：连通性检查，检查源和目的之间是否存在结构和功能上的连接。
- FXP：X 传播验证，检查未知值 X 是否会传播到危险位置。
- AEP：自动提取属性，自动从设计中抽取属性并证明。
- DPV：数据路径验证，使用 HECTOR 技术验证 C/C++ 与 RTL 之间的数据变换块。
- SEQ：顺序等价检查，比较两个 RTL 设计并验证功能等价性。
- FuSa：功能安全分析，分析 Z01X fault 的可控性和可观测性并计算 FMEDA 指标。
- FTA：形式测试平台分析，检查注入故障是否能被形式测试平台检测到。

- FLP：低功耗验证，检查带电源意图的设计是否符合功能规格，可与 CC 和 SEQ 配合使用。

1.2 本手册的使用方式

- 这是一份快速参考，展示常见流程、命令和变量。
- 命令后面的选项通常只是示例，不是唯一写法。
- 更详细的说明可查 Synopsys 文档中心的 SolvNet 页面。

1.3 通用运行方式

```
# 带 Verdi 交互式运行
vcf -f run.tcl -verdi

# 不带 Verdi 的交互式运行
vcf -f run.tcl

# 批处理 / 回归模式
vcf -f run.tcl -batch

# Elite 授权模式
vcf -f run.tcl -verdi -fml_elite

# Token 授权模式
vcf -f run.tcl -verdi -fml_token
```

1.4 通用帮助命令

```
vcf -help
vcf> help
vcf> help report*
vcf> report_fv -help
vcf> man report_fv
vcf> view_help report_fv
vcf> report_fml_var fml_max_*
vcf> report_app_var *fml*
```

2. FPV：功能属性验证应用

2.1 流程

- Set Up App
- Compile Design
- Add Clock/Reset Info
- Finish
- Generate Reset State
- Debug & Fix

- Run Verification

2.2 设计搭建与初始化

设置应用模式

```
set_fml_appmode FPV
```

设定黑盒

```
set_blackbox -designs module
set_blackbox -cells hier_instance
report_blackbox
```

加载 Assertion IP

```
aip_load -protocol "AHB AXI5"
```

读取并展开设计

```
read_file -top top -sva -format verilog -vcs {-f filelist}
analyze -format verilog -vcs {-f filelist}
elaborate top -sva
```

时钟、复位与常量

```
create_clock clk -period 100
create_reset rst -sense high
set_constant testmode -value 1'b0
set_change_at -clock clk -default
```

推断 formal setup

```
infer_formal_setup -type clock
formal_setup_config
report_formal_setup
```

通过仿真初始化设计

```
sim_run -stable
sim_force signal -apply 3'b000
sim_run 3 -clk clk
sim_set_state -uninitialized -user_only -apply 0
sim_save_reset
view_trace -reset
sim_get -signals {control_reg*}
```

设计复杂度与 setup 检查

```
report_fv_complexity
check_fv_setup
fv_setup_config
report_fv_setup -list
```

2.3 输入格式

- RTL 文件：Verilog、SystemVerilog、VHDL
- SVA 文件
- Tcl 文件

2.4 验证与调试

属性管理

```
fvassume <name> -expr {<expression>}
fvassert <name> -expr {<expression>}
fvcover <name> -expr {<expression>}
fvdisable property
fvenable property
fvassert property
fvassume property
fvcover property
```

计算资源

```
set_fml_var fml_max_time 24H
set_fml_var fml_max_memory 32GB
set_fml_var fml_progress_time_limit 2H
set_grid_usage -type RSH=12
set_grid_usage -file hostfile
```

空洞性与 witness

```
set_fml_var fml_vacuity_on true
set_fml_var fml_witness_on false
```

结果加速

```
fvlearn_config -local_dir rma_db
```

运行验证

```
check_fv
check_fv -stop
```

```
check_fv -block
check_fv -run_finish {report_fv -list}
```

报告结果

```
report_fv
report_fv -list
report_fv -verbose
```

保存和恢复会话

```
save_session -session session
restore_session -session session
vcf -restore -session session
```

查看 trace

```
view_trace -property property
fvtrace -property property
```

验证摘要

```
compute_verification_summary
set_verification_summary
waive_verification_summary
report_verification_summary
```

2.5 性能与收敛

proof 资源倾向

```
set_fml_var fml_effort default
set_fml_var fml_effort high
set_fml_var fml_effort bug_hunting
set_fml_var fml_effort discovery
```

进度查询

```
report_fml_engines
report_fml_jobs
report_fml_hosts
```

抽象与约束优化

```
snip_driver net
set_abstractions -construct {mult=16 count=4}
report_abstractions
```

```
compute_reduced_constraints -property property
report_reduced_constraints
get_reduced_constraints
compute_formal_core -property property
report_formal_core
get_formal_core
compute_proof_assist -property property
report_proof_assist
```

任务管理

```
fvtask -create task
fvtask new_task -copy task
fvtask new_task -copy task [-assumes <list>] [-asserts <list>] [-covers <list>]
 ↪  [-constants <list>] [-snips <list>] [-changeats<list>] [-keep_status]
 ↪  [-keep_task]
fvtask task
get_fvtask
```

退出

```
quit
```

2.6 第一个工程的全部源码

上面的命令列表用于按用途检索，其中 `top/filelist.f` 等名称属于参数示意。下面则列出全部实际文件，帮助读者从命令清单进入可保存的教学工程。

2.6.1 被验证的计数器 RTL

规格：复位后计数为零；使能时从 0 计到 9 后回到 0；没有使能时保持。RTL 文件只包含设计，不把待验证结论写成环境假设。

保存为 案例工程/rtl/counter.sv：

```
module counter (
    input  logic       clk,
    input  logic       rst_n,
    input  logic       enable,
    output logic [3:0] count
);
  always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n)             count <= 4'd0;
    else if (enable) begin
      if (count == 4'd9)    count <= 4'd0;
      else                  count <= count + 4'd1;
    end
  end
endmodule
```

2.6.2 完整断言与绑定文件

history_valid 在复位后建立历史有效标志。ap_range 检查范围，ap_hold 检查保持，ap_wrap 检查回绕；两条 cover 检查最高计数值及回绕场景是否能够出现。末尾 bind 将检查器接入同名设计信号，不需要修改 RTL。

保存为 案例工程/formal/counter_props.sv：

```
module counter_props (
    input logic       clk, rst_n, enable,
    input logic [3:0] count
);
  logic history_valid;
  always_ff @(posedge clk or negedge rst_n)
    if (!rst_n) history_valid <= 1'b0;
    else        history_valid <= 1'b1;

  ap_range: assert property (@(posedge clk) disable iff (!rst_n)
      count <= 4'd9);

  ap_hold: assert property (@(posedge clk) disable iff (!rst_n)
      history_valid && $past(!enable) |-> count == $past(count));

  ap_wrap: assert property (@(posedge clk) disable iff (!rst_n)
      history_valid && $past(enable && count == 4'd9)
      |-> count == 4'd0);

  cp_reaches_nine: cover property (@(posedge clk) disable iff (!rst_n)
      count == 4'd9);
  cp_wrap: cover property (@(posedge clk) disable iff (!rst_n)
      count == 4'd9 ##1 count == 4'd0);
endmodule

bind counter counter_props u_counter_props (.*);
```

2.6.3 编译文件列表

先列 RTL，再列包含 bind 的属性文件；路径相对于运行 VC Formal 的 案例工程/ 目录，而不是 Tcl 文件所在目录。

保存为 案例工程/filelist_counter.f：

```
./rtl/counter.sv
./formal/counter_props.sv
```

2.6.4 完整运行脚本

依次选择应用、读入设计与 SVA、定义时钟复位、保存初始状态、检查设置、列出属性并开始证明。初始化步骤需要在实际安装环境核对复位状态，不能把声明复位等同于已经完成复位。

保存为 案例工程/scripts/fpv_counter.tcl：

```
# Run from the example project root with: vcf -f scripts/fpv_counter.tcl -batch
set_fml_appmode FPV
read_file -top counter -sva -format sverilog -vcs {-f filelist_counter.f}
create_clock clk -period 100
create_reset rst_n -sense low
sim_run -stable
sim_save_reset
check_fv_setup
report_fv_setup -list
report_fv -list
set_fml_var fml_max_time 1H
set_fml_var fml_max_memory 8GB
check_fv -block
report_fv -verbose
```

运行命令（Bash）：

```
# 从包含“案例工程”目录的上一级目录开始。
cd 案例工程
mkdir -p logs
set -o pipefail
vcf -f scripts/fpv_counter.tcl -batch 2>&1 | tee logs/fpv_counter.log
```

这些命令应在本机具备相应 VC Formal 版本和许可证时执行。日志中的属性状态才是证明结论；命令退出、脚本结束或没有打印错误都不能代替属性结果检查。

结果阅读。在初始状态正确的前提下，正确版预期满足这些断言，并存在到达 9 和回绕的覆盖轨迹。这不是本次 VC Formal 的实际运行结果。该属性集合也不是全部计数器规格：例如每次普通递增的精确关系仍可另加属性，不能仅凭范围成立就认定全部功能正确。

3. AEP：自动提取属性应用

3.1 作用

AEP 会自动从设计里提取属性，并进行验证，适合用来快速覆盖常见状态机、死锁和约束类检查。

3.2 设计搭建与初始化

设置应用模式

```
set_fml_appmode AEP
```

打开 VHDL 和特定检查

```
set_app_var fml_enable_vhdl_aep true
set_app_var fml_enable_vhdl_bus_check true
set_app_var fml_enable_vhdl_cov true
```

```
set_fml_var fml_aep_unique_name true
```

读取和展开设计

```
read_file -top top -aep AEP_type -format RTL_format -vcs {-f filelist}
read_file -top top -aep fsm_deadlock+fsm_livelock -format RTL_format -vcs {-f
↪  filelist}
analyze -format RTL_format -vcs {-f filelist}
elaborate top -aep AEP_type
```

时钟、复位与初始化

```
create_clock clk -period 100
create_reset rst -sense high
set_constant testmode -value 1'b0
sim_run -stable
sim_force signal -apply 3'b000
sim_run 3 -clk clk
sim_set_state -uninitialized -user_only -apply 0
sim_save_reset
view_trace -reset
sim_get -signals {control_reg*}
```

Setup 检查

```
check_fv_setup
fv_check_config
report_fv_setup -list
```

3.3 AEP 专属配置

FSM fairness

```
aep_generate -type fsm_fairness
aep_generate -type fsm_fairness -tcl fairness.tcl
```

控制某些状态不生成属性

```
aep_config -dont_generate -type fsm_livelock {state_reg::state_name}
```

3.4 验证与调试

属性管理

```
fvdisable property
fvenable property
fvwaive property
fvunwaive property
```

资源配置与运行

```
set_fml_var fml_max_time 24H
set_fml_var fml_max_memory 32GB
set_fml_var fml_progress_time_limit 2H
set_grid_usage -type RSH=12
set_grid_usage -file hostfile
check_fv
check_fv -stop
check_fv -block
check_fv -run_finish {report_fv -list}
```

报告与 waiver

```
report_fv
report_fv -list
report_fv -verbose
save_waiver_file -file aep.el
read_waiver_file -elfiles aep.el
```

会话与 trace

```
save_session -session session
restore_session -session session
vcf -restore -session session
view_trace -property property
fvtrace -property property
```

3.5 性能与任务

- 进度查询同 FPV。
- 任务管理同 FPV。
- 退出命令：`quit`

4. CC：连通性检查应用

4.1 作用

CC 用来检查源和目的之间是否存在结构和功能上的连接，特别适合总线、端口和路径连通性确认。

4.2 设计搭建与初始化

设置应用模式

```
set_fml_appmode CC
```

黑盒设置

```
set_blackbox -designs module
set_blackbox -cells hier_instance
report_blackbox
set_fml_var fml_cc_autobbox false
```

读取设计

```
read_file -top top -format verilog -vcs {-f filelist}
analyze -format verilog -vcs {-f filelist}
elaborate top
```

时钟、复位和常量

```
create_clock clk -period 100
create_reset rst -sense high
set_constant testmode -value 1'b0
```

信号别名

```
set_cc_alias <alias name> <RTL signal name>
report_cc_alias
delete_cc_alias <alias name>
```

指定连接

```
add_cc -src signal -dest signal
load_cc_set_param param_name "%<value>%"
load_cc -format csv csv file
```

保存初始状态

```
sim_save_reset
```

Setup 与复杂度查询

```
check_fv_setup
fv_setup_config
report_fv_setup -list
report_fv_complexity
```

4.3 结果与调试

```
fvdisable prop_name
fvenable prop_name
set_fml_var fml_max_time 24H
set_fml_var fml_max_memory 32GB
```

```
set_fml_var fml_progress_time_limit 2H
set_grid_usage -type RSH=10
set_grid_usage -file hostfile
set_fml_var fml_vacuity_on true
fvlearn_config -local_dir rma_db
check_fv
check_fv -stop
check_fv -block
check_fv -run_finish {report_fv -list}
report_load_cc
report_fv
report_fv -list
report_fv -verbose
report_fv -formatCC csv|tcl|path
save_session -session session
restore_session -session session
vcf -restore -session session
view_trace -property prop_name
view_schematic -prop prop_name
list_path -src signal -dest signal
list_path -cc_prop prop_name
```

4.4 连通性提取与覆盖

```
generate_cc -src signal -dest signal -run
set_fml_var fml_enable_cc_extraction_latency true
generate_cc -src signal -dest signal
generate_cc -run -seq_depth <depth>
set_app_var fml_cc_coverage_analyzer true
set_app_var fml_cov_tgl_input_port true
read_file -top top -cov tgl
compute_cc_cov
save_cc_cov_results
```

4.5 其它

- 进度查询同 FPV。
- 任务管理同 FPV。
- 退出命令：`quit`

5. FCA：形式覆盖率分析应用

5.1 作用

FCA 主要用于把仿真覆盖与形式证明结合起来，帮助判断哪些地方已经覆盖到、哪些地方仍然存在验证盲区。

5.2 设计搭建与初始化

设置应用模式

```
set_fml_appmode COV   ;# UG 此版本的 FCA/COV 示例；如工具提示不支持，检查本地帮助
```

覆盖相关配置

```
set_app_var fml_enable_vhdl_cov true
set_fml_var fml_reset_property_check true
set_fml_var fml_cov_enable_branch_cov true
set_app_var fml_cov_tgl_input_port true
```

覆盖数据库

```
read_covdb -cov_input simv.vdb -cov_dut Testbench.Dut_instance [-elfile elfile]
 ↪  [-check_el]
save_covdb -status covered -name cov.db
view_coverage -cov_input cov.db -elfiles elfiles
```

黑盒与编译

```
set_blackbox -designs module
set_blackbox -cells hier_instance
report_blackbox
read_file -top my_top [-sva] -cov Coverage_Metric -format RTL_format -vcs {-f
 ↪  my_filelist}
analyze -format RTL_format -vcs {-f my_filelist}
elaborate my_top [-sva] -cov Coverage_Metric
```

时钟、复位、常量

```
create_clock clk -period 100
create_reset rst -sense high
set_constant testmode -value 1'b0
```

5.3 结果与调试

```
check_fv_setup
fv_setup_config
report_fv_setup -list
sim_run -stable
sim_save_reset
fvdisable prop_name
fvdisable -type toggle
fvenable prop_name
set_fml_var fml_max_time 24H
set_fml_var fml_max_memory 16GB
```

```
set_fml_var fml_progress_time_limit 2H
set_grid_usage -type LSF=100 -control {bsub -q ...}
report_fv
report_fv -list > list_report.txt
report_fv -verbose > verbose_report.txt
save_cov_exclusion -file exclusion.el
save_session -session session
restore_session -session session
vcf -restore -session session
```

5.4 Auto Scale

```
cov_config -auto_scale
compute_cov
cov_config -line_task <number>
cov_config -branch_task <number>
cov_config -cond_task <number>
cov_config -fsm_task <number>
cov_config -tgl_task <number>
cov_config -line_limit <number>
cov_config -branch_limit <number>
cov_config -cond_limit <number>
cov_config -fsm_limit <number>
cov_config -tgl_limit <number>
cov_config -auto_scale -end_task_cmd "save_covdb -name cc_tgl.vdb -update"
```

5.5 其它

- 进度查询同 FPV。
- 任务管理同 FPV。
- 退出命令：`quit`

6. FXP：形式 X 传播验证应用

6.1 作用

FXP 用于检查未知值 X 是否会传播到危险位置，帮助发现 X 传播带来的潜在功能风险。

6.2 设计搭建与初始化

```
set_fml_appmode FXP
set_blackbox -design module
set_blackbox -cells hier_instance
report_blackbox
read_file -top top -sva -format sverilog -vcs {-f filelist}
analyze -format sverilog -vcs {-f filelist}
elaborate top -sva
```

```
create_clock clk -period 100
create_reset rst -sense high
set_constant testmode -value 1'b0
sim_run -stable
sim_force signal -apply 3'b000
sim_run 3 -clk clk
sim_set_state -uninitialized -user_only -apply 0
sim_save_reset
view_trace -reset
sim_get -signals {control_reg*}
check_fv_setup
fv_check_config
report_fv_setup -list
```

6.3 FXP 专属配置

```
fxp_generate [<list-of-scopes>] -name <my_name> -type {<list-of-types>}
fxp_assume -injectx [-name <name>] [-reset] [-oba] [-xassign] [-undef] [-condition
↪  <expr>] [-scope <scope_name>] <signal_name>
fxp_assume -nox [-name <name>] [-reset] [condition <condition> [<signal_name>]]
fxp_assert [-name <name>] [-condition <expr>] <signal_name>
```

6.4 验证与调试

```
fvdisable property
fvenable property
set_fml_var fml_max_time 24H
set_fml_var fml_max_memory 32GB
set_fml_var fml_progress_time_limit 2H
set_grid_usage -type RSH=12
set_grid_usage -file hostfile
check_fv
check_fv -stop
check_fv -block
check_fv -run_finish {report_fv -list}
report_fv
report_fv -list
report_fv -verbose
fxp_compute_rootcause property
fxp_report_rootcause
save_session -session session
restore_session -session session
vcf -restore -session session
view_trace -property property
fvtrace -property property
```

6.5 性能与任务

- 进度查询同 FPV。

- 任务管理同 FPV。
- 退出命令：`quit`

7. SEQ：顺序等价检查应用

7.1 作用

SEQ 用于比较两个 RTL 设计，确认在映射关系已知的前提下它们在功能上等价。

7.2 设计搭建与初始化

设置应用模式

```
set_fml_appmode SEQ
seq_config -recipe orch_generic_medium
seq_config -map_uninit -map_x zero
```

黑盒与编译

```
set_blackbox -designs module
set_blackbox -cells hier_instance
report_blackbox
analyze -format verilog -library spec -vcs {-f sfilelist}
analyze -format verilog -library impl -vcs {-f ifilelist}
elaborate_seq -spectop <DUT_top> -impltop impl
analyze -format verilog -vcs {-f filelist}
elaborate_seq -spectop top -same_design
```

时钟、复位、常量与初始化

```
create_clock clk -period 100
create_reset rst -sense high
set_constant testmode -value 1'b0
sim_run -stable
sim_force signal -apply 3'b000
sim_run 3 -clk clk
sim_set_state -uninitialized -user_only -apply 0
sim_save_reset
view_trace -reset
sim_get -signals {control_reg*}
```

映射设计

```
map_by_name
seqmap
sequnmap
seq_read_mapping_file mappings.tcl
```

```
report_seq_mappings
```

7.3 验证与调试

```
check_fv_setup
fv_setup_config
report_fv_setup -list
report_fv_complexity
fvassume <name> -expr {<expression>}
fvassert <name> -expr {<expression>}
fvcover <name> -expr {<expression>}
fvdisable property
fvenable property
fvassert property
fvassume property
fvcover property
set_fml_var fml_max_time 24H
set_fml_var fml_max_memory 32GB
set_fml_var fml_progress_time_limit 2H
set_grid_usage -type RSH=10
set_grid_usage -file hostfile
set_backup_grid_usage -file hostfile
set_fml_var fml_vacuity_on true
set_fml_var fml_witness_on true
check_fv
check_fv -stop
check_fv -block
check_fv -run_finish {report_fv -list}
resume_seq
report_fv
report_fv -list
report_fv -verbose
report_proofs
save_session -session session
restore_session -session session
vcf -restore -session session
view_trace -property property
fvtrace -property property
set_debug property
seq_debug_report property
```

7.4 性能与收敛

```
set_fml_var fml_effort default
set_fml_var fml_effort high
set_fml_var fml_effort bug_hunting
set_fml_var fml_effort bitlevel_hard
set_fml_var fml_effort no_decompose_expensive
fvlearn_config -local_dir rma_db
```

```
report_fml_engines
report_fml_jobs
report_fml_hosts
snip_driver net
set_seq_abstractions -construct {mult=16 add=4}
report_abstractions
compute_reduced_constraints -property property
report_reduced_constraints
get_reduced_constraints
compute_formal_core -property property
report_formal_core
get_formal_core
```

7.5 任务与退出

```
fvtask -create task
fvtask new_task -copy task
fvtask new_task -copy task [-assumes <list>] [-asserts <list>] [-covers <list>]
↪  [-constants <list>] [-snips <list>] [-changeats<list>]
fvtask task
get_fvtask
quit
```

8. FRV：形式寄存器验证应用

8.1 作用

FRV 会基于 IP-XACT 或 RALF 中的寄存器定义自动创建检查，并对寄存器行为进行形式验证。

8.2 设计搭建与初始化

```
set_fml_appmode FRV
set_blackbox -designs module
set_blackbox -cells hier_instance
report_blackbox
frv_load -ipxact axi4lite_dmac.xml -auto_load
frv_load -ral axi4lite_dmac.ralf -top block:REG_BLOCK -auto_load
# -top 名称来自 RALF 规格；Quick Reference Guide 省略此推荐参数
read_file -top top -sva -format verilog -vcs {-f filelist} bind_frv.sv
analyze -format verilog -vcs {-f filelist} bind_frv.sv
elaborate top -sva
create_clock clk -period 100
create_reset rst -sense high
set_constant testmode -value 1'b0
set_change_at -clock clk -default
sim_run -stable
sim_force signal -apply 3'b000
sim_run 3 -clk clk
```

```
sim_set_state -uninitialized -user_only -apply 0
sim_save_reset
view_trace -reset
sim_get -signals {control_reg*}
check_fv_setup
fv_setup_config
report_fv_setup -list
report_fv_complexity
```

8.3 输入格式

- RTL 文件：Verilog、SystemVerilog、VHDL
- SVA 文件
- Tcl 文件
- IPXACT 文件：寄存器定义 XML
- RALF 文件：寄存器定义文件

8.4 验证与调试

```
fvassume <name> -expr {<expression>}
fvassert <name> -expr {<expression>}
fvcover <name> -expr {<expression>}
fvdisable property
fvenable property
fvassert property
fvassume property
fvcover property
set_fml_var fml_max_time 24H
set_fml_var fml_max_memory 32GB
set_fml_var fml_progress_time_limit 2H
set_grid_usage -type RSH=10
set_grid_usage -file hostfile
set_fml_var fml_vacuity_on true
set_fml_var fml_witness_on false
check_fv
check_fv -stop
check_fv -block
check_fv -run_finish {report_fv -list}
report_fv -class register
report_fv -class register -list
report_fv -class register -verbose
frv_report
save_session -session session
restore_session -session session
vcf -restore -session session
view_trace -property property
view_trace -vacuity property
view_trace -witness property
fvtrace -property property
```

```
set_fml_var fml_effort default
set_fml_var fml_effort high
set_fml_var fml_effort bug_hunting
set_fml_var fml_effort discovery
fvlearn_config -local_dir rma_db
report_fml_engines
report_fml_jobs
report_fml_hosts
snip_driver net
set_abstractions -construct {mult=16 count=4}
report_abstractions
compute_reduced_constraints -property property
report_reduced_constraints
get_reduced_constraints
compute_formal_core -property property
report_formal_core
get_formal_core
fvtask -create task
fvtask new_task -copy task
fvtask new_task -copy task [-assumes <list>] [-asserts <list>] [-covers <list>]
↪  [-constants <list>] [-snips <list>] [-changeats<list>] [-keep_status]
↪  [-keep_task]
fvtask task
get_fvtask
quit
```

9. FTA：形式测试平台分析应用

9.1 作用

FTA 用于判断注入的故障能否被形式测试平台检测到，用来衡量测试平台质量。

9.2 设计搭建与初始化

```
set_fml_appmode FTA
set_blackbox -designs module
set_blackbox -cells hier_instance
report_blackbox
fta_init
configure_fta_props
read_file -top top -sva -format verilog -vcs {-f filelist} -inject_fault all
set_app_var fml_multi_step_fta_flow true
analyze -format verilog -vcs {-f filelist}
elaborate top -sva -inject_fault all
read_faultdb -name certitude_db
save_faultdb -name certitude_db
create_clock clk -period 100
create_reset rst -sense high
```

```
set_constant testmode -value 1'b0
set_change_at -clock clk -default
sim_run -stable
sim_force signal -apply 3'b000
sim_run 3 -clk clk
sim_set_state -uninitialized -user_only -apply 0
sim_save_reset
view_trace -reset
sim_get -signals {control_reg*}
report_fv_complexity
check_fv_setup
fv_setup_config
report_fv_setup -list
```

9.3 验证与调试

```
fvassume <name> -expr {<expression>}
fvassert <name> -expr {<expression>}
fvcover <name> -expr {<expression>}
fvdisable property
fvenable property
fvassert property
fvassume property
fvcover property
set_fml_var fml_max_time 24H
set_fml_var fml_max_memory 32GB
set_fml_var fml_progress_time_limit 2H
set_grid_usage -type RSH=12
set_grid_usage -file hostfile
set_fml_var fml_vacuity_on true
set_fml_var fml_witness_on false
compute_fta -par_task FPV
check_fv
check_fv -stop
check_fv -block
check_fv -run_finish {report_fv -list}
report_fv -list
report_fv -verbose
fta_report -instance instance
get_fta_faults instance
get_activated_props fault_id
read_fta_waiver -elfiles filelist
remove_fta_waiver -elfiles filelist
save_session -session session
restore_session -session session
vcf -restore -session session
view_trace -property property
fvtrace -property property
compute_verification_summary
set_verification_summary
```

```
waive_verification_summary
report_verification_summary
cluster_fta_faults
report_fta_fault_clusters
export_fault
set_fml_var fml_fta_seq_debug true
debug_fta
debug_fta -dump_setup
debug_fta -run_user_setup
set_fml_var fml_effort default
set_fml_var fml_fta_high_effort true
set_fml_var fml_qual_fault_in_fcore true
set_fml_var fml_fta_enable_cover true
report_fml_engines
report_fml_jobs
report_fml_hosts
```

9.4 任务与退出

```
fvtask -create task
fvtask -delete task
fvtask task
quit
```

10. FuSa：功能安全应用

10.1 作用

FuSa 用于分析 controllability 和 observability，辅助计算 FMEDA 指标。

10.2 设计搭建与初始化

```
set_fml_appmode FUSA
set_blackbox -designs module
set_blackbox -cells hier_instance
report_blackbox
fusa_config -fdb_campaign fdbname
fusa_config -sff input.sff
read_file -top top -sva -format verilog -vcs {-f filelist}
analyze -format verilog -vcs {-f filelist}
elaborate top -sva
create_clock clk -period 100
create_reset rst -sense high
set_constant testmode -value 1'b0
sim_run -stable
sim_force signal -apply 3'b000
sim_run 3 -clk clk
sim_set_state -uninitialized -user_only -apply 0
```

```
sim_save_reset
view_trace -reset
sim_get -signals {control_reg*}
check_fv_setup
fv_setup_config
report_fv_setup -list
report_fv_complexity
fusa_observation -add obs_point
fusa_detection -add det_point
```

10.3 额外黑盒与检查生成

```
fusa_blackbox -all -all_auto_path -seq_path
fusa_generate
```

10.4 验证与报告

```
fvassume <name> -expr {<expression>}
fvassert <name> -expr {<expression>}
fvcover <name> -expr {<expression>}
fvdisable property
fvenable property
set_fml_var fml_max_time 24H
set_fml_var fml_max_memory 32GB
set_fml_var fml_progress_time_limit 2H
set_grid_usage -type RSH=10
set_grid_usage -file hostfile
set_fml_var fusa_run_mode structural
check_fv
set_fml_var fusa_run_mode control
check_fv
set_fml_var fusa_run_mode observe
check_fv
check_fv -stop
check_fv -block
report_fv
report_fv -list
report_fv -verbose
fusa_report_total_faults
fusa_report
fusa_report -list
fusa_report -fdb_version
fusa_report -all_failure_modes
fusa_save
fusa_save -sff sff
save_session -session session
restore_session -session session
vcf -restore -session session
set_fml_var fml_effort default
```

```
set_fml_var fml_effort easy
set_fml_var fml_effort high
set_fml_var fml_effort bounded
set_fml_var fml_effort bug_hunting
set_fml_var fml_effort no_decompose_expensive
report_fml_engines
report_fml_jobs
report_fml_hosts
quit
```

11. FSV：形式安全验证应用

11.1 作用

FSV 用于确认安全域和非安全域之间不会发生意外数据传播，也用于发现后门和非法路径风险。

11.2 设计搭建与初始化

```
set_fml_appmode FSV
set_blackbox -designs module
set_blackbox -cells hier_instance
fsv_blackbox instance
report_blackbox
read_file -top top -sva -format verilog -vcs {-f filelist}
analyze -format verilog -vcs {-f filelist}
elaborate top -sva
create_clock clk -period 100
create_reset rst -sense high
set_constant testmode -value 1'b0
sim_run -stable
sim_force signal -apply 3'b000
sim_run 3 -clk clk
sim_set_state -uninitialized -user_only -apply 0
sim_save_reset
view_trace -reset
sim_get -signals {control_reg*}
check_fv_setup
fv_setup_config
report_fv_setup -list
report_fv_complexity
```

11.3 验证与调试

```
fsv_generate -src src_sig -dest dest_sig
fvassume <name> -expr {<expression>}
fvassert <name> -expr {<expression>}
fvcover <name> -expr {<expression>}
fvdisable property
```

```
fvenable property
fvassert property
fvassume property
fvcover property
set_fml_var fml_max_time 24H
set_fml_var fml_max_memory 32GB
set_fml_var fml_progress_time_limit 2H
set_grid_usage -type RSH=10
set_grid_usage -file hostfile
set_backup_grid_usage -file hostfile
set_fml_var fml_vacuity_on true
set_fml_var fml_witness_on false
check_fv
check_fv -stop
check_fv -block
check_fv -run_finish {report_fv -list}
fsv_report
report_fv
report_fv -list
report_fv -verbose
save_session -session session
restore_session -session session
vcf -restore -session session
view_trace -property property
view_trace -vacuity property
view_trace -witness property
set_fml_var fml_effort default
set_fml_var fml_effort high
set_fml_var fml_effort bounded
set_fml_var fml_effort bug_hunting
set_fml_var fml_effort easy
set_fml_var fml_effort no_decompose_expensive
set_fml_var fml_enable_resume true
report_fml_engines
report_fml_jobs
report_fml_hosts
snip_driver net
set_abstractions -construct {mult=16 count=4}
report_abstractions
fvtask -create task
fvtask new_task -copy task
fvtask new_task -copy task [-assumes <list>] [-asserts <list>] [-covers <list>]
↪  [-constants <list>] [-snips <list>] [-changeats<list>] [-keep_status]
↪  [-keep_task]
fvtask task
get_fvtask
quit
```

12. DPV：数据路径验证应用

12.1 作用

DPV 使用 HECTOR 技术验证数据变换块，适合在 untimed C/C++ 与 RTL 模型之间做一致性验证。

12.2 设计搭建与初始化

```
set_fml_appmode DPV
get_fml_appmode
# Quick Reference Guide 原有写法：set_hector_comp_use_new_flow true
# User Guide §DPV 的示例：set _hector_comp_use_new_flow true
# 两种写法不一致；必须依本机帮助确认，不能两者无条件同时使用。
set_aep_selection default
list_aep_selection
set_blackbox module
set_blackbox hier_instance
ignore_functions "list_of_functions"
get_blackbox
create_design -name spec|impl -top spectop|impltop -clock clk -reset resetN
 ↪  -negReset -options "compilation_options" -lang "c|c++|verilog|sverilog"
cppan -D<defines> -I <include_directories> <C/C++ filenames>
scdtan -D<defines> -I <include_directories> <C/C++ filenames>
vcs -sverilog -pvalue+<> -f filelist
vlogan <options> <filelist>
vhdlan <options> <filelist>
compile_design spec|impl
fvassume -always -expr (impl.testmode==1'b0)
map_by_name -specphase <phase> -implphase <phase> -inputs|-outputs
report_dpv_mappings
report_fv_complexity
```

12.3 证明流程

```
set_user_assumes_lemmas_procedure "ual"
set_hector_case_splitting_procedure "case_split_strategy"
fvassume -expr "spec.opcode(1)==3'd3"
fvassert -expr "impl.vld(7) |-> (spec.out(1)==impl.res(7))"
fvcover -expr "impl.res(3)==4'd6"
fvdelete <propName>
caseAssume (spec.mult(1)==2'd0)
set_hector_rew_use_dps_engine true
set_hector_rew_dps_solve_script __hector_orch_custom_dps2
set_hector_rew_dps_resource_limit 1200
run_all_hdps_options
set_app_var fml_vacuity_on true
set_app_var fml_witness_on true
set_resource_limit 200
```

```
set_hector_task_timeout 1000
set_host_file hostfile
check_fv
solveNB proofName
solveNB_init proofName
check_fv -stop
check_fv -block
proofwait
```

12.4 报告与调试

```
listproofs
listtask
listassumes
listlemmas
listcovers
save_session
restore_session
vcf -restore -session session
view_trace -property propertyname
simcex propertyname -print -joint_compile
simcex propertyname -gdb -print
view_trace -vacuity property
view_trace -witness property
set_app_var dpv_dynamic_orch true
set_app_var dpv_do_parallel_proof_size 4
set_app_var dpv_do_ignore_script true
set_custom_solve_script "orch_abo_sat"
set_hector_multiple_solve_scripts true
set_hector_multiple_solve_scripts true
set_hector_multiple_solve_scripts_list [list scripts]
show_all_solve_scripts
get_all_solve_scripts
report_fml_engines
report_fml_jobs
set_cutpoint impltop.partial
cutpoint cutname = impl.partial(3)
conflictcore
fvtask -create task
fvtask -delete task
fvtask task
quit
```

12.5 输入格式

- C/C++/SystemC 文件
- RTL 文件
- SVA 文件
- Tcl 文件

13. LPC：低功耗连通性检查应用

13.1 作用

LPC 用于检查电源意图和低功耗连接是否符合功能要求，常与 CC 和 SEQ 配合使用。

13.2 设计搭建与初始化

```
set_fml_appmode CC
fvlp_instrument -type "iso pnm"
set_fml_var fvlp_enable_pst_constraint true
set_blackbox -designs module
set_blackbox -cells hier_instance
report_blackbox
set_fml_var fml_cc_autobbox false
read_file -top top -format verilog -vcs {-f filelist}
analyze -format verilog -vcs {-f filelist}
elaborate top
read_upf <upf file>
create_clock clk -period 100
create_reset rst -sense high
set_constant testmode -value 1'b0
get_fvlp_free_supply_nets
fvlp_set_supply_aon <supply name>
fvlp_set_supply_onoff_net -netname <ctrl sig> <supply name>
flp_cc_vacuity_gen_split <default for all vacuous property/property name>
flp_cc_vacuity_report <default for all vacuous properties/property name>
```

13.3 验证与调试

```
fvdisable prop_name
fvenable prop_name
set_fml_var fml_max_time 24H
set_fml_var fml_max_memory 32GB
set_fml_var fml_progress_time_limit 2H
set_grid_usage -type RSH=10
set_grid_usage -file hostfile
set_fml_var fvlp_enable_vacuity_creation true
set_fml_var fvlp_enable_witness_creation true
check_fv
check_fv -stop
check_fv -block
check_fv -run_finish {report_fv -list}
report_load_cc
report_fv
report_fv -list
report_fv -verbose
report_fv -formatCC csv|tcl|path
save_session -session session
```

```
restore_session -session session
vcf -restore -session session
view_trace -property prop_name
view_schematic -prop prop_name
fvlp_cc_print_path prop_name
report_trace_paths [get_trace_paths -from <src sig> -to <dest sig>]
fvlp_cc_trace_fanin -dest <dest signal> -prop <prop_name>
set_cc_alias <alias name> <RTL signal name>
report_cc_alias
delete_cc_alias <alias name>
report_fv_setup -list
add_cc -src signal -dest signal -lpa_type <type>
load_cc_set_param param_name "%<value>%"
load_cc -format csv csv file
flp_check_clamp_value -lpa_type <type> -location <self/parent> -clamp_value <0/1/
↪  latch> -enable <enable signal> <port/pin name>
report_fv_complexity
sim_save_reset
check_fv_setup
fv_setup_config
report_fml_engines
report_fml_jobs
report_fml_hosts
fvtask -create task
fvtask new_task -copy task
fvtask new_task -copy task [-assumes <list>] [-asserts <list>] [-covers <list>]
↪  [-constants <list>] [-snips <list>] [-changeats<list>] [-keep_status]
↪  [-keep_task]
fvtask task
get_fvtask
quit
```

14. 一页版速查

如果你只想快速记住核心差异，可以按下面的方式理解：

- FPV 是基础模式，其他多数应用都继承它的属性管理、资源配置、任务、trace、session 和退出流程。
- AEP 重点在属性自动提取和 FSM fairness。
- CC 重点在别名、连接定义和连通性提取。
- FCA 重点在 coverage DB、覆盖签收和 Auto Scale。
- FXP 重点在 X 注入、X 去除和根因分析。
- SEQ 重点在双设计映射和等价收敛。
- FRV 重点在寄存器描述文件和 register-class 报告。
- FTA 重点在故障注入、fault DB、fault clustering 和 advanced debugging。
- FuSa 重点在 fault list、检测点和安全度量报告。
- FSV 重点在源/目的路径上的安全传播检查。
- DPV 重点在 proof procedure、lemmas、HDPS 和动态编排。

- LPC 重点在 UPF、电源供给和低功耗连通性。